SPARX: An Open-Source, Automated, Programmatically Generated, Frequency-Scalable Six-Port Receiver in 130-nm CMOS

Open in Colab

Authors: Simon Dorrer*, David Kellerer-Pirklbauer*, and Harald Pretl

*S. Dorrer and D. Kellerer-Pirklbauer contributed equally to this work, see the author contributions at the end of the notebook.

Affiliation: Institute for Integrated Circuits and Quantum Computing, Johannes Kepler University (JKU), Linz, Austria

Email: {simon.dorrer, david.kellerer-pirklbauer, harald.pretl}@jku.at

Paper: S. Dorrer, D. Kellerer-Pirklbauer, G. Zachl, and H. Pretl, "SPARX: An Open-Source, Automated, Programmatically Generated, Frequency-Scalable Six-Port Receiver in 130-nm CMOS," accepted and presented at the 2026 IEEE Nordic Circuits and Systems Conference (NorCAS), 2026, not yet available in IEEE Xplore [1].

Repository: S. Dorrer, D. Kellerer-Pirklbauer, and H. Pretl, "GitHub Repository of SPARX: An Open-Source, Automated, Programmatically Generated, Frequency-Scalable Six-Port Receiver in 130-nm CMOS," 2026, https://github.com/iic-jku/SG13CMOS_SPARX, doi: 10.5281/zenodo.19654232 [2].

Documentation: https://iic-jku.github.io/SG13CMOS_SPARX/

License: Solderpad Hardware License v2.1 (`Apache-2.0 WITH SHL-2.1`), the license of the SPARX repository.

Keywords: CMOS-only, EDA, frequency-scalable, IHP Open-PDK, large-signal noise, mm-wave, open-source, programmatic layout, RFIC, Schottky barrier diode, six-port receiver.

Abstract

This notebook generates, verifies, and simulates SPARX, a frequency-scalable mm-wave six-port receiver in the 130-nm IHP Open-PDK that uses CMOS devices only. One Python program on GDSFactory derives the complete chip layout from a single input, the design frequency, with closed-form models for the line widths and lengths instead of optimization loops. At 160 GHz, the design frequency of the chip taped out in March 2026, the notebook builds the layout and checks it with the KLayout design rule check of the IHP Open-PDK. It then fits the EM simulation of the passive six-port core with snp2le into a passive lumped-element netlist that carries the thermal noise of the core, and it characterizes the Schottky-diode power detector and the post-layout receiver in VACASK with harmonic balance, shooting periodic steady state, their small-signal and noise extensions, and transient noise. The detector reaches a responsivity of 41 V/W and a double-sideband noise figure of 42 dB at a 2 GHz intermediate frequency. The receiver reproduces the I/Q imbalance of the detector-

loaded EM core within 0.1 dB and 0.1°, and its noise figure, simulated through the noisy core model, is 48.5 dB at the I output and 46.3 dB at the Q output. All tools are open source and installed by the notebook itself, and a complete run in Google Colab takes one to two hours.

Reading Guide

This notebook is saved with the outputs of a complete run, so it can be read without running it. The PDF export next to it shows the same content. For an overview in two minutes, watch the video in Section 1.2, look at the finished chip at the end of Section 3.10, and read the results below. Section 6 summarizes them.

Section	Step	Result
3	layout generation	complete chip from the design frequency `f` alone, every RF dimension from closed-form models
4.2	full top-level KLayout DRC (`regular`, all rules)	no violations apart from the 33 antenna violations that IHP waived for the tapeout
5.2	snp2le fit of the 7-port EM result of the core	passive lumped-element netlist with thermal noise, all 49 S-parameters within 0.45 dB and 9.9° at 160 GHz
5.3	detector transfer curve, HB and PSS	responsivity of 41.2 V/W, 1 dB compression point at -2.7 dBm
5.4	detector noise, HBNOISE and PNOISE	DSB noise figure of 42.2 dB at a 2 GHz IF, 2.2 dB above the small-signal noise at the dc operating point
5.5	detector transient noise	within 1.4 dB of HBNOISE
5.6	receiver with post-layout detectors	I/Q amplitude imbalance of 2.0 dB and phase difference of 85.4°, predicted by the detector-loaded EM core within 0.1 dB and 0.1°
5.7	receiver noise figure	48.5 dB at the I output and 46.3 dB at the Q output

To run the notebook, open it in Colab with the badge at the top, choose the run mode in the next cell, and select *Runtime → Run all*. It installs every tool itself, see Section 2.

> **Run mode:** With `FULL_RUN = 1` in the next cell, the default, *Runtime → Run all* runs every cell and regenerates all outputs. This takes one to two hours on Google Colab, so the notebook can run while you read it. With `FULL_RUN = 0`, the time-intensive cells are skipped and the notebook finishes in about 5 minutes. The skipped cells are:
>
> - Section 4.2: the KLayout DRC and its violation count
> - Section 5.5: the transient noise of the detector
> - Section 5.6: the receiver with post-layout detectors
> - Section 5.7: the noise figure of the receiver
>
> A skipped cell does not regenerate its output. Running a cell clears its stored output, so the results of the skipped cells remain visible only in this notebook before it runs and in the PDF export next to it.

```
FULL_RUN = 1  # 1 = run every cell, 0 = skip the time-intensive cells listed above
```

1 Introduction

The six-port junction was first proposed for power measurement [3] and later used as a direct-conversion communications receiver [4]. At mm-wave frequencies, a passive front-end outperforms an active one in bandwidth, linearity, and frequency scalability [5], and fully integrated six-port receivers have reached the D-band. The closest precedent of this work is the first fully integrated six-port receiver above 100 GHz [6], with HBT power detectors, a low-noise amplifier, and a voltage-controlled oscillator in the 130-nm IHP SiGe BiCMOS technology. All published mm-wave six-port layouts known to the authors were drawn by hand, so every new frequency band costs weeks of layout work, and open-source RFIC flows on the IHP Open-PDK have so far been demonstrated on hand-drawn layouts only [7].

SPARX, the **S**ix-**P**ort **A**utomated Receiver (**RX**), addresses both points. Its layout is generated by Python code from the design frequency, with custom RF devices in a GDSFactory add-on for the IHP Open-PDK, and its four power detectors use Schottky barrier diodes (SBDs) instead of HBTs, so the chip needs CMOS devices only. A 160 GHz receiver of 1 mm × 1.4 mm was taped out on an IHP open MPW run in March 2026. The design and its flow are described in the NorCAS 2026 paper [1] that this notebook accompanies, and the complete flow is in the SPARX repository [2]. To the authors' knowledge, SPARX is also the first RF building block characterized with harmonic balance and periodic steady-state analyses, including their small-signal and noise extensions, using only open-source tools [1].

This notebook reproduces the main steps of that flow in Google Colab:

1. Section 2 describes the open-source design flow and installs the tools.
2. Section 3 generates the complete layout from the design frequency (`f`): the six-port core of three branch-line couplers (BLCs), a Wilkinson power divider (WPD), and a hairpin coupled-line bandpass filter (BPF), the four SBD power detectors, the pads, the seal ring, and the metal fill.
3. Section 4 checks the chip with the KLayout design rule check (DRC) of the IHP Open-PDK.
4. Section 5 converts the EM result of the core into a lumped-element netlist with snp2le, characterizes the power detector, and simulates the post-layout receiver and its noise figure in VACASK.
5. Section 6 concludes the notebook.

1.1 Six-Port Receiver Principle

The six-port receiver performs complex down-conversion by additive mixing [4]. The LO and RF signals on the ports P_1 and P_2 are superimposed in the linear, passive six-port core under four relative phases of 0, $\pi/2$, π, and $3\pi/2$. The four resulting waveforms drive four power detectors, whose square-law nonlinearity produces two differential in-phase and quadrature pairs at the outputs $V_{\mathrm{I}+}$, $V_{\mathrm{I}-}$, $V_{\mathrm{Q}+}$, and $V_{\mathrm{Q}-}$. The baseband signal follows by subtraction, $V_{\mathrm{I}} = V_{\mathrm{I}+} - V_{\mathrm{I}-}$ and $V_{\mathrm{Q}} = V_{\mathrm{Q}+} - V_{\mathrm{Q}-}$, after a calibration of the static imbalances of the network [8]. With $f_{\mathrm{RF}} = f_{\mathrm{LO}}$ the receiver converts directly to baseband (homodyne). With an LO offset, the same network works as a heterodyne receiver with a low intermediate frequency (IF) [9]. The simulations of Section 5 use this mode, with the LO at 159 GHz and the RF at 161 GHz, which gives a 2 GHz IF.

The next cell downloads the block diagram and the logo files that the layout uses from the SPARX repository and shows the block diagram: the six-port core with the BPF and the WPD in the LO path, the three BLCs, the 52 Ω termination of the RF coupler, and the four power detectors.

```
import os
```

```
import shutil
import time
import urllib.request

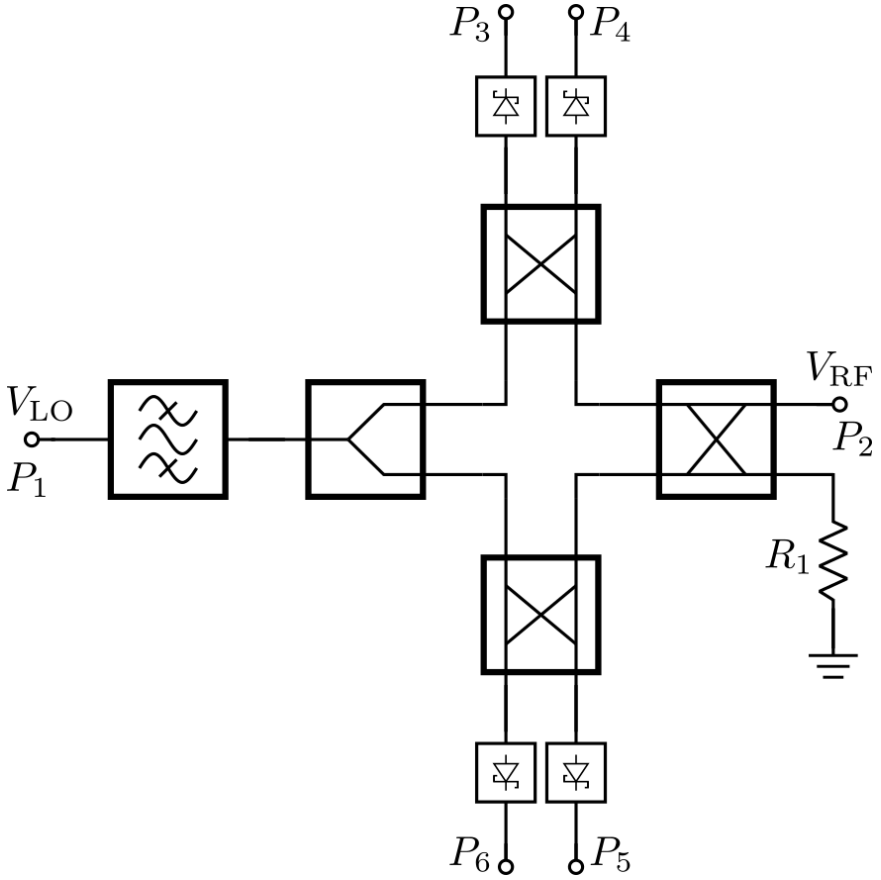
# the runtime of the whole notebook is measured from here to the last cell
NOTEBOOK_START = time.time()
print(f"Notebook started at {time.strftime('%Y-%m-%d %H:%M:%S %Z')}")

# branch of the SPARX repository the assets (logos, block diagram, video) are taken from
SPARX_BRANCH = "next_release"
ASSET_URL    = f"https://raw.githubusercontent.com/iic-jku/SG13CMOS_SPARX/{SPARX_BRANCH}/sscs-ose-code-a-chip/assets"
ASSET_DIR    = "/tmp/assets"

# download the block diagram and the logo GDS files, the video is streamed below
shutil.rmtree(ASSET_DIR, ignore_errors=True)
os.makedirs(ASSET_DIR)
for asset in ["sparx_blockdiagram.png", "jku_logo_m5.gds", "IWS.gds", "D_Kellerer.gds", "supervisors.gds"]:
    urllib.request.urlretrieve(f"{ASSET_URL}/{asset}", f"{ASSET_DIR}/{asset}")

from IPython.display import Image, display
display(Image(filename=f"{ASSET_DIR}/sparx_blockdiagram.png", width=576))
```

```
Notebook started at 2026-10-30 21:11:04 UTC
```

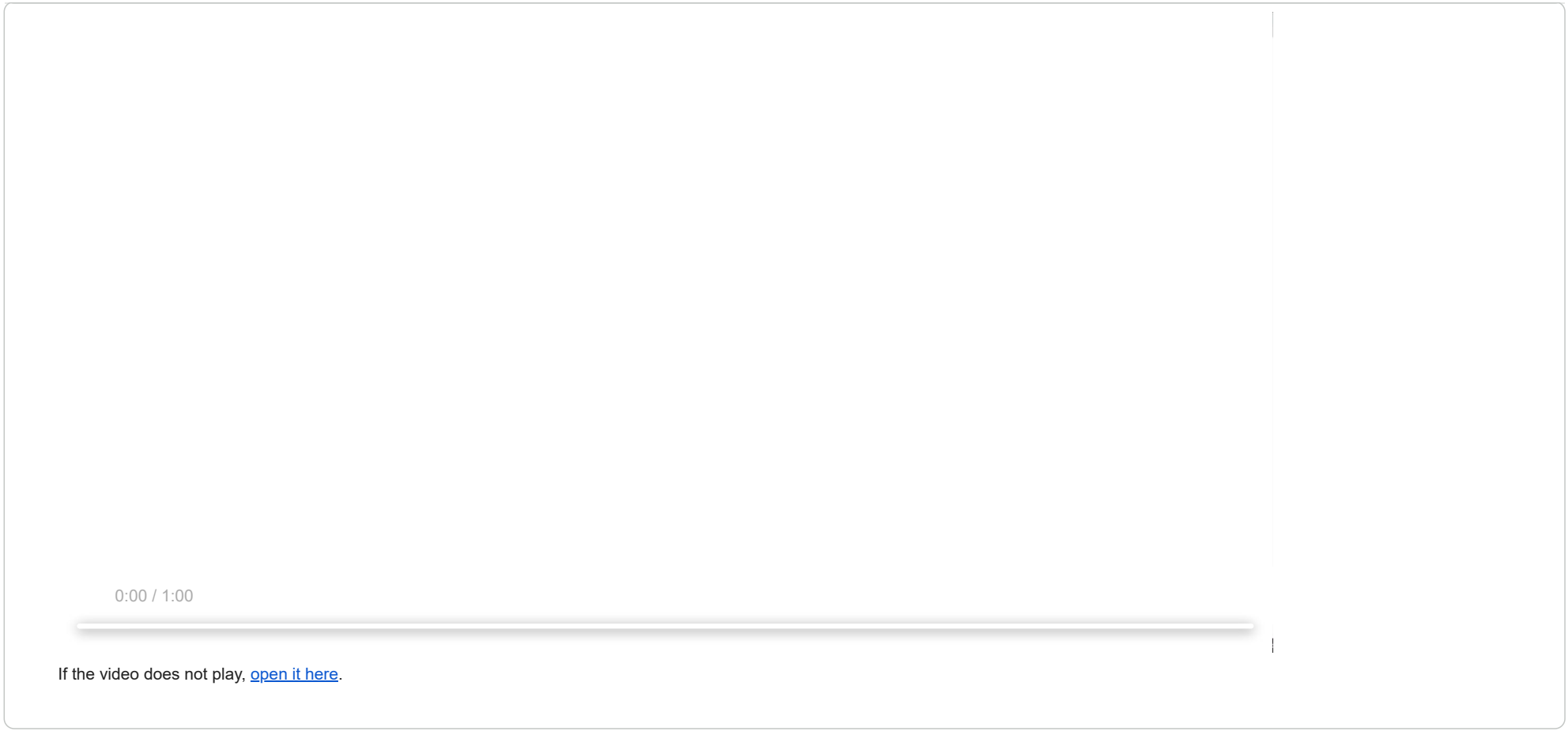


1.2 Video Demonstration

The video shows the generator building thirteen six-port receivers from 60 GHz to 300 GHz in 20 GHz steps, which takes less than one minute.

```
from IPython.display import HTML, display

# the video is streamed from GitHub instead of embedded, which keeps the notebook small enough for Colab to render
video_url = f"{ASSET_URL}/sparx_gen_60ghz_to_300ghz.mp4"
display(HTML(
    f'<video src="{video_url}" width="1024" controls muted playsinline></video>'
    f'<p>If the video does not play, <a href="{video_url}" target="_blank">open it here</a>.</p>'
))
```



If the video does not play, open it here.

2 Open-Source Design Flow and Setup

SPARX is designed with open-source tools only. In the repository [2], the schematics are drawn in Xschem and simulated with ngspice and VACASK [10]. The layout comes from the Python generator `scripts/six_port_gen.py` on GDSFactory [11] with the IHP add-on [12]. KLayout and Magic with Netgen run DRC, LVS, and parasitic extraction, gds2palace [13] converts the passive structures into AWS Palace models for the EM simulation, and snp2le [14] turns the resulting S-parameters into netlists for both circuit simulators. All tools and the IHP Open-PDK [15] run in the IIC-OSIC-TOOLS Docker image [16], and one Makefile drives the flow.

This notebook runs the parts of the flow that fit into a Colab session:

Step	Tools	In this notebook
layout generation	GDSFactory, IHP add-on	complete chip at the frequency `f`
DRC	KLayout	complete chip

Step	Tools	In this notebook
LVS, parasitic extraction	KLayout, Magic, Netgen	repository only, the post-layout netlist of the detector is reused
EM simulation	gds2palace, AWS Palace	repository only, the 160 GHz result of the core is reused
lumped-element model	snp2le	fit of the six-port core
circuit simulation	VACASK	detector and receiver testbenches

The notebook fetches every tool at the version it was tested with, so a later run uses the same tools:

Tool	Version	Section
IHP Open-PDK	commit `6f788ad` of the `dev` branch	2.2
GDSFactory add-on for the IHP Open-PDK	commit `4e5b296` of the `main` branch	2.1
GDSFactory, KFactory, doroutes	9.51.0, 3.0.4, 0.5.0	2.1
SciPy, schemdraw, ziamath	1.18.1, 0.23, 0.13	2.1
KLayout (Python package and application)	0.30.12	2.1, 4.1
snp2le, scikit-rf	0.1.12, 2.1.0	5.2
VACASK, OpenVAF-Reloaded	VACASK `fb8cc20`, both from the IIC-OSIC-TOOLS 2026.10 image	5.1

> **Running on Google Colab:** Open the notebook with the badge at the top and select *Runtime → Run all*. The notebook installs every tool itself. If Colab asks to restart the session after the package installation, confirm and select *Runtime → Run all* again. A complete run takes one to two hours on the free tier, more than half of it in the receiver noise figure of Section 5.7. `FULL_RUN = 0` at the top skips the time-intensive cells, and every simulation cell has its own `RUN_...` flag at its top.

> **Running locally:** For the full flow, install the IIC-OSIC-TOOLS container (tag 2026.10 or later), clone the repository, and run `make all` to build and verify the receiver at 160 GHz, or `make build-layout FREQ=<GHz>` to generate the layout at another frequency.

2.1 GDSFactory Add-On for the IHP Open-PDK

The add-on [12] wraps the foundry PCells and the layer stack of the IHP Open-PDK for GDSFactory and adds a library of RF devices, among them transmission lines and corners, the branch-line coupler, the Wilkinson power divider, and the hairpin coupled-line bandpass filter used here. Because it wraps the foundry PCells instead of copying them, a layout change inside a PDK cell needs no update of the add-on, only a change of the cell's interface does.

The next cell clones the add-on at the commit `IHP_COMMIT` and installs it with pip, which also installs its Python dependencies. GDSFactory and the other packages are pinned to the versions this notebook was tested with. To try a newer add-on, replace the commit hash.

```
PDK_ROOT = "/tmp/pdks"  # IHP-Open-PDK checkout
IHP_DIR  = "/tmp/ihp"   # IHP-Open-PDK GDSFactory add-on checkout

# the commits this notebook was tested with, of the add-on main branch and the IHP-Open-PDK dev branch
IHP_COMMIT = "4e5b2960f4b40507bb0ac54e8de3770517f291d3"
```

```
PDK_COMMIT = "6f788ad1f64130ced1d8849f5e4a84d7803576c8"

# the add-on reads PDK_ROOT to find the KLayout PCells of the IHP-Open-PDK
os.environ["PDK_ROOT"] = PDK_ROOT
os.environ["PDK"]      = "ihp-sg13g2"

! rm -rf {IHP_DIR} && git -c init.defaultBranch=main init -q {IHP_DIR}
! git -C {IHP_DIR} fetch -q --depth 1 https://github.com/iic-jku/IHP-GDSFactory-Addon.git {IHP_COMMIT} && git -C {IHP_DIR} checkout -q FETCH_HEAD
# the Python packages are pinned to the versions this notebook was tested with
# on Colab, kfactory 3.0 needs newer requests, toolz and cachetools than google-colab, ibis-framework and pymc declare. Only code this notebook never runs depends o
%pip install -q --no-warn-conflicts {IHP_DIR} gdsfactory==9.51.0 kfactory==3.0.4 klayout==0.30.12 doroutes==0.5.0 scipy==1.18.1 "schemdraw[svgmath]==0.23" ziamath==

# gdsfactory pulls in IPython 9 and the decorator it needs, but the Colab kernel only starts with its preinstalled IPython, so both are restored
import sys, IPython, decorator
if "google.colab" in sys.modules:
    %pip install -q --no-warn-conflicts ipython=={IPython.__version__} decorator=={decorator.__version__}
```

```
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Building wheel for ihp (pyproject.toml) ... done
  Building wheel for rectpack (pyproject.toml) ... done
```

2.2 IHP Open-PDK

The IHP Open-PDK [15] provides the technology files, the KLayout PCells, the DRC decks, and the device models. The next cell fetches it at the commit `PDK_COMMIT` of its `dev` branch, together with the two submodules that the KLayout PCells import.

```
! rm -rf {PDK_ROOT} && git -c init.defaultBranch=main init -q {PDK_ROOT}
! git -C {PDK_ROOT} fetch -q --depth 1 https://github.com/iic-jku/IHP-Open-PDK.git {PDK_COMMIT} && git -C {PDK_ROOT} checkout -q FETCH_HEAD
! git -C {PDK_ROOT} submodule update -q --init --depth 1 \
    ihp-sg13g2/libs.tech/klayout/python/pycell4klayout-api \
    ihp-sg13g2/libs.tech/klayout/python/pypreprocessor
```

2.3 Python Modules

GDSFactory builds a layout as a hierarchy of Python objects. A `gf.Component` holds polygons, ports, and references to other components, and `connect()` moves a reference so that one of its ports meets a port of another component. Every block of the chip is placed this way. `ihp.PDK.activate()` makes the layers, cross sections, and cells of the add-on the active PDK. The helper `snap_to_grid()` rounds a length down to the 5 nm grid of the PDK, so that computed lengths such as $\lambda_g/4$ cannot produce off-grid geometry.

```
import gdsfactory as gf
import ihp
from pathlib import Path
import scipy
from math import sqrt

ihp.PDK.activate()


def snap_to_grid(value: float) -> float:
    """Snap a length in um down to the nearest PDK grid point."""
    return round(value - value % ihp.tech.nm, 3)
```

3 Six-Port Receiver Layout

This section builds the complete chip as one GDSFactory component, block by block, in the order of the generator `scripts/six_port_gen.py` of the repository. Every dimension of the RF structures follows from the design frequency `f`, and every block is placed by connecting its ports to blocks that are already placed, so the layout rescales without any manual coordinates when `f` changes. The pads, the power detectors, and the seal ring margin do not scale.

The next cell creates the top-level component that collects all blocks.

```
c = gf.Component("Six-Port-CAC")
```

3.1 Design Parameters and Closed-Form Models

All passive RF structures are built from embedded microstrip lines, with the signal conductor on TopMetal2 over a ground plane on Metal5. The width w of a line sets its characteristic impedance and its length sets the phase, and both follow from closed-form models, so no optimization loop or training data is needed [1]. The width comes from the IPC-2141 surface-microstrip formula [17], solved for w,

$$Z_0 = \frac{87\ \Omega}{\sqrt{\varepsilon_\mathrm{r} + 1.41}} \ln \frac{5.98\, h_1}{0.8\, w + t},$$

and the length from the guided wavelength $\lambda_\mathrm{g} = c_0 / (f \sqrt{\varepsilon_{\mathrm{r,eff}}})$ with the embedded-microstrip approximation

$$\varepsilon_{\mathrm{r,eff}} = \varepsilon_\mathrm{r} \left[1 - \exp\left(-1.55\, \frac{h_2}{h_1} \right) \right],$$

which accounts for the dielectric above the conductor. For TopMetal2 over Metal5, the conductor thickness is $t = 3$ µm, the dielectric height between the two layers is $h_1 = 5.65$ µm, and the total dielectric height is $h_2 = 10.15$ µm, including 1.5 µm of passivation above TopMetal2. With $\varepsilon_\mathrm{r} = 4.1$, this gives at 160 GHz:

Quantity	Value at 160 GHz
effective permittivity $\varepsilon_{\mathrm{r,eff}}$	3.85
free-space wavelength λ_0	1874 µm
guided wavelength λ_g and $\lambda_\mathrm{g}/4$	955 µm and 238.8 µm
width for $Z_0/\sqrt{2} = 35.4\ \Omega$ (BLC)	12.51 µm
width for $Z_0 = 50\ \Omega$ (BLC, interconnects)	7.20 µm
width for $\sqrt{2}\,Z_0 = 70.7\ \Omega$ (WPD)	2.51 µm

The widths are truncated to 10 nm and the lengths to the 5 nm grid. An EM simulation of lines with these widths gives 39.4 Ω, 51.1 Ω, and 74.1 Ω, which is 11.5 %, 2.2 %, and 4.7 % above the targets, and the 50 Ω line loses 1.26 dB/mm at 160 GHz [1]. IPC-2141 states the formula for $0.1 \le w/h_1 \le 2$, and the 12.51 µm line lies just outside that range, at $w/h_1 = 2.2$. For stack-ups with $h_1 < t$, such as TopMetal2 over TopMetal1, the formula is off by more than 14 % at every width, which is why Metal5 is the ground plane.

The parameters of the next cell set the whole RF layout:

Parameter	Meaning	Value
`f`	design frequency, every RF dimension scales with it	160 GHz
`e_r`, `Z0`	relative permittivity of the back-end oxide, reference impedance	4.1, 50 Ω
`signal_cross_section`, `ground_cross_section`	signal and ground layer of every line	TopMetal2, Metal5
`order`, `filter_type`, `ripple_dB`	BPF order, prototype (`"butter"` or `"cheby"`), and Chebyshev ripple	3, `"butter"`, 3 dB
`bandwidth`	absolute bandwidth handed to the BPF synthesis	1 GHz
`wpd_shape`	routing shape of the WPD arms, `"C"` or `"U"`	`"U"`
`connection_length_bpf`, `connection_length_wpd`, `connection_length_blc`	line extensions at the ports of the BPF, the WPD, and the BLCs	10 µm, 0 µm, 0 µm
`connection_length_bpf_wpd`	line between the WPD input and the BPF	0.7 $\lambda_\mathrm{g}/4$

```
# RF Design Parameters
e_r = 4.1   # relative permittivity (from process specifications)
Z0  = 50    # characteristic impedance in Ω
f   = 160e9 # design frequency in Hz

# Transmission Line Metal Stack
signal_cross_section = "topmetal2_routing"  # signal conductor layer (TopMetal2)
ground_cross_section = "metal5_routing"     # ground conductor layer (Metal5)

# Effective Dielectric Constant & Wavelength
e_eff = ihp.cells.waveguides._calculate_effective_dielectric_constant(
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    e_r=e_r,
)

c0           = scipy.constants.c            # speed of light in vacuum in m/s
wavelength   = c0 / f * 1e6 / sqrt(e_eff)   # guided wavelength in µm
wavelength_4 = wavelength / 4               # quarter-wavelength in µm
```

```
# Snap down to the 5 nm grid of the PDK
wavelength   = snap_to_grid(wavelength)       # λ in µm
wavelength_4 = snap_to_grid(wavelength_4)     # λ/4 in µm
wavelength_8 = snap_to_grid(wavelength / 8)   # λ/8 in µm

# Bandpass Filter Parameters
order                 = 3        # filter order
bandwidth             = 1e9      # 3 dB bandwidth in Hz
filter_type           = "butter" # filter type: "butter" (Butterworth) or "cheby" (Chebyshev)
ripple_dB             = 3        # passband ripple in dB, applies to Chebyshev only
connection_length_bpf = 10       # connection length between the bandpass filter and the circuit in µm

# Wilkinson Power Divider Parameters
wpd_shape                 = "U"                                    # routing shape of the WPD: "C" or "U"
connection_length_wpd     = 0                                      # connection length at WPD ports in µm
connection_length_bpf_wpd = snap_to_grid(wavelength_4 * 3.5 / 5)   # connection length between the WPD and the bandpass filter in µm

# Branch-Line Coupler Parameters
connection_length_blc = 0  # connection length at branch-line coupler ports in µm
```

The next cell prints the effective permittivity, the guided wavelength, and the three line widths for the frequency `f` set above, so that a new `f` can be checked before the layout is built.

```
import contextlib
import io

# the quantities the RF devices below are generated from, for the frequency f set above
print(f"effective permittivity: {e_eff:.3f}")
print(f"guided wavelength: {wavelength:.2f} um, lambda/4 = {wavelength_4:.2f} um, lambda/8 = {wavelength_8:.2f} um")
for name, Z in (("Z0/sqrt(2)", Z0 / sqrt(2)), ("Z0", Z0), ("sqrt(2) Z0", Z0 * sqrt(2))):
    with contextlib.redirect_stdout(io.StringIO()):  # the add-on prints every width it calculates
        width = ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z, e_r=e_r, signal_cross_section=signal_cross_section, ground_cross_section=ground_cross_section)
    print(f"line width for {name} = {Z:.1f} Ohm: {width:.2f} um")
```

```
effective permittivity: 3.847
guided wavelength: 955.33 um, lambda/4 = 238.83 um, lambda/8 = 119.42 um
line width for Z0/sqrt(2) = 35.4 Ohm: 12.51 um
line width for Z0 = 50.0 Ohm: 7.20 um
line width for sqrt(2) Z0 = 70.7 Ohm: 2.51 um
```

3.2 Layout Constants

The device dimensions, routing widths, pad geometry, no-fill zones, and spacings below do not scale with `f`. They have the values of the constants in `scripts/six_port_gen.py` and are grouped by purpose: the MIM capacitors, transistors, and resistors of the power

detector, the routing widths of its nets, the probe pads, the Metal5 no-fill zones around the pads, and the spacings that place the detectors, the pads, and the seal ring.

```
# CMIM capacitor dimensions (um)
CMIM_SIZE = 10  # width and length of standard CMIM cap
CMIM_PITCH = 11.2  # center-to-center pitch in cap arrays
CMIM_SPACING_TM1 = 0.52  # min TM1 spacing between caps (design rule 5.17)
CMIM_FILL_WIDTH = 6.8  # fill capacitor width
CMIM_FILL_LENGTH = 9  # fill capacitor length
CAP_BUS_WIDTH = 4  # metal bus width connecting cap plates

# MOSFET output stage parameters
NUM_GATES = 24  # total gate fingers per transistor (incl. dummies)
NMOS_WIDTH = 60  # nmos finger width (um)
PMOS_WIDTH = 120  # pmos finger width (um)
GATE_LENGTH = 0.13  # drawn gate length (um)
GUARD_RING_DISTANCE = 2  # guard ring distance (um)
GATE_EXTENSION_NMOS = 2.15  # gate poly extension for nmos via connection
GATE_EXTENSION_PMOS = 3.55  # gate poly extension for pmos via connection
NMOS_PMOS_SPACING = 15  # vertical gap between nmos and pmos drain vias

# Resistor dimensions (um)
RSIL_WIDTH = 0.5  # rsil resistor width (XR1, XR5, termination)
RSIL_LENGTH = 2.5  # rsil resistor length
RPPD_WIDTH = 0.5  # rppd resistor width (XR2, XR4)
RPPD_LENGTH = 1.5  # rppd resistor length
RHIGH_WIDTH = 0.5  # rhigh resistor width (XR3)
RHIGH_LENGTH = 2  # rhigh resistor length

# Routing widths (um)
ROUTE_WIDTH_VDD = 8  # VDD power bus (TM1)
ROUTE_WIDTH_VSS = 2  # VSS ground bus (M5)
ROUTE_WIDTH_SIGNAL = 4  # signal lines to probe pads (M3/M4)
ROUTE_WIDTH_MIN_M1 = 0.26  # minimum Metal1 route width
ROUTE_WIDTH_BIAS = 0.5  # bias network routing
ROUTE_WIDTH_BIAS_M1 = 0.46  # bias Metal1 routing
ROUTE_WIDTH_GATE = 0.6  # gate connection routing (M2/M3)
ROUTE_WIDTH_REPLICA = 0.61  # replica circuit drain routing
ROUTE_WIDTH_SBD_SUB = 5.7  # Schottky diode substrate connection
ROUTE_WIDTH_SBD_GUARD = 0.3  # SBD D1-D2 guard ring connection
ROUTE_MIN_STRAIGHT = 10  # minimum straight length to avoid self-routing

# Probe pad parameters (um)
PROBE_PITCH = 125  # probe pad pitch
PROBE_GROUND_WIDTH = 120  # ground pad width (left probe)
PROBE_GROUND_WIDTH_RIGHT = 125  # ground pad width (right probe)
PROBE_SIGNAL_SIZE = 65  # signal pad width and length

# Nofill exclusion zones (um)
```

```
NOFILL_GSG_SIZE = 105  # nofill square around GSG probes
NOFILL_VDD_SIZE = 110  # nofill square around VDD pads
NOFILL_SIDE_WIDTH = 235  # side nofill width (85 + pitch + 25)
NOFILL_SIDE_OFFSET = 12.5  # side nofill centering offset ((110 - 85) / 2)

# Layout spacing (um)
RFIN_GAP = 40  # gap between BLC port and PD rfin
RFIN_PD = 18.4 # part of the connection that is part of the PD
PROBE_PD_GAP = 5  # gap between PD edge and probe edge
SEALRING_MARGIN = 50  # margin around circuit for sealring

# Six-port network geometry
CORNER_Z_FACTOR = 1.314  # impedance scaling for tline corners
CONNECTION_LEN_TERM = 30  # termination connection (um)
CONNECTION_LEN_BPF_PAD = 30  # BPF-to-pad connection (um)
CONNECTION_LEN_BLC_PAD = 40  # BLC-to-pad connection (um)
CONNECTION_LEN_BLC_EXTRA = 25  # extra pad clearance added to BLC-to-pad (um)

# Logo placement
LOGO_VERTICAL_CLEARANCE = 20  # minimal vertical clearance between logo and probe pads
```

3.3 Branch-Line Couplers

The branch-line coupler (BLC) is a 90° hybrid: a square of four $\lambda_g/4$ lines, two opposite branches with $Z_0/\sqrt{2}$ and two with Z_0 [18]. A wave entering the input port splits into two halves of -3 dB at the through and the coupled port, 90° apart, while the fourth port is isolated. The three BLCs combine the LO and RF waves under the four relative phases of the six-port.

The BLC is a cell of the add-on (`ihp.cells.branch_line_coupler`). It takes the widths for Z_0 and $Z_0/\sqrt{2}$ from the closed form above and the branch length from the effective permittivity, and it joins the branches with chamfered corners:

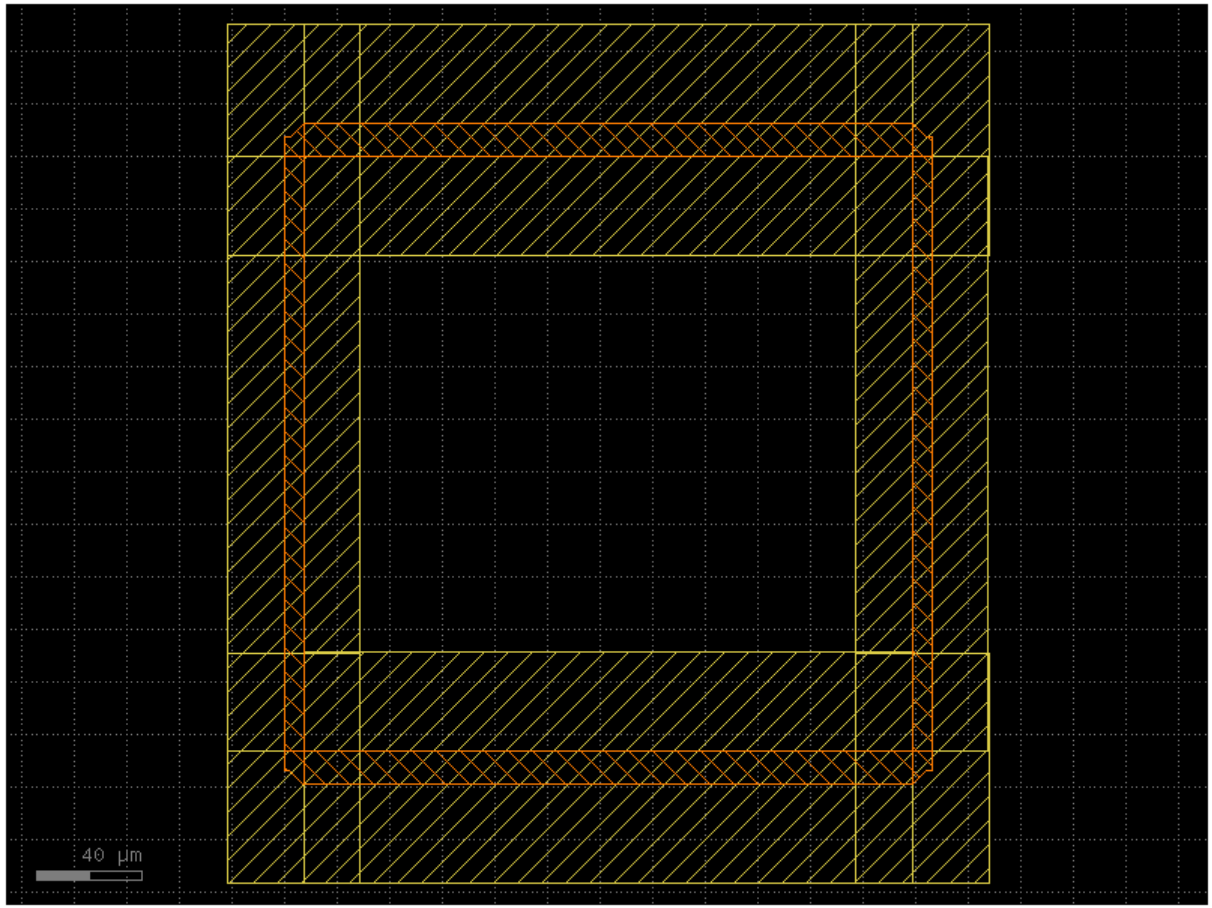
BLC at 160 GHz	Value
branches with $Z_0/\sqrt{2} = 35.4\ \Omega$	12.51 µm wide, 238.8 µm long
branches with $Z_0 = 50\ \Omega$	7.20 µm wide, 238.8 µm long
footprint	289 µm × 326 µm

Without any tuning, the EM simulation of BLCs generated for 60 GHz, 160 GHz, and 300 GHz gives an input match of -16 dB or better and an isolation of -15 dB to -17 dB at the design frequency [1]. The amplitude imbalance between the through and the coupled port grows from 1.2 dB at 60 GHz to 1.8 dB at 300 GHz, and the phase difference deviates from the ideal -90° by 1° to 6°. Imbalances of this size are acceptable for a six-port, whose calibration removes them.

The next cell generates the BLC and plots it. Three references of this cell become BLC_3, the coupler at the RF input, and the two output couplers BLC_1 and BLC_2.

```
blc = ihp.cells.branch_line_coupler(
    frequency=f,
    connection_length=connection_length_blc,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
    e_r=e_r,
)
blc.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 12.51 um for Z0: 35.36 ohm
Calculated Z0: 35.3772968531251 ohm for width: 12.51 um
Calculated Z0: 50.03094234385018 ohm for width: 7.2 um
Calculated Z0: 50.03094234385018 ohm for width: 7.2 um
```



BLC_3 is added to the top-level component first.

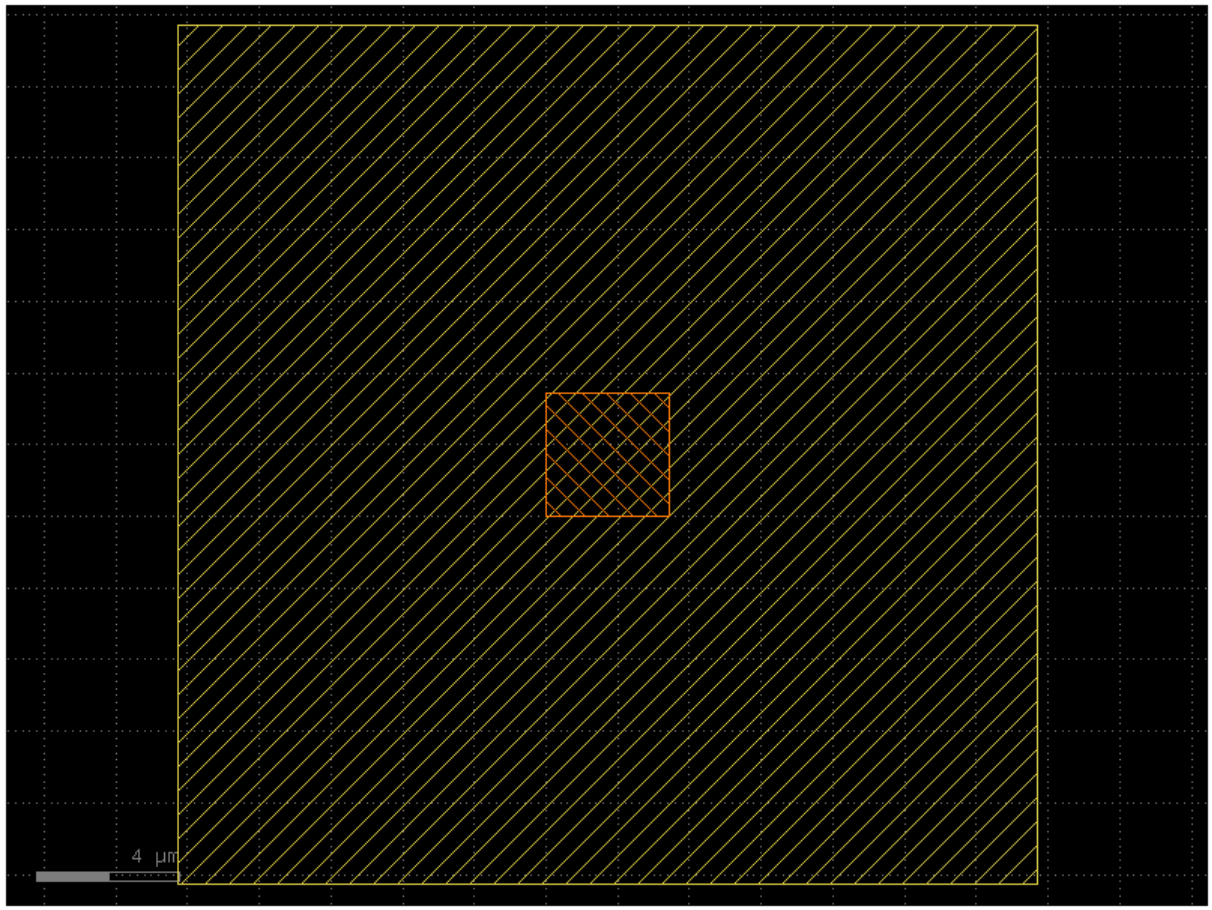
```
blc_3_ref = c.add_ref(blc)
```

The three BLCs sit corner to corner, which keeps the paths between them short and limits conductor loss and phase mismatch.
`tline_corner` joins two corner ports with a square TopMetal2 patch that has a port on each side, over an oversized Metal5 ground plate.
Its side is the width of a line with `CORNER_Z_FACTOR` × Z_0 = 65.7 Ω, which is 3.42 µm.

```
corner = ihp.cells.tline_corner(
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0 * CORNER_Z_FACTOR,
)

corner.plot()
corner_top_ref = c.add_ref(corner)
```

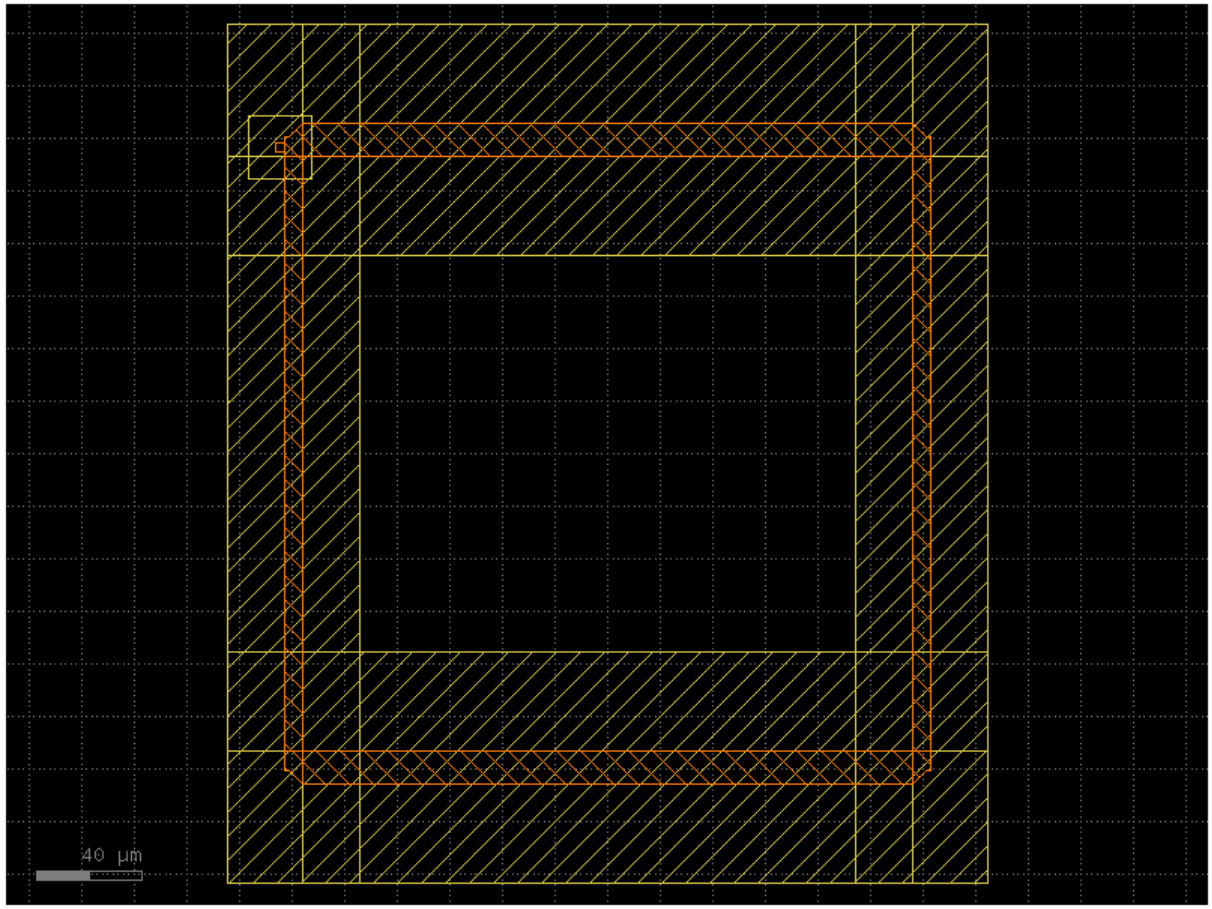
```
Calculated width: 3.42 um for Z0: 65.7 ohm
```



BLC_3 is centered at the origin, and the first corner piece is attached to its port `e1`. The corner square is narrower than the 7.2 µm port of the coupler, so the width check of `connect()` is switched off with `allow_width_mismatch=True`.

```
blc_3_ref.center = (0, 0)
corner_top_ref.connect("e1", blc_3_ref.ports["e1"], allow_width_mismatch=True)

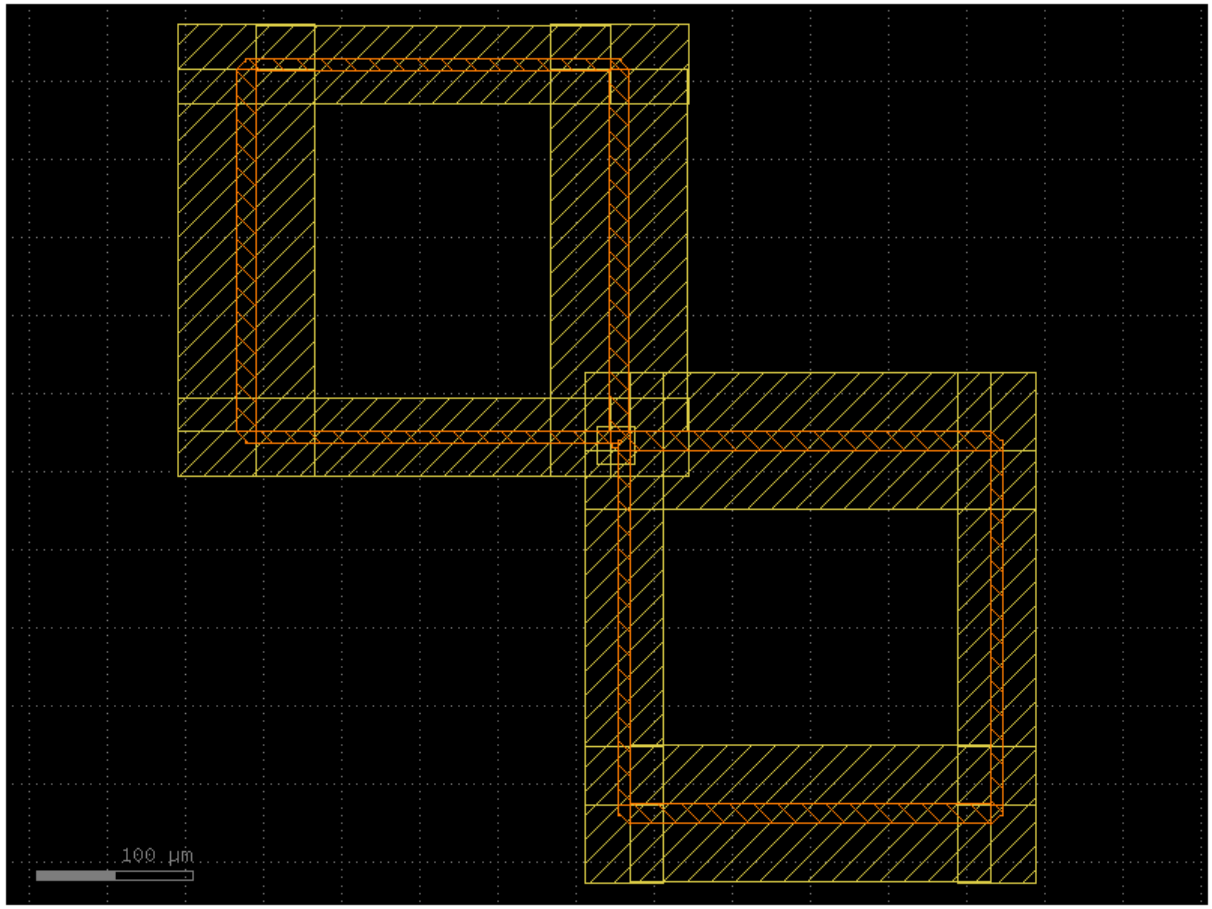
c.plot()
```



BLC_1, the upper output coupler, is attached to the other side of the corner piece through its port `e4`.

```
blc_1_ref = c.add_ref(blc)
blc_1_ref.connect("e4", corner_top_ref.ports["e4"], allow_width_mismatch=True)

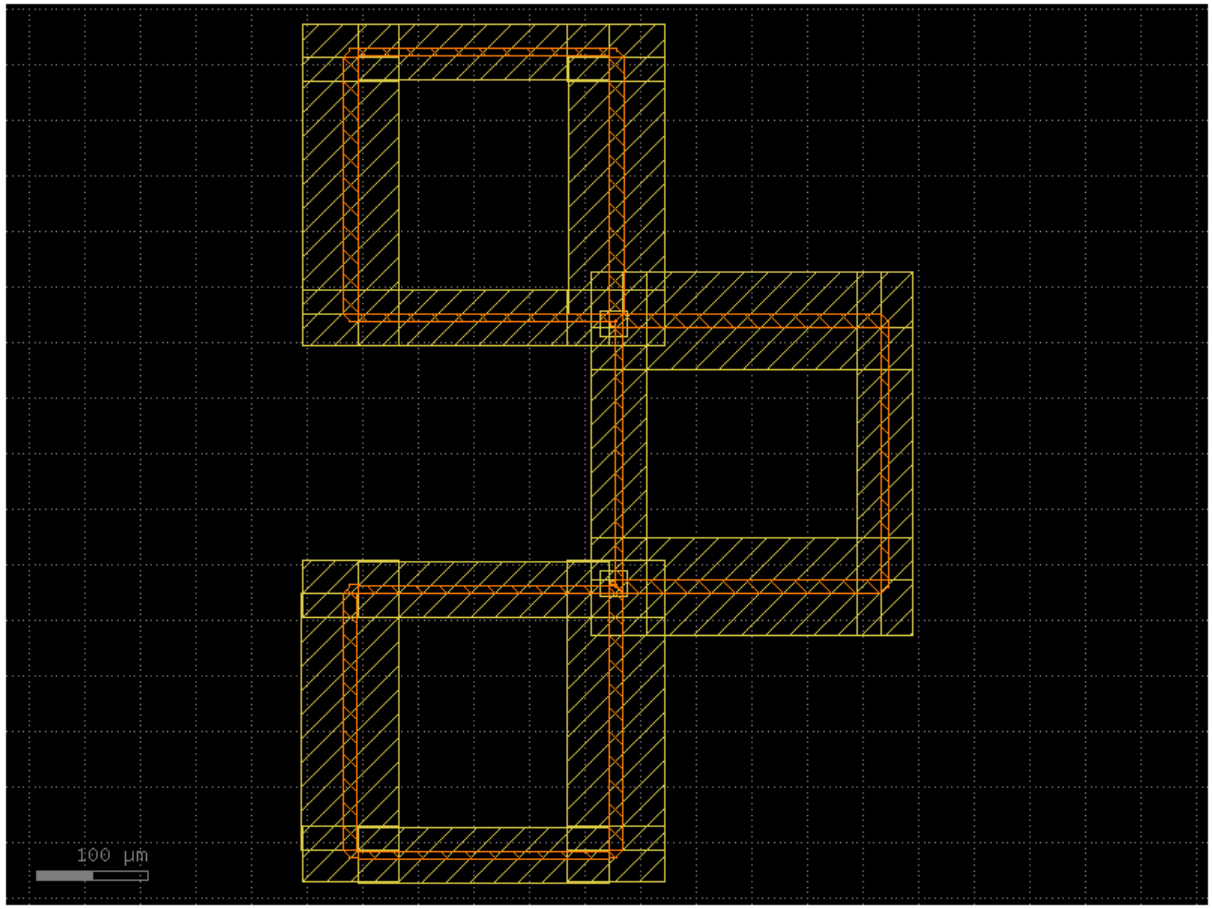
c.plot()
```



A second corner piece joins port `e4` of BLC_3 to BLC_2, the lower output coupler.

```
corner_bot_ref = c.add_ref(corner)
corner_bot_ref.connect("e1", blc_3_ref.ports["e4"], allow_width_mismatch=True)
blc_2_ref = c.add_ref(blc)
blc_2_ref.connect("e1", corner_bot_ref.ports["e2"], allow_width_mismatch=True)

c.plot()
```



3.4 Wilkinson Power Divider

The Wilkinson power divider (WPD) splits the filtered LO equally and in phase into the paths to BLC_1 and BLC_2 [19]. Each arm is a $\lambda_g/4$ line with $\sqrt{2}\,Z_0$, which transforms the 50 Ω load at its end to 100 Ω, so that the two arms in parallel present 50 Ω at the input.

A standard WPD has an isolation resistor of $2Z_0$ between its outputs, which carries current only when the outputs see different signals. In the forward-dividing LO path of the six-port, both outputs are matched, in phase, and equal in amplitude, so the resistor can be omitted. At mm-wave frequencies it also has to be: a 100 Ω resistor with its connections exceeds the $\lambda_0/10$ rule of thumb for lumped elements and would add an unintended, frequency-dependent element. The price is an output isolation of about -7 dB, close to the -6 dB of an ideal divider without resistor [1].

The WPD is a cell of the add-on (`ihp.cells.wilkinson_power_divider`) and sits in the free space between BLC_1 and BLC_2:

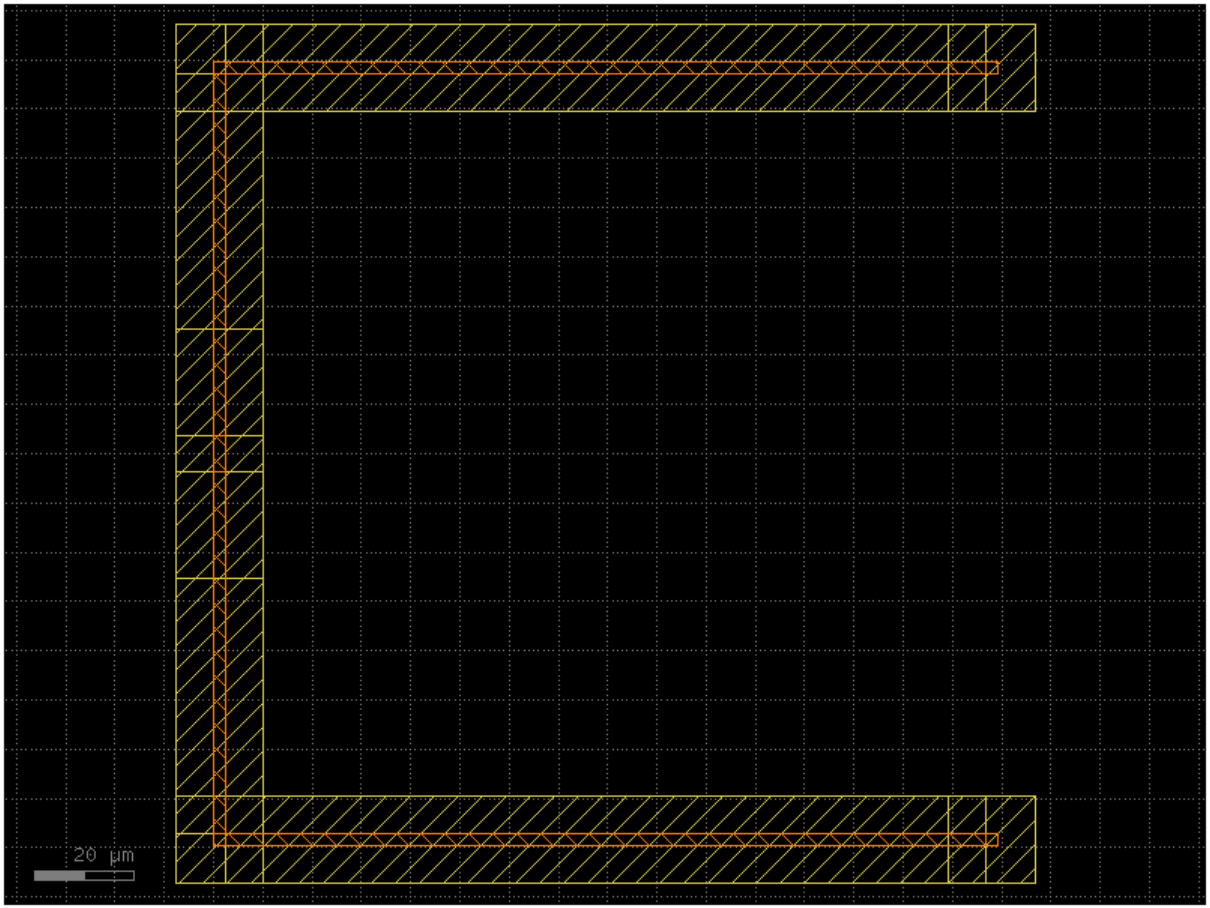
WPD at 160 GHz	Value
arms with $\sqrt{2}\,Z_0 = 70.7\ \Omega$	2.51 µm wide, 238.8 µm long
input and outputs with $Z_0 = 50\ \Omega$	7.20 µm wide
footprint with `wpd_shape = "U"`	174 µm × 174 µm

The EM simulation gives an input match below -16 dB, an excess loss of 0.3 dB to 0.7 dB above the ideal 3 dB split, and, owing to the symmetry, an amplitude and phase imbalance between the outputs within 0.02 dB and 0.1° [1].

```
wpd = ihp.cells.wilkinson_power_divider(
    frequency=f,
    connection_length=connection_length_wpd,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
    e_r=e_r,
    shape=wpd_shape,
)

wpd.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 2.5100000000000002 um for Z0: 70.71 ohm
Calculated Z0: 70.75520001248569 ohm for width: 2.51 um
Calculated width: 2.5100000000000002 um for Z0: 70.71 ohm
Calculated Z0: 70.75520001248569 ohm for width: 2.51 um
```



The input port `e1` is turned to face left, where the line from the BPF attaches, and the WPD is added to the chip.

```
wpd.ports["e1"].orientation = 0
wpd_ref = c.add_ref(wpd)
```

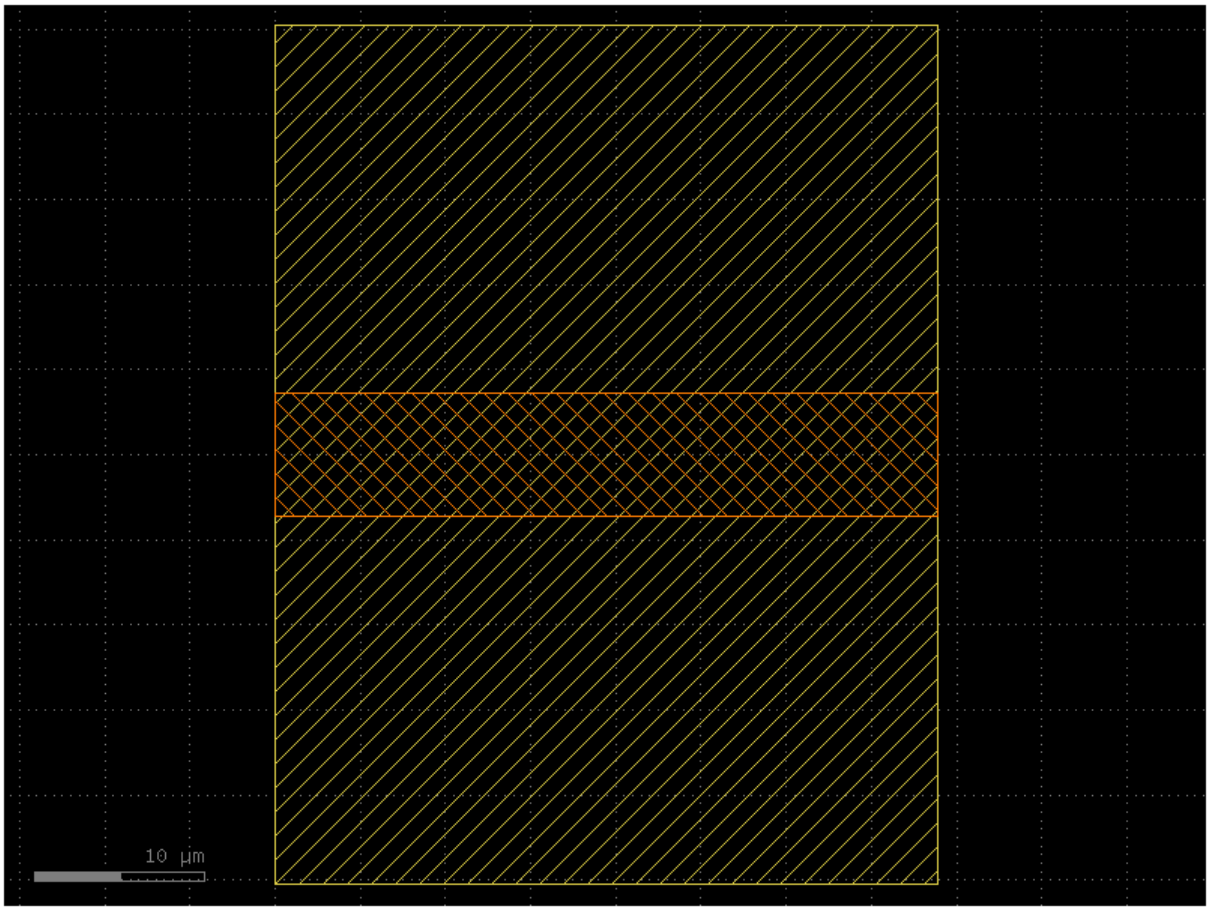
Two equal 50 Ω lines connect the WPD outputs to the inputs of BLC_1 and BLC_2. Each is half as long as the vertical distance between the two coupler ports minus the height of the WPD. One line cell serves both legs, so the two LO paths are identical.

```
connection_length_wpd_blc_one_leg = blc_1_ref.ports["e1"].center[1] - blc_2_ref.ports["e4"].center[1] - (wpd_ref.ports["e2"].center[1] - wpd_ref.ports["e3"].center[

connection_wpd_blc = ihp.cells.tline(
    length=snap_to_grid(connection_length_wpd_blc_one_leg / 2) + ihp.tech.nm,  # one grid step of margin
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
)

connection_wpd_blc.plot()
```

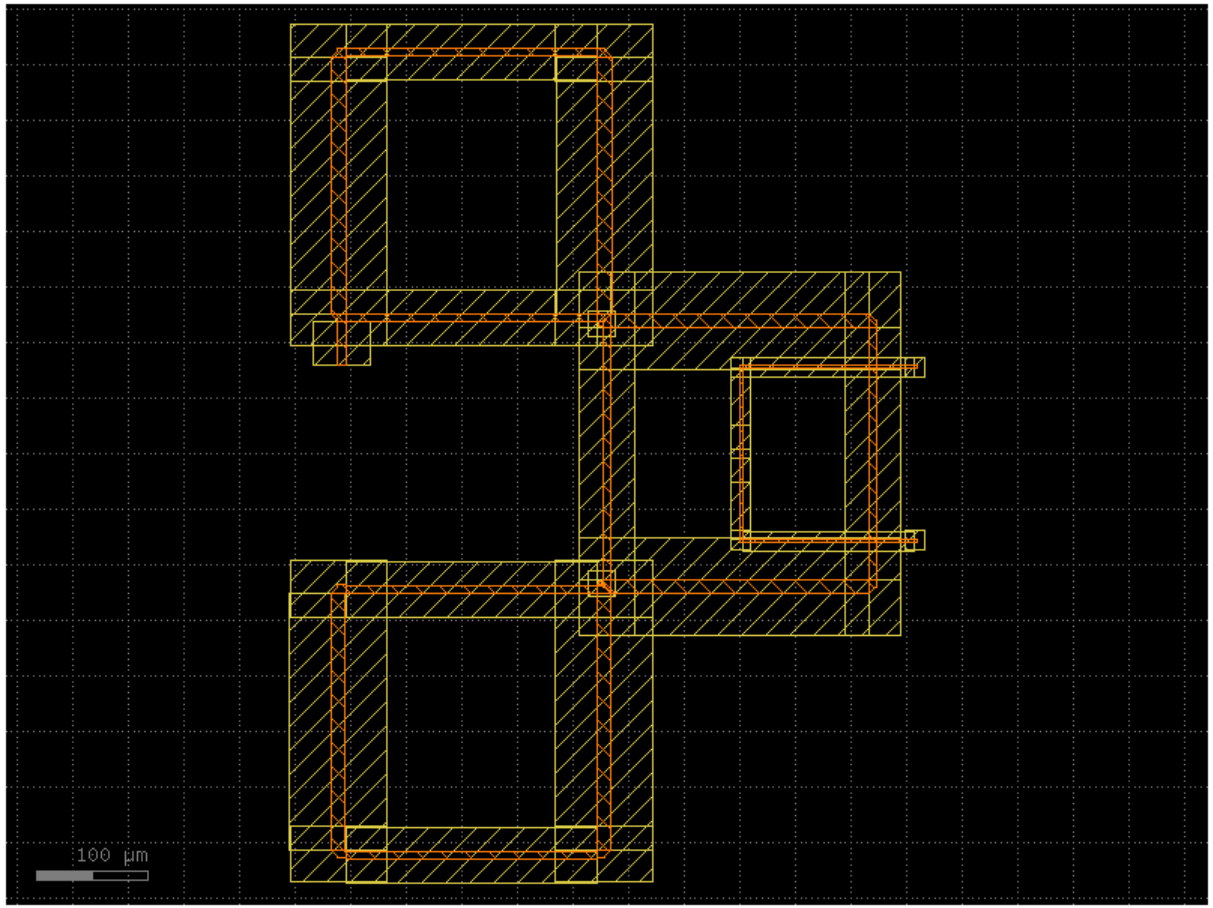
```
Calculated width: 7.2 um for Z0: 50 ohm
```



The upper leg is attached to port `e1` of BLC_1.

```
connection_wpd_blc_top_ref = c.add_ref(connection_wpd_blc)
connection_wpd_blc_top_ref.connect("e1", blc_1_ref.ports["e1"])

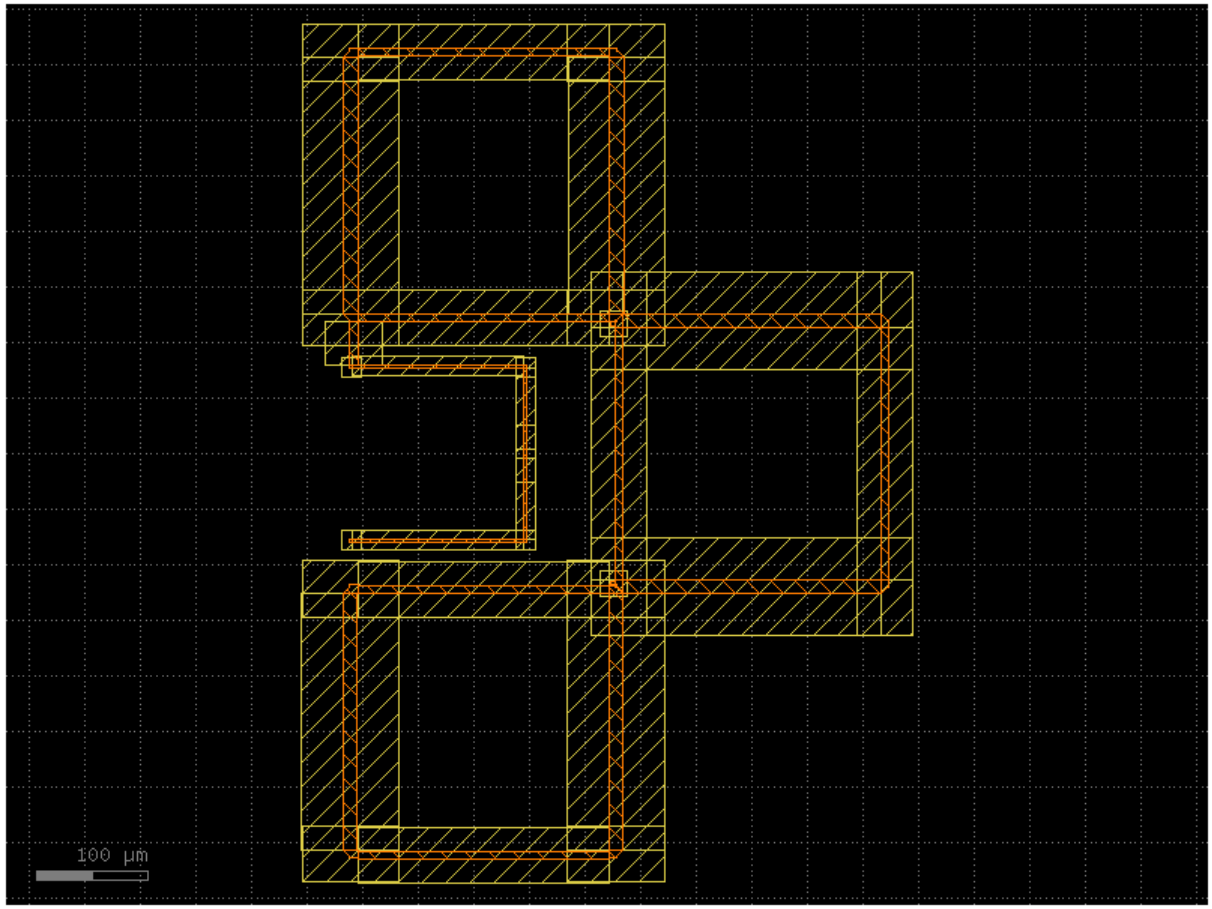
c.plot()
```



The WPD is attached with its output `e3` to the free end of the upper leg, so its position follows from the couplers and needs no coordinates.

```
wpd_ref.connect("e3", connection_wpd_blc_top_ref.ports["e2"])

c.plot()
```

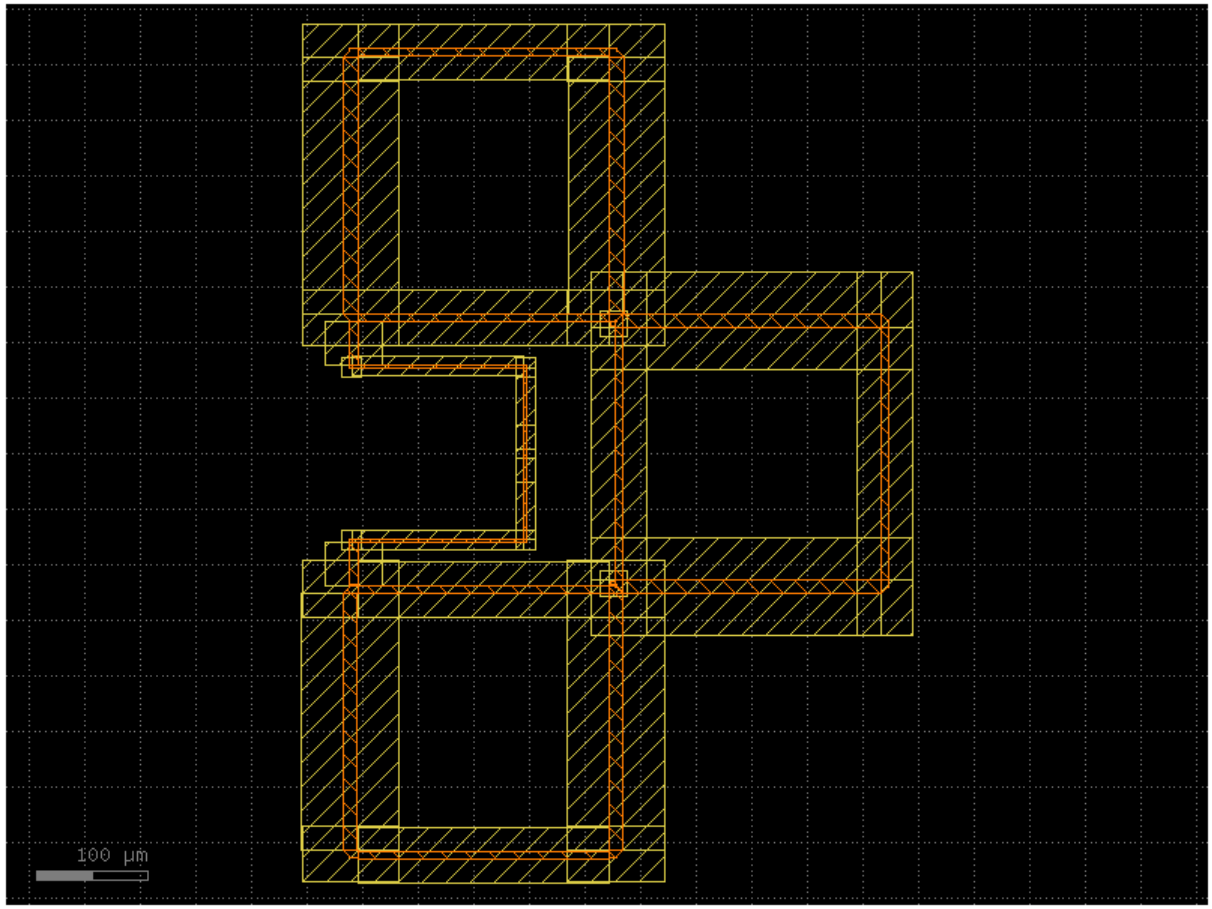


The lower leg joins port `e4` of BLC_2 to the second WPD output `e2`.

```
connection_wpd_blc_bot_ref = c.add_ref(connection_wpd_blc)
connection_wpd_blc_bot_ref.connect("e1", blc_2_ref.ports["e4"])

wpd_ref.connect("e2", connection_wpd_blc_bot_ref.ports["e2"])

c.plot()
```



3.5 Hairpin Coupled-Line Bandpass Filter

The hairpin coupled-line bandpass filter (BPF) at the LO input limits the LO to the band around `f`. It is synthesized from a Butterworth or Chebyshev prototype [20], here a third-order Butterworth filter, from coupled $\lambda_g/4$ sections that are folded into hairpins for a compact layout. It is a cell of the add-on (`ihp.cells.hairpin_coupled_line_bandpass_filter`):

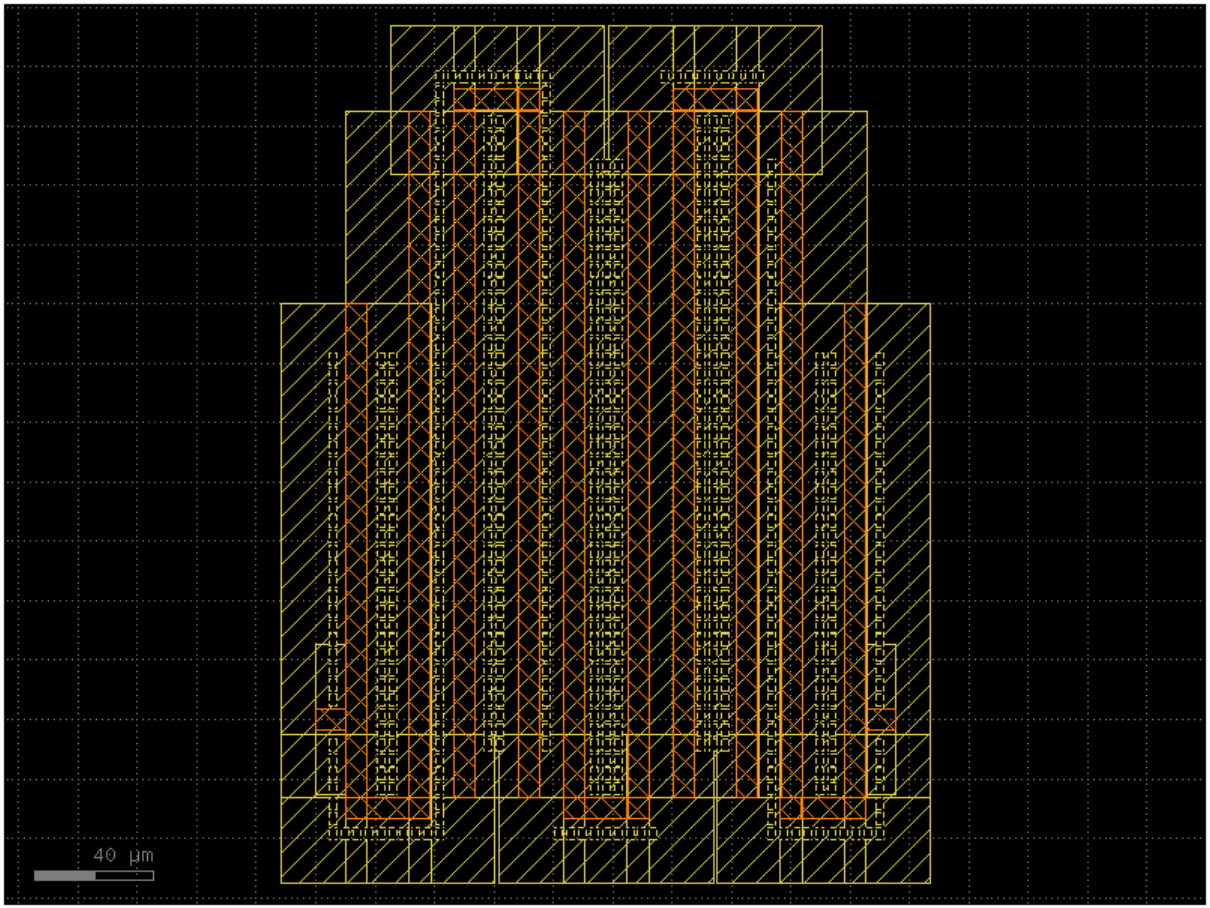
BPF at 160 GHz	Value
section length	238.8 µm
footprint	219 µm × 289 µm

The meanders and coupled lines of the hairpin filter show the limits of the closed-form models. At 160 GHz the filter loses about 10 dB in its passband and matches only to about -6 dB, so it is the block that would profit most from an EM-driven optimization [2].

```
bandpass_filter = ihp.cells.hairpin_coupled_line_bandpass_filter(
    frequency=f,
    bandwidth=bandwidth,
    order=order,
    filter_type=filter_type,
    ripple_dB=ripple_dB,
    connection_length=connection_length_bpf,
    signal_cross_section=signal_cross_section,
    ground_cross_section=ground_cross_section,
    Z0=Z0,
    e_r=e_r,
)

bandpass_filter_ref = c.add_ref(bandpass_filter)
bandpass_filter.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 7.13 um for Z0: 50.25 ohm
Calculated Z0: 50.26863690347976 ohm for width: 7.13 um
Calculated width: 7.2 um for Z0: 50 ohm
Calculated width: 7.2 um for Z0: 50.0 ohm
Calculated Z0: 50.03094234385018 ohm for width: 7.2 um
Calculated width: 7.2 um for Z0: 50 ohm
```



A 50 Ω line of length `connection_length_bpf_wpd`, 0.7 $\lambda_g/4$ or 167 µm at 160 GHz, leads from the WPD input `e1` to the left, and the BPF is attached to its end.

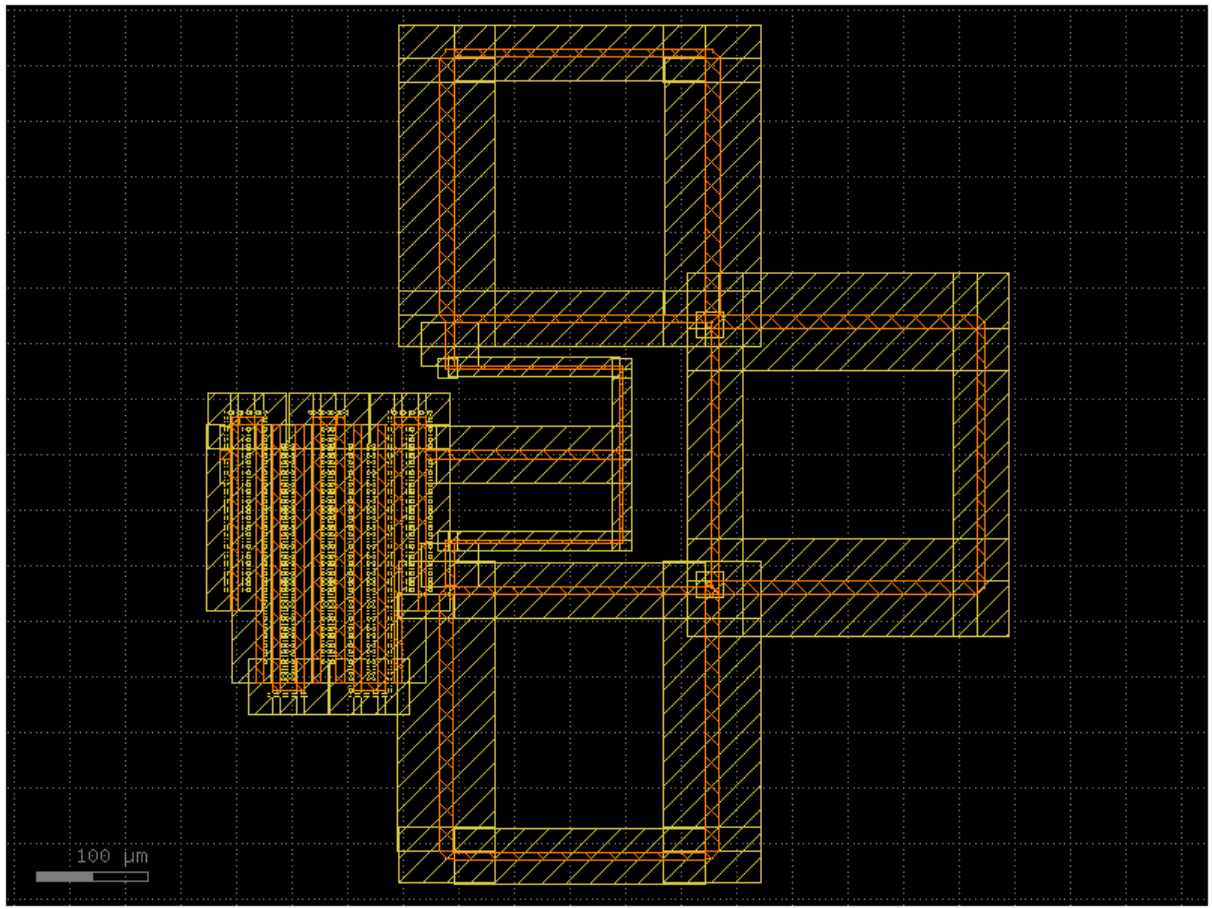
```
connection_bpf_wpd = c.add_ref(ihp.cells.tline(
    length=connection_length_bpf_wpd,
    signal_cross_section=signal_cross_section,
```

```
    ground_cross_section=ground_cross_section,
    Z0=Z0,
))

connection_bpf_wpd.connect("e1", wpd_ref.ports["e1"])
bandpass_filter_ref.connect("e1", connection_bpf_wpd.ports["e2"])

c.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
```



3.6 Termination of the RF Coupler

The fourth port of BLC_3 (`e3`) needs a matched termination for six-port operation. It ends in a 52 Ω silicided poly resistor, R_4 in the paper [1], which is contacted on Metal1. The next cell adds a 30 µm long 50 Ω line from that port and a via stack from TopMetal2 down to Metal1

at its end.

```
connection_blc_R_termination = ihp.cells.straight(
    length=CONNECTION_LEN_TERM,
    cross_section=signal_cross_section,
    width=ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r, signal_cross_section=signal_cross_section, ground_cross_section=ground_cross_section),
)
connection_blc_R_termination_ref = c.add_ref(connection_blc_R_termination)
connection_blc_R_termination_ref.connect("e1", blc_3_ref.ports["e3"])

via_M1_TM2 = ihp.cells.via_stack(
    top_layer="TopMetal2",
    bottom_layer="Metal1",
    vn_columns=2,
    vn_rows=2,
    vt1_columns=2,
    vt1_rows=2,
    vt2_columns=1,
    vt2_rows=4,
)

via_M1_TM2.plot()
via_M1_TM2.ports["top"].orientation = 180
via_M1_TM2.ports["bottom"].orientation = 180
via_M1_TM2_ref = c.add_ref(via_M1_TM2)
via_M1_TM2_ref.connect("top", connection_blc_R_termination_ref.ports["e2"], allow_width_mismatch=True)

c.plot()
```

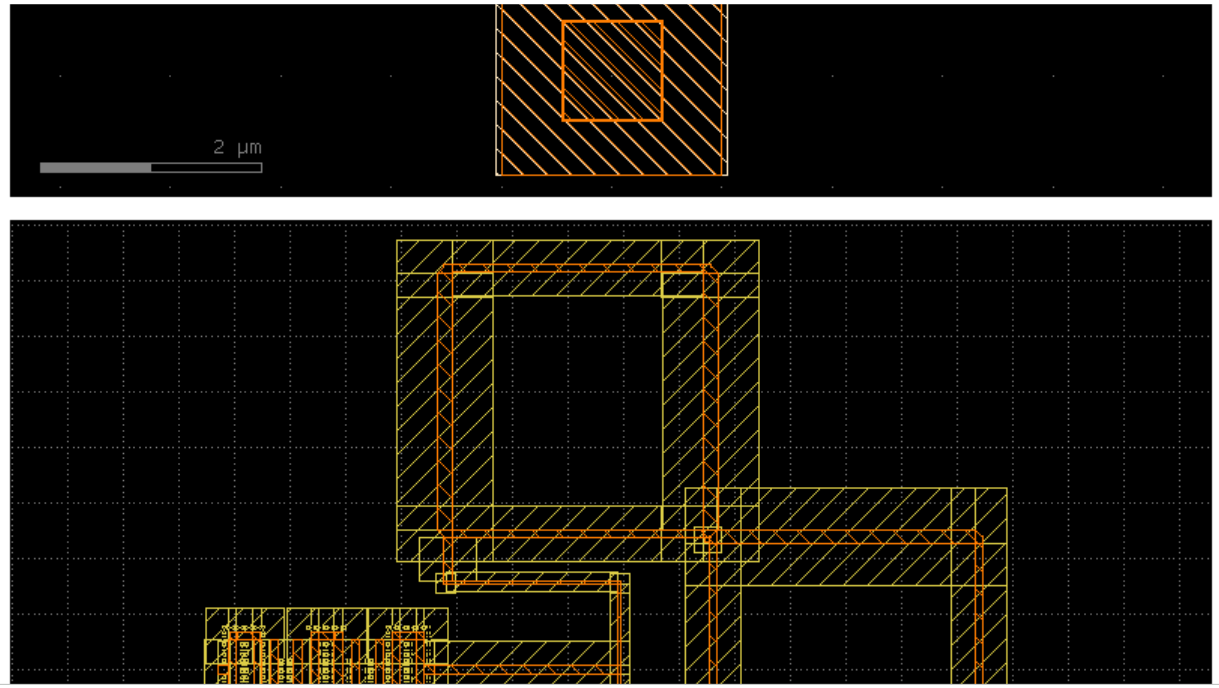


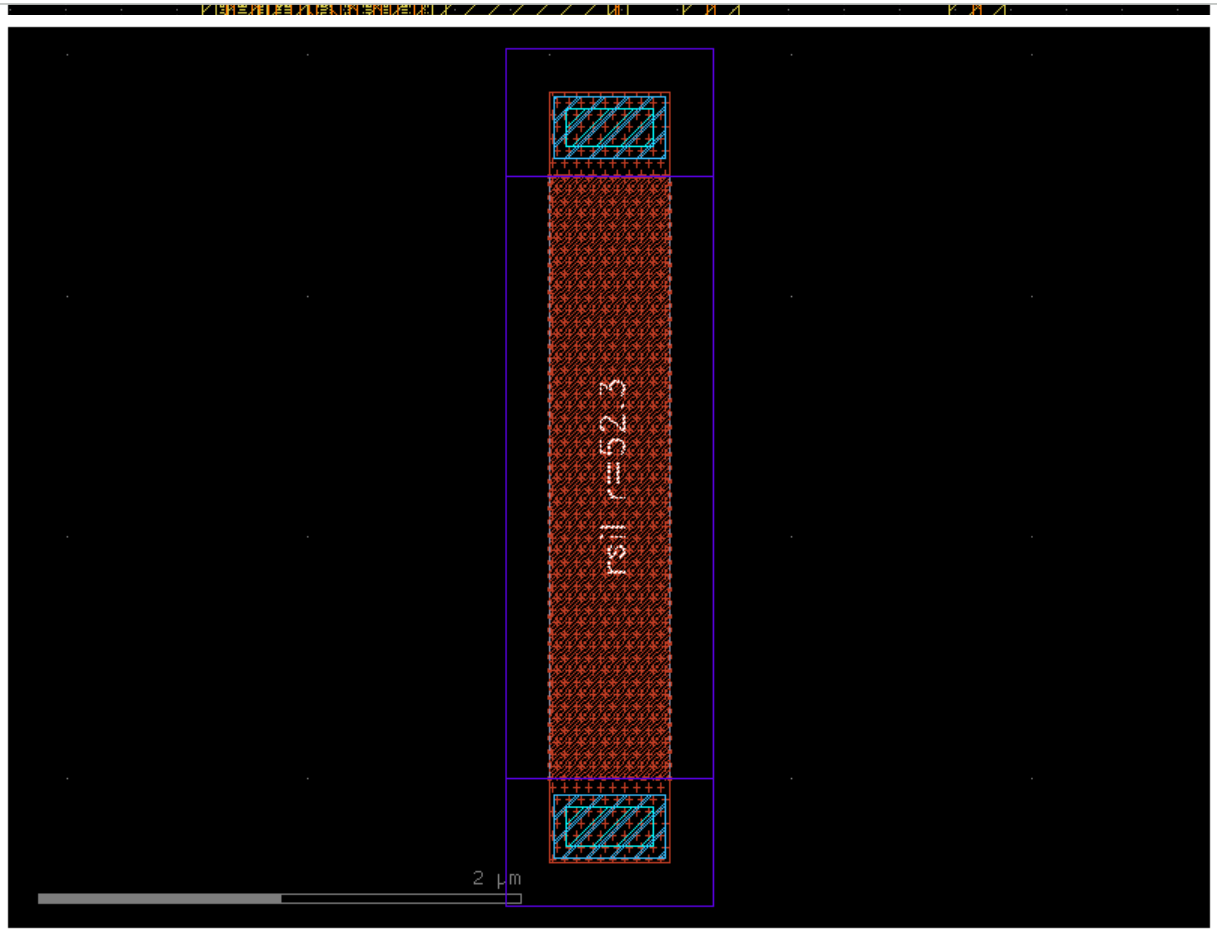
```
Calculated width: 7.2 um for Z0: 50 ohm
```



The resistor is an `rsil` device of 0.5 µm × 2.5 µm (`RSIL_WIDTH` × `RSIL_LENGTH`), the same device as the input termination R_1 of the power detectors.

```
R_termination = ihp.cells.rsil(
    width=RSIL_WIDTH,
    length=RSIL_LENGTH,
)
R_termination.plot()
```





The resistor is attached to the bottom of the via stack. A second via stack takes its other end up to Metal5, where a short Metal5 line joins the ground plane that the metal fill creates later.

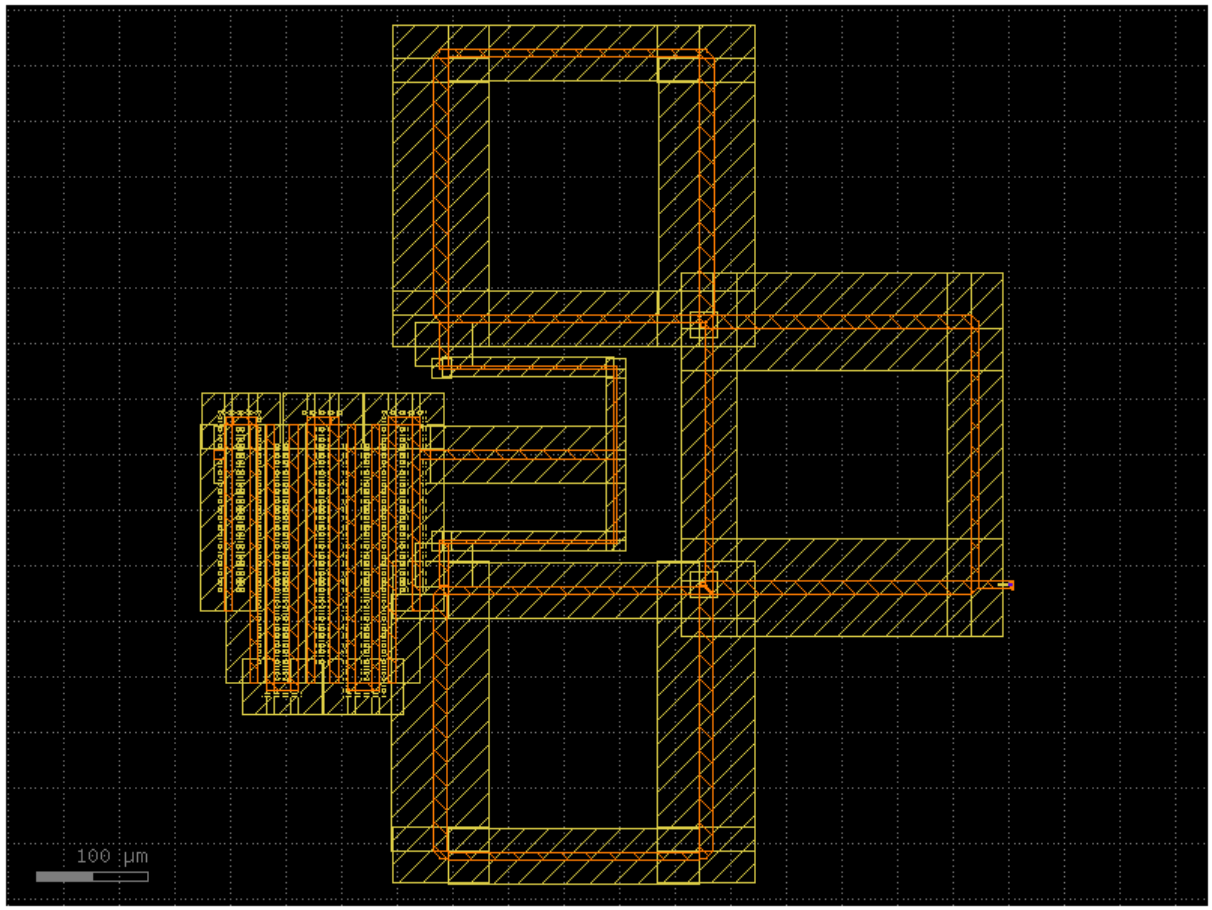
```
R_termination_ref = c.add_ref(R_termination)
R_termination_ref.connect("e1", via_M1_TM2_ref.ports["bottom"], allow_width_mismatch=True, allow_layer_mismatch=True)

via_M1_M5 = ihp.cells.via_stack(
    top_layer="Metal5",
    bottom_layer="Metal1",
    vn_columns=2,
    vn_rows=2,
)
via_M1_M5.ports["top"].orientation = 180
via_M1_M5.ports["bottom"].orientation = 0
```

```
via_M1_M5_ref = c.add_ref(via_M1_M5)
via_M1_M5_ref.connect("bottom", R_termination_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

connection_R_termination_vss = ihp.cells.straight(
    length=10, # manual measurement to fit the design
    cross_section=ground_cross_section,
    width=2, # manual measurement to fit the design
)
connection_R_termination_vss_ref = c.add_ref(connection_R_termination_vss)
connection_R_termination_vss_ref.connect("e1", via_M1_M5_ref.ports["top"], allow_width_mismatch=True, allow_layer_mismatch=True)

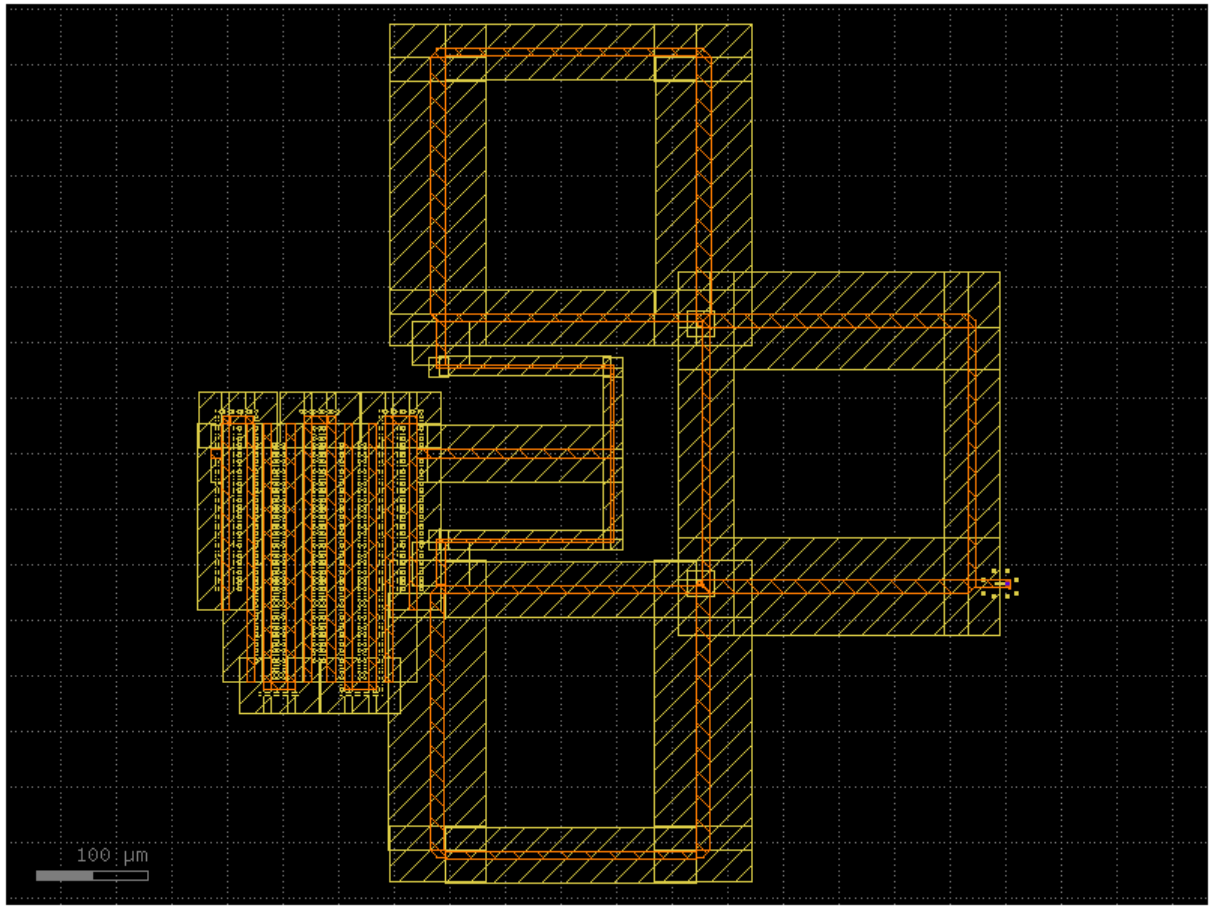
c.plot()
```



A Metal5 no-fill rectangle, 20 µm larger than the ground connection, keeps the ground plane away from the resistor and its vias, which reduces the parasitic capacitance of the termination.

```
# r termination metal5 nofill
c.add_ref(gf.components.rectangle(size=(connection_R_termination_vss_ref.xsize+20, connection_R_termination_vss_ref.ysize+20), laye

c.plot()
```



3.7 Probe Pads for the LO and RF Inputs

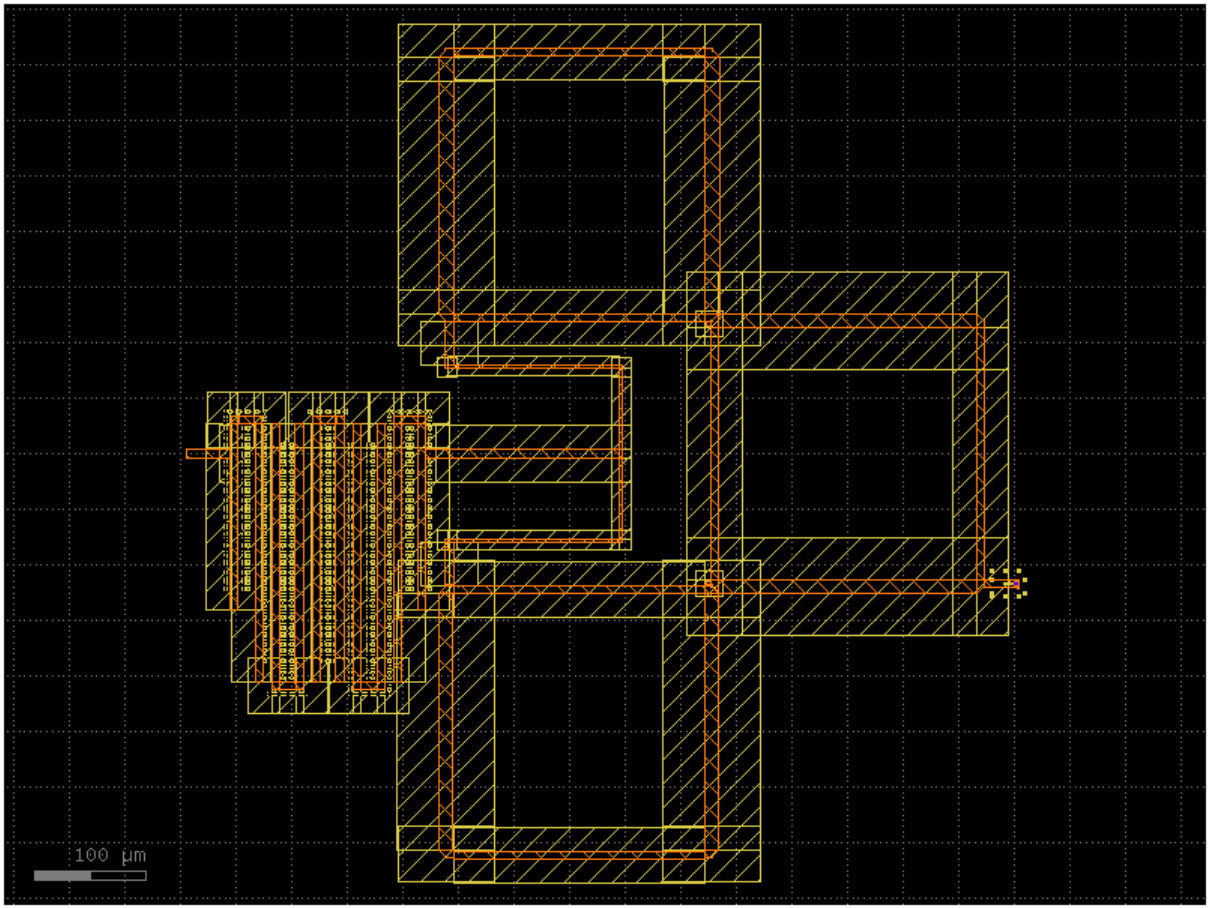
The LO and RF inputs are probed on the wafer with ground-signal-ground (GSG) probes at a pitch of 125 µm. On the LO side, a 30 µm long 50 Ω line first extends the free port of the BPF towards the pads.

```
connection_bpf_pad = c.add_ref(ihp.cells.straight(
    length=CONNECTION_LEN_BPF_PAD,
    cross_section=signal_cross_section,
```

```
    width=ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r, signal_cross_section=signal_cross_section, ground_cross_section=ground_cross_section),
))
connection_bpf_pad.connect("e1", bandpass_filter_ref.ports["e2"], allow_width_mismatch=True)

c.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
```

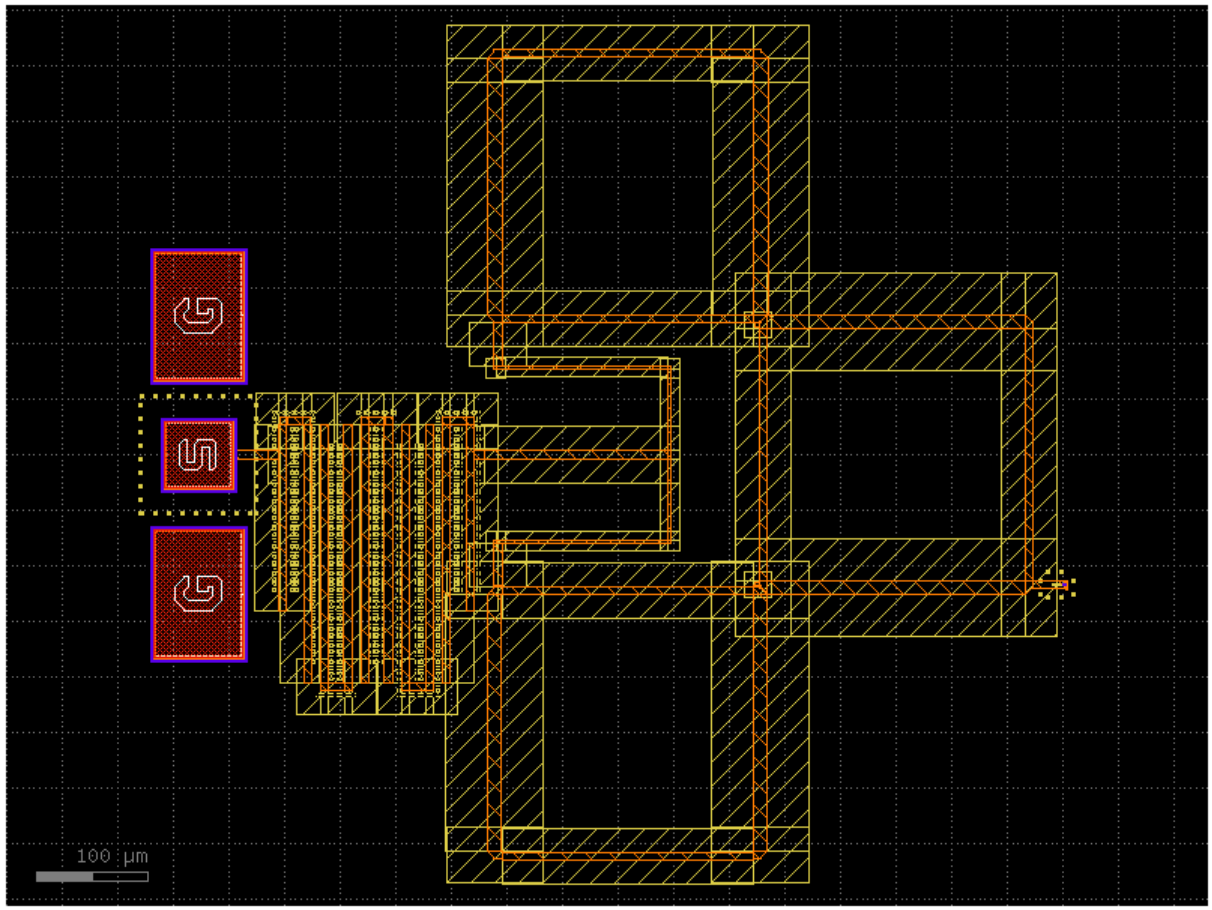


The LO pads form a GSG array (`bondpad_array`) with its ground pads tied to the substrate. The next cell rotates the array by -90° into a vertical column, connects its signal pad to the end of the line from the filter, adds a 105 µm square Metal5 no-fill region at its center to keep the pad capacitance low, and copies the pad ports to the chip with the prefix `probe_left_` for the pad labels of Section 3.11.

```
probe_left = c.add_ref(ihp.cells.bondpad_array(
    config="GSG",
    pitch=PROBE_PITCH,
```

```
    width_ground=PROBE_GROUND_WIDTH,
    width_signal=PROBE_SIGNAL_SIZE,
    length_signal=PROBE_SIGNAL_SIZE,
    signal_cross_section="topmetal1_routing",
    ground_cross_section="metal1_routing",
    ground_connection="psub",
)).rotate(-90)
probe_left.center = bandpass_filter_ref.ports["e2"].center
probe_left.xmax = bandpass_filter_ref.ports["e2"].center[0] - 15
probe_left.connect("e1", connection_bpf_pad.ports["e2"], allow_width_mismatch=True)

c.add_ref(gf.components.rectangle(size=(NOFILL_GSG_SIZE, NOFILL_GSG_SIZE), layer=ihp.tech.LAYER.Metal5nofill)).center = probe_left.center
c.add_ports(probe_left.ports, prefix="probe_left_")
c.plot()
```



The RF pads on the right are built the same way, with the prefix `probe_right_`. Two 50 Ω line pieces lead from port `e2` of BLC_3 to the pads. The first is three line widths long, the second `CONNECTION_LEN_BLC_PAD` + `CONNECTION_LEN_BLC_EXTRA` minus one line width.

```
width_Z0 = ihp.cells.waveguides._calculate_width_from_Z0(Z0=Z0, e_r=e_r, signal_cross_section=signal_cross_section, ground_cross_section=ground_cross_section)

connection_blc_pad_1 = c.add_ref(ihp.cells.straight(
    length=width_Z0 * 3,
    cross_section=signal_cross_section,
    width=width_Z0,
))
connection_blc_pad_1.connect("e1", blc_3_ref.ports["e2"])

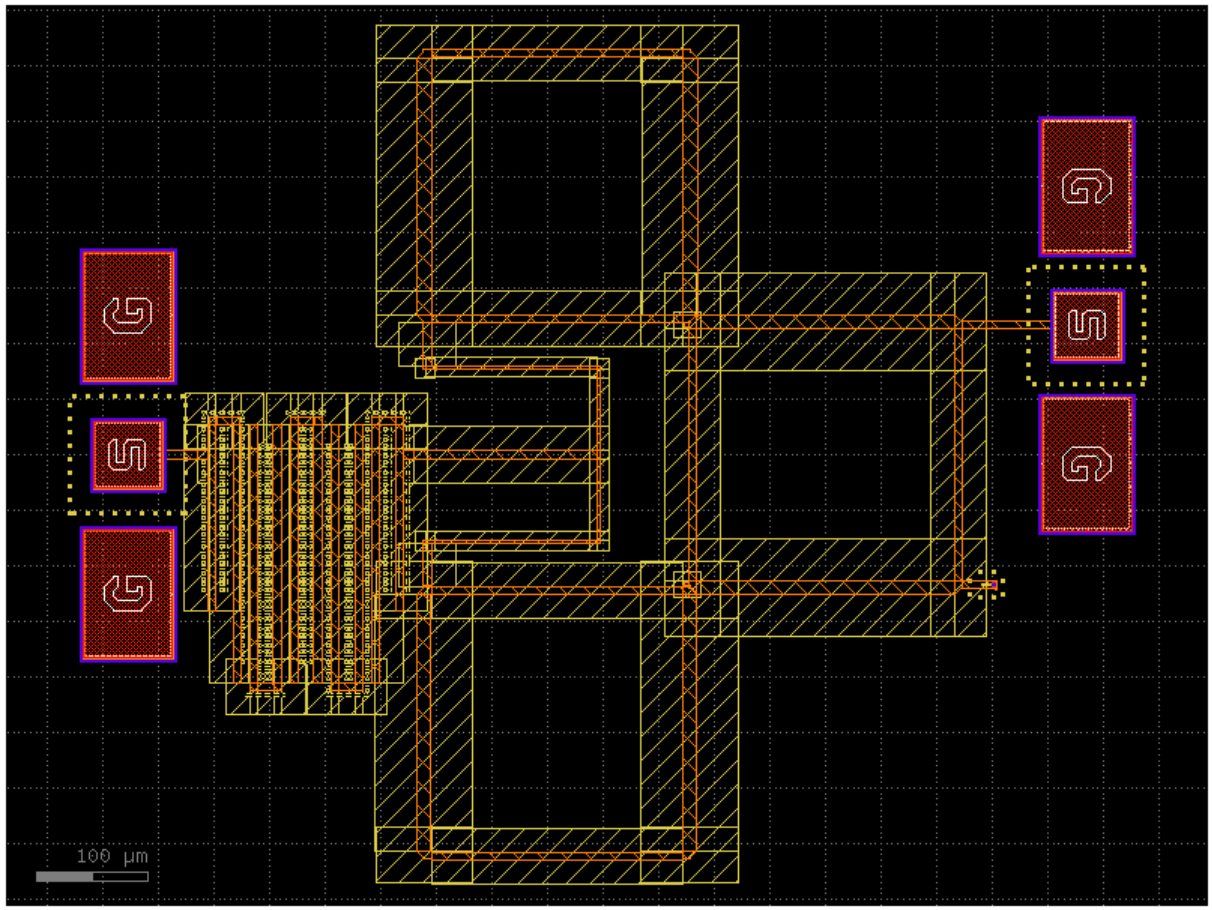
connection_blc_pad_2 = c.add_ref(ihp.cells.straight(
    length=round(CONNECTION_LEN_BLC_PAD + CONNECTION_LEN_BLC_EXTRA - width_Z0, 2),
    cross_section=signal_cross_section,
    width=width_Z0,
))
connection_blc_pad_2.connect("e1", connection_blc_pad_1.ports["e2"])

# probe pads right
probe_right = c.add_ref(ihp.cells.bondpad_array(
    config="GSG",
    pitch=PROBE_PITCH,
    width_ground=PROBE_GROUND_WIDTH_RIGHT,
    width_signal=PROBE_SIGNAL_SIZE,
    length_signal=PROBE_SIGNAL_SIZE,
    signal_cross_section="topmetal1_routing",
    ground_cross_section="metal1_routing",
    ground_connection="psub",
)).rotate(-90)
probe_right.connect("e1", connection_blc_pad_2.ports["e2"], allow_width_mismatch=True)

c.add_ref(gf.components.rectangle(size=(NOFILL_GSG_SIZE, NOFILL_GSG_SIZE), layer=ihp.tech.LAYER.Metal5nofill)).center = probe_right.center
c.add_ports(probe_right.ports, prefix="probe_right_")

c.plot()
```

```
Calculated width: 7.2 um for Z0: 50 ohm
```



3.8 SBD Power Detector

Each of the four six-port outputs drives a power detector built around a forward-biased Schottky barrier diode (SBD) [1]. The input from the BLC output is ac-coupled through the MIM capacitor C_1 to the diode D_1 and broadband-terminated by the silicided poly resistor R_1. The diode current drives a transimpedance amplifier (TIA), a CMOS inverter with the feedback resistor R_2, which delivers a buffered output V_{out}. A replica branch with copies of R_1, D_1, and R_2 and an inverter of one tenth the size produces V_{ref}. The output is read differentially as $V_{\mathrm{out}} - V_{\mathrm{ref}}$, which removes the dc offset, rejects common-mode noise, and tracks thermal drift. MIM capacitor arrays decouple the supply and the bias node on chip, and the 5 pF low-pass capacitor of each output sits off chip.

Symbol	Type	Value
V_{DD}	supply	1.5 V

Symbol	Type	Value
R_1	silicided poly (`rsil`)	52 Ω
R_2	poly (`rppd`)	911 Ω
R_3	high-resistance poly (`rhigh`), bias from V_{DD}	6.23 kΩ
C_1	MIM	150 fF
M_1, M_2	NMOS and PMOS, L = 0.13 µm	W = 5 µm and 10 µm in the replica, ten times that in the detector
D_1	SBD (`schottky_nbl1`)	one unit cell

The transistors have minimum length and run in strong inversion for bandwidth, with 4 mA in the detector branch and 400 µA in the replica. The layout keeps the RF input path short: one via stack runs from the TopMetal2 input down to the top plate of C_1 on TopMetal1, and a second one from its bottom plate on Metal5 straight down to the diode. Metal5 no-fill regions keep the ground plane away from the input, and the devices of the detector and the replica sit next to each other for matching.

The next cell draws the schematic with `schemdraw`. The node voltages are the simulated dc operating point, and the currents at the bottom are the bias currents of the detector branch (4 mA) and the replica (400 µA). Each replica device carries the label of the device it copies, and the detector transistors are labeled $10\,M_1$ and $10\,M_2$, since they are ten times the size of the replica transistors.

```
# Drawing of SBD-based Power Detector
import matplotlib
matplotlib.rcParams.update({
    "mathtext.fontset": "cm",
    "font.family": "serif",
})
import schemdraw as sd
import schemdraw.elements as elm
sd.svgconfig.svg2 = False

with sd.Drawing() as d:
    d.config(unit=2, fontsize=16)

    # Vrf Input, C1, D1
    Vrf = elm.Dot(open=True).label(r'$V_\mathrm{rf}$', loc='left', ofst=0.15)
    elm.Line().right().length(1.0)
    C1 = elm.Capacitor().right().label(r'$C_\mathrm{1}$', loc='top', ofst=0.15)
    elm.Line().right().length(1.0)
    node_rfin_int = elm.Dot().label(r'$1.128\,\mathrm{V}$', loc='bottom', ofst=0.15)
    elm.Line().right().length(1.0)
    D1 = elm.Schottky().right().label(r'$D_\mathrm{1}$', loc='top', ofst=0.15)
    elm.Line().right().length(1.0)
    node_bb_int = elm.Dot().label(r'$0.756\,\mathrm{V}$', loc='bottom', ofst=(-0.75, -0.15))

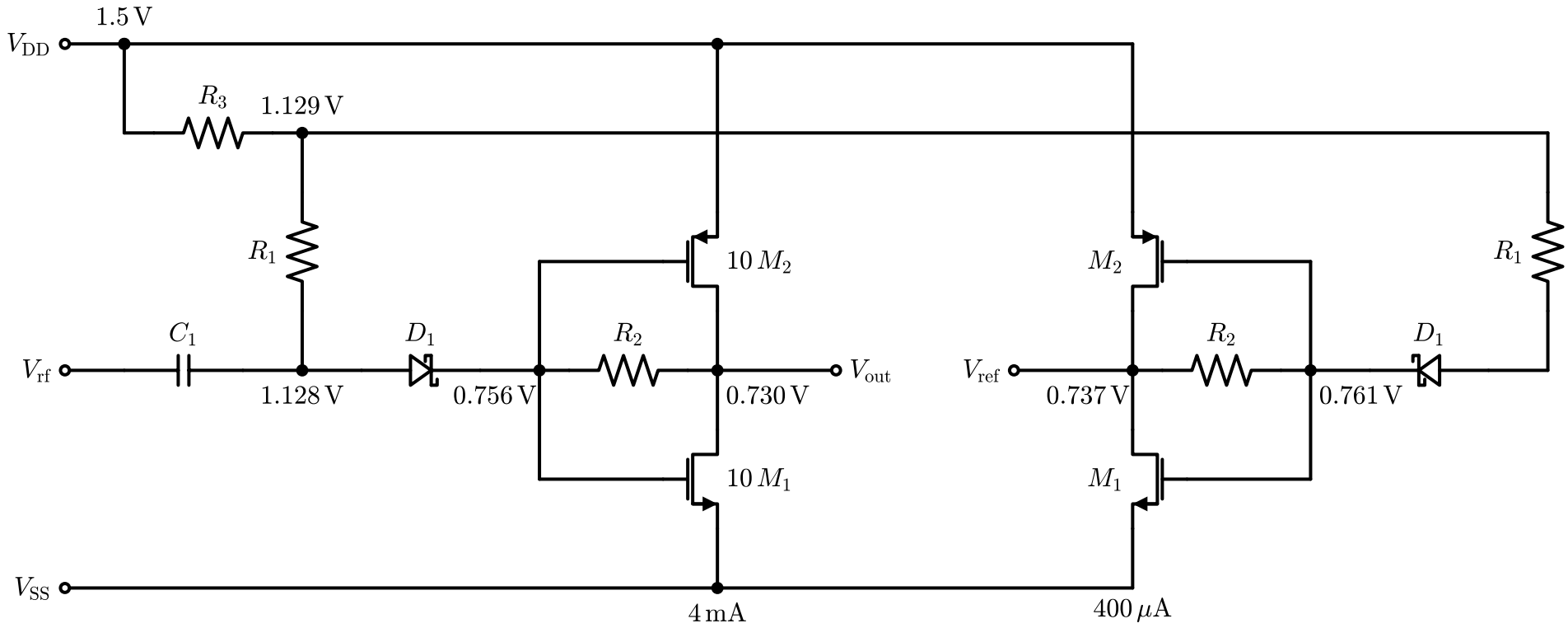
    # R1, R3, VDD
    elm.Line().at(node_rfin_int.end).up().length(1.0)
    R1 = elm.Resistor().up().label(r'$R_\mathrm{1}$', loc='top', ofst=0.15)
```

```
node_bias1 = elm.Line().up().length(1.0).dot().label(r'$1.129\,\mathrm{V}$', loc='right', ofst=0.15)
elm.Line().left().length(0.5)
R3 = elm.Resistor().left().label(r'$R_\mathrm{3}$', loc='top', ofst=0.15)
elm.Line().left().length(0.5)
node_vdd = elm.Line().up().length(1.5).dot()
elm.Dot().label(r'$1.5\,\mathrm{V}$', loc='top', ofst=0.15)
elm.Line().left().tox(Vrf.start)
vdd = elm.Dot(open=True).label(r'$V_\mathrm{DD}$', loc='left', ofst=0.15)

# M3, M4, R2, VSS
elm.Line().at(node_bb_int.end).right().length(0.5)
R2 = elm.Resistor().right().label(r'$R_\mathrm{2}$', loc='top', ofst=0.15)
elm.Line().right().length(0.5)
node_vout = elm.Dot().label(r'$0.730\,\mathrm{V}$', loc='bottom', ofst=(0.85, -0.15))
elm.Line().down().length(1.0)
M3 = elm.AnalogNFet(offset_gate=False).anchor('drain').theta(0).label(r'$10\,M_\mathrm{1}$', loc='right', ofst=0.15).reverse()
elm.Line().at(M3.source).down().length(1.0)
elm.Dot().label(r'$4\,\mathrm{mA}$', loc='bottom', ofst=0.15)
elm.Line().left().tox(Vrf.start)
vss = elm.Dot(open=True).label(r'$V_\mathrm{SS}$', loc='left', ofst=0.15)
elm.Line().at(M3.gate).left().tox(node_bb_int.end)
elm.Line().up().toy(node_bb_int.end)
elm.Line().at(node_vout.end).up().length(1.0)
M4 = elm.AnalogPFet(offset_gate=False).anchor('drain').theta(0).label(r'$10\,M_\mathrm{2}$', loc='right', ofst=0.15).reverse()
elm.Line().at(M4.source).up().toy(node_vdd.end).dot()
elm.Line().left().tox(node_vdd.end)
elm.Line().at(M4.gate).left().tox(node_bb_int.end)
elm.Line().down().toy(node_bb_int.end)
elm.Line().at(node_vout.end).right().length(2.0)
Vout = elm.Dot(open=True).label(r'$V_\mathrm{out}$', loc='right', ofst=0.15)

# Replica Circuit: R5, D2, R4, M1, M2
elm.Line().at(node_bias1.end).right().length(21.0)
elm.Line().down().length(1.0)
R5 = elm.Resistor().down().label(r'$R_\mathrm{1}$', loc='top', ofst=0.15)
elm.Line().down().length(1.0)
elm.Line().left().length(1.0)
D2 = elm.Schottky().left().label(r'$D_\mathrm{1}$', loc='top', ofst=0.15)
elm.Line().left().length(1.0)
node_bias2 = elm.Dot().label(r'$0.761\,\mathrm{V}$', loc='bottom', ofst=(0.85, -0.15))
elm.Line().at(node_bias2.end).left().length(0.5)
R4 = elm.Resistor().left().label(r'$R_\mathrm{2}$', loc='top', ofst=0.15)
elm.Line().left().length(0.5)
node_vref = elm.Dot().label(r'$0.737\,\mathrm{V}$', loc='bottom', ofst=(-0.75, -0.15))
elm.Line().down().length(1.0)
M1 = elm.AnalogNFet(offset_gate=False).anchor('drain').theta(0).label(r'$M_\mathrm{1}$', loc='left', ofst=0.15)
elm.Line().at(M1.source).down().toy(vss.end).label(r'$400\,\mu\mathrm{A}$', loc='left', ofst=0.15)
elm.Line().left().tox(node_vout.end)
elm.Line().at(M1.gate).right().tox(node_bias2.end)
elm.Line().up().toy(node_bias2.end)
```

```
    elm.Line().at(node_vref.end).up().length(1.0)
    M2 = elm.AnalogPFet(offset_gate=False).anchor('drain').theta(0).label(r'$M_\mathrm{2}$', loc='left', ofst=0.15)
    elm.Line().at(M2.source).up().toy(node_vdd.end)
    elm.Line().left().tox(node_vout.end)
    elm.Line().at(M2.gate).right().tox(node_bias2.end)
    elm.Line().down().toy(node_bias2.end)
    elm.Line().at(node_vref.end).left().length(2.0)
    Vref = elm.Dot(open=True).label(r'$V_\mathrm{ref}$', loc='left', ofst=0.15)
```



The function `powdet_sbd()` in the next cell draws the complete detector as one GDSFactory cell. The `@gf.cell` decorator caches the result, so the four detectors of the chip are references to one cell. The function builds the detector in seven steps:

1. RF input: the via stack from the TopMetal2 input to C_1 on TopMetal1, and the via stack from the bottom plate of C_1 on Metal5 down to the diode D_1. The replica diode D_2 sits right next to D_1.
2. Bias decoupling C_2: 30 MIM capacitors of 10 µm × 10 µm in five rows of six, placed above C_1 and connected plate to plate, between the bias node and ground.
3. Terminations R_1: one `rsil` resistor for each diode, routed on Metal1 and Metal2.

4. TIA: one NMOS and one PMOS device with 24 gate fingers each. 20 fingers form the detector inverter, 2 the replica inverter, and the 2 outermost fingers are dummies. The drains are collected through via stacks.
5. Feedback resistors R_2: two `rppd` resistors from the diode nodes to `vout` and `vref`.
6. Supply decoupling and bias: 28 MIM capacitors C_3 and two MIM fill capacitors C_4 between VDD and VSS, the bias resistor R_3 (`rhigh`) from VDD, the bias routing to both diodes, and the substrate connection of the diodes.
7. Ports and no-fill zones: the TopMetal2 feed line of length `RFIN_PD` with the port `rfin`, and the ports `vout`, `vref`, `vdd`, and `vss` for the top level. No-fill regions on every metal layer keep the metal fill out of the detector, and the function prints the port table of the finished cell.

```
@gf.cell
def powdet_sbd() -> gf.Component:
    """Schottky-diode power detector with transimpedance amplifier.

    Ports: rfin (TM2), vout (M4), vref (M4), vss (M5), vdd (TM1).
    """
    c = gf.Component("powdet_sbd")

    # via from rfin on TM2 to C1 on TM1
    via_tm1_tm2 = ihp.cells.via_stack(
        top_layer="TopMetal2",
        bottom_layer="TopMetal1",
        vt2_columns=4,
        vt2_rows=3,
    )

    via_tm1_tm2.ports["bottom"].orientation = 0
    via_tm1_tm2_ref = c.add_ref(via_tm1_tm2)
    # reference cmim capacitor and connect to via
    cmim = ihp.cells.cmim(width=CMIM_SIZE, length=CMIM_SIZE)
    c1_ref = c.add_ref(cmim)
    c1_ref.connect("T", via_tm1_tm2_ref.ports["bottom"], allow_width_mismatch=True)


    # via cell from Metal2 to Metal5
    via_m2_m5 = ihp.cells.via_stack(
        top_layer="Metal5",
        bottom_layer="Metal2",
        vn_columns=6,
        vn_rows=3,
    )

    # via from CMIM C1 to XQ1
    via_m2_m5.ports["top"].orientation = 0
    via_m2_m5_ref = c.add_ref(via_m2_m5)
    via_m2_m5_ref.connect("top", c1_ref.ports["B"], allow_width_mismatch=True)
```

```
# create D1 and connect to rfin via C1 and the vias
schottky = ihp.cells.schottky()
schottky.locked = False
schottky.add_label(text="schottky_nbl1", layer=ihp.tech.LAYER.TEXTdrawing)

d1_ref = c.add_ref(schottky)
d1_ref.connect("e4", via_m2_m5_ref.ports["bottom"], allow_width_mismatch=True, allow_layer_mismatch=True)

# Create D2 nearby D1 for better matching
d2_ref = c.add_ref(schottky)
d2_ref.ymin = d1_ref.ymin
d2_ref.xmin = d1_ref.xmax  # min spacing according to design rules 5.10

# Create capacitor array for c2
c2 = gf.Component("C2")
cmim = ihp.cells.cmim(width=CMIM_SIZE, length=CMIM_SIZE)
cap_connection_thickness = CAP_BUS_WIDTH  # thickness of the metal connection between caps
# 30 cmim capacitors in five rows of six, column pitch 0.52 + 11.2 for the TM1 spacing
spacing_between_caps = CMIM_SPACING_TM1
top_row = c2.add_ref(
    gf.components.array(component=cmim, columns=6, rows=1, column_pitch=spacing_between_caps + CMIM_PITCH)
)

middle_row = c2.add_ref(
    gf.components.array(component=cmim, columns=6, rows=1, column_pitch=spacing_between_caps + CMIM_PITCH)
)
bottom_row = c2.add_ref(
    gf.components.array(
        component=cmim,
        columns=6,
        rows=3,
        column_pitch=spacing_between_caps + CMIM_PITCH,
        row_pitch=-(spacing_between_caps + CMIM_PITCH),
    )
)
top_row.ymin = middle_row.ymax + spacing_between_caps  # min spacing according to design rules 5.17
bottom_row.ymax = middle_row.ymin - spacing_between_caps  # min spacing according to design rules 5.17
c2.add_ports(top_row.ports, prefix="TR_")
c2.add_ports(middle_row.ports, prefix="MR_")
c2.add_ports(bottom_row.ports, prefix="BR_")


# Orient outermost ports inward for horizontal cap-to-cap connection
for prefix in [
    "TR_T_1_",
    "TR_B_1_",
    "MR_T_1_",
    "MR_B_1_",
    "BR_T_1_",
    "BR_B_1_",
```

```
        "BR_T_2_",
        "BR_B_2_",
        "BR_T_3_",
        "BR_B_3_",
    ]:
        c2.ports[f"{prefix}1"].orientation = 0
        c2.ports[f"{prefix}6"].orientation = 180

    c2_connection_t = gf.routing.route_bundle_electrical(
        component=c2,
        ports1=[
            c2.ports["TR_T_1_1"],
            c2.ports["BR_T_2_1"],
            c2.ports["BR_T_3_1"],
            c2.ports["MR_T_1_1"],
            c2.ports["BR_T_1_1"],
        ],
        ports2=[
            c2.ports["TR_T_1_6"],
            c2.ports["BR_T_2_6"],
            c2.ports["BR_T_3_6"],
            c2.ports["MR_T_1_6"],
            c2.ports["BR_T_1_6"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    c2_connection_b = gf.routing.route_bundle_electrical(
        component=c2,
        ports1=[
            c2.ports["TR_B_1_1"],
            c2.ports["BR_B_2_1"],
            c2.ports["BR_B_3_1"],
            c2.ports["MR_B_1_1"],
            c2.ports["BR_B_1_1"],
        ],
        ports2=[
            c2.ports["TR_B_1_6"],
            c2.ports["BR_B_2_6"],
            c2.ports["BR_B_3_6"],
            c2.ports["MR_B_1_6"],
            c2.ports["BR_B_1_6"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.Metal5drawing,
```

```
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# orient top row ports to face downwards to connect to bottom row ports
ihp.cells.utils.change_port_orientation(
    c2, ["TR_T_1_1", "TR_T_1_2", "TR_T_1_3", "TR_T_1_4", "TR_T_1_5", "TR_T_1_6"], 270
)
ihp.cells.utils.change_port_orientation(
    c2, ["TR_B_1_1", "TR_B_1_2", "TR_B_1_3", "TR_B_1_4", "TR_B_1_5", "TR_B_1_6"], 270
)

# orient bottom row ports to face downwards to connect to top row ports
ihp.cells.utils.change_port_orientation(
    c2, ["BR_T_3_1", "BR_T_3_2", "BR_T_3_3", "BR_T_3_4", "BR_T_3_5", "BR_T_3_6"], 90
)
ihp.cells.utils.change_port_orientation(
    c2, ["BR_B_3_1", "BR_B_3_2", "BR_B_3_3", "BR_B_3_4", "BR_B_3_5", "BR_B_3_6"], 90
)

c2_connection_top = gf.routing.route_bundle_electrical(
    component=c2,
    ports1=[
        c2.ports["BR_T_3_1"],
        c2.ports["BR_T_3_2"],
        c2.ports["BR_T_3_3"],
        c2.ports["BR_T_3_4"],
        c2.ports["BR_T_3_5"],
        c2.ports["BR_T_3_6"],
    ],
    ports2=[
        c2.ports["TR_T_1_1"],
        c2.ports["TR_T_1_2"],
        c2.ports["TR_T_1_3"],
        c2.ports["TR_T_1_4"],
        c2.ports["TR_T_1_5"],
        c2.ports["TR_T_1_6"],
    ],
    route_width=cap_connection_thickness,  # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

c2_connection_bottom = gf.routing.route_bundle_electrical(
    component=c2,
```

```
        ports1=[
            c2.ports["BR_B_3_1"],
            c2.ports["BR_B_3_2"],
            c2.ports["BR_B_3_3"],
            c2.ports["BR_B_3_4"],
            c2.ports["BR_B_3_5"],
            c2.ports["BR_B_3_6"],
        ],
        ports2=[
            c2.ports["TR_B_1_1"],
            c2.ports["TR_B_1_2"],
            c2.ports["TR_B_1_3"],
            c2.ports["TR_B_1_4"],
            c2.ports["TR_B_1_5"],
            c2.ports["TR_B_1_6"],
        ],
        route_width=cap_connection_thickness,  # adjust for thicker connections
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # orient top row ports to face upwards to connect the ground plates to the other caps
    ihp.cells.utils.change_port_orientation(c2, ["TR_B_1_1", "TR_B_1_2", "TR_B_1_3", "TR_B_1_6"], 90)
    ihp.cells.utils.change_port_orientation(c2, ["TR_T_1_6"], 0)  # for connection to bias1
    ihp.cells.utils.change_port_orientation(c2, ["BR_B_3_4", "BR_B_3_5"], 270)

    c2_ref = c.add_ref(c2)
    c2_ref.rotate(-90)
    c2_ref.center = c1_ref.center
    c2_ref.ymin = c1_ref.ymax + 10
    c2_ref.xmax = c1_ref.xmax + c1_ref.xsize + CMIM_SPACING_TM1

    ## end of c2 generation

    r_sil = ihp.cells.rsil(width=RSIL_WIDTH, length=RSIL_LENGTH)

    # let the ports of xr1 and xr5 face each other for easier routing
    r_sil.ports["e2"].orientation = 0
    xr1_ref = c.add_ref(r_sil.copy())

    r_sil.ports["e2"].orientation = 180
    xr5 = r_sil.copy()
    xr5_ref = c.add_ref(xr5)

    # spacing between D1 and xr1/xr5
    spacing = 1
    xr1_ref.xmax = d1_ref.xmax - 1.88
```

```
xr1_ref.ymax = d1_ref.ymin - spacing

xr5_ref.xmin = d2_ref.xmin + 1.88
xr5_ref.ymin = xr1_ref.ymin

route_xr1_xr5 = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[xr1_ref.ports["e2"]],
    ports2=[xr5_ref.ports["e2"]],
    route_width=ROUTE_WIDTH_MIN_M1,
    layer=ihp.tech.LAYER.Metal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# via for connection D1 to xr1 and xr5 to D2
via_m1_m2 = ihp.cells.via_stack(
    top_layer="Metal2",
    bottom_layer="Metal1",
    vn_columns=6,
    vn_rows=2,
)

# via D1 -> xr1
via_m1_m2.ports["top"].orientation = 90
via_m1_m2.ports["bottom"].orientation = 0
via_m1_m2_ref = c.add_ref(via_m1_m2.copy())
via_m1_m2_ref.center = d1_ref.center
via_m1_m2_ref.ymax = d1_ref.ymin

# route from D1 to via
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[d1_ref.ports["e4"]],
    ports2=[via_m1_m2_ref.ports["top"]],
    route_width=max(via_m1_m2_ref.xsize, via_m1_m2_ref.ysize),
    layer=ihp.tech.LAYER.Metal2drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# route from via to xr1
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[via_m1_m2_ref.ports["bottom"]],
    ports2=[xr1_ref.ports["e1"]],
```

```
    route_width=min(via_m1_m2_ref.xsize, via_m1_m2_ref.ysize),
    layer=ihp.tech.LAYER.Metal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# connect second diode to input resistors
# via D2 -> xr5
via_m1_m2.ports["top"].orientation = 90
via_m1_m2.ports["bottom"].orientation = 180
via_m1_m2_ref = c.add_ref(via_m1_m2.copy())
via_m1_m2_ref.center = d2_ref.center
via_m1_m2_ref.ymax = d2_ref.ymin

# route from D2 to via
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[d2_ref.ports["e4"]],
    ports2=[via_m1_m2_ref.ports["top"]],
    route_width=max(via_m1_m2_ref.xsize, via_m1_m2_ref.ysize),
    layer=ihp.tech.LAYER.Metal2drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# route from via to xr5
route = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[via_m1_m2_ref.ports["bottom"]],
    ports2=[xr5_ref.ports["e1"]],
    route_width=min(via_m1_m2_ref.xsize, via_m1_m2_ref.ysize),
    layer=ihp.tech.LAYER.Metal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

## Build transimpedance amplifier with Schottky diode as input device.
# M1 (nmos, 20 gates) + M3 (nmos, 2 gates) + 2 dummy gates = 24
# M2 (pmos, 20 gates) + M4 (pmos, 2 gates) + 2 dummy gates = 24
ng_mos = NUM_GATES
width_nmos = NMOS_WIDTH  # um
width_pmos = PMOS_WIDTH  # um

c_output = gf.Component("output_stage")
```

```
m1_3 = ihp.cells.nmos(
    w=width_nmos,
    l=GATE_LENGTH,
    ng=ng_mos,
    guardRingType="psub",
    guardRingDistance=GUARD_RING_DISTANCE,
)

via_gat_m1 = ihp.cells.via_stack(
    top_layer="Metal1",
    bottom_layer="GatPoly",
    vn_columns=1,
    vn_rows=2,
)

via_gat_m2 = ihp.cells.via_stack(
    top_layer="Metal2",
    bottom_layer="GatPoly",
    vn_columns=1,
    vn_rows=3,
)

via_gat_m3 = ihp.cells.via_stack(
    top_layer="Metal3",
    bottom_layer="GatPoly",
    vn_columns=1,
    vn_rows=3,
)

m1_3.locked = False
via_gat_m1.ports["bottom"].orientation = 90
via_gat_m2.ports["bottom"].orientation = 90
via_gat_m3.ports["bottom"].orientation = 90
via_gat_m3.ports["top"].orientation = 270  # turn top port for later connection
gate_ports = m1_3.get_ports_list(layer=ihp.tech.LAYER.GatPolydrawing)
gate_ports.sort(key=lambda p: p.center[0])  # left to right, so list indices match the G_ numbering
gate_straight = ihp.cells.straight(
    length=GATE_EXTENSION_NMOS, cross_section="gatpoly_routing", width=GATE_LENGTH  # gate poly width
)
gate_via_refs_nmos = []
for i, p in enumerate(gate_ports):
    if p.name in ["G_1", "G_24"]:  # these gates will be connected to metal1
        gate_via_refs_nmos.append(m1_3.add_ref(via_gat_m2))
    elif p.name in ["G_12", "G_13"]:  # these gates will be connected to metal2
        gate_via_refs_nmos.append(m1_3.add_ref(via_gat_m2))
    else:  # all others to metal3
        gate_via_refs_nmos.append(m1_3.add_ref(via_gat_m3))
    gate_straight_ref = m1_3.add_ref(gate_straight)
    gate_straight_ref.connect("e1", m1_3.ports[p.name], allow_width_mismatch=True, allow_layer_mismatch=True)
```

```
    via_gat_m2_ref = gate_via_refs_nmos[-1]
    via_gat_m2_ref.connect(
        "bottom", gate_straight_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True
    )

# connect gates 1 and 24 with metal1 to ground
via_gat_m2.ports["top"].orientation = 0
start_point = gate_via_refs_nmos[23].ports["top"].center[0]  # G_24's via
end_point = m1_3.ports["DS_29"].center[0]  # right guard-ring tap
straight_ref = m1_3.add_ref(
    ihp.cells.straight(
        length=abs(end_point - start_point),
        cross_section="metal2_routing",
        width=1.11,  # manual measure of the height of the via
    )
)
straight_ref.connect("e1", gate_via_refs_nmos[23].ports["top"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_m1_m2 = ihp.cells.via_stack(
    top_layer="Metal2",
    bottom_layer="Metal1",
    vn_columns=3,
    vn_rows=1,
)
via_m1_m2_ref = m1_3.add_ref(via_m1_m2)
via_m1_m2_ref.connect("top", straight_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

via_gat_m2.ports["top"].orientation = 180
straight_ref = m1_3.add_ref(
    ihp.cells.straight(
        length=abs(end_point - start_point),
        cross_section="metal2_routing",
        width=1.11,  # manual measure of the height of the via
    )
)
straight_ref.connect(
    "e1", gate_via_refs_nmos[0].ports["top"], allow_width_mismatch=True, allow_layer_mismatch=True
)
via_m1_m2_ref = m1_3.add_ref(via_m1_m2)
via_m1_m2_ref.connect("top", straight_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

# connect Gates 2-11 and 14-23 (all 20 gates) together with metal3
start_point = m1_3.ports["G_2"].center
end_point = m1_3.ports["G_23"].center
straight_ref = m1_3.add_ref(
    ihp.cells.straight(
        length=abs(end_point[0] - start_point[0]),
        cross_section="metal3_routing",
        width=max(
            gate_via_refs_nmos[1].xsize, gate_via_refs_nmos[1].ysize
        ),  # manual measure of the height of the via
```

```
    )
)
straight_ref.center = (
    (start_point[0] + end_point[0]) / 2,
    gate_via_refs_nmos[1].center[1],
)  # align with the via of the first M1 gate

# connect Gates 12-13 (2 gates) together with metal2
start_point = m1_3.ports["G_12"].center
end_point = m1_3.ports["G_13"].center
straight_ref = m1_3.add_ref(
    ihp.cells.straight(
        length=abs(end_point[0] - start_point[0]),
        cross_section="metal2_routing",
        width=0.6,  # manual measure of the height of the via
    )
)
straight_ref.center = (
    (start_point[0] + end_point[0]) / 2,
    gate_via_refs_nmos[11].center[1],
)  # align with the via of the first M3 gate

# connect source to GND
# Source and drain fingers by their deterministic left-to-right port names
# (DS_ ports are renumbered by position in the IHP PDK, so these lists are
# stable across tool versions). DS_15 is the center column that also holds
# the shortened drain via stack.
m1_3_source_ports = [m1_3.ports[f"DS_{i}"] for i in (2, 3, 5, 7, 9, 11, 13, 17, 19, 21, 23, 25, 27, 28)]
m1_3_drain_ports = [m1_3.ports[f"DS_{i}"] for i in (4, 6, 8, 10, 12, 15, 18, 20, 22, 24, 26)]

# source ground connection
source_straight = ihp.cells.straight(
    length=3.28,  # source finger to the center of the top guardring segment
    # (was |DS_1.y - DS_27.y| under the old port enumeration, whose
    # guard-rail pin ports are no longer emitted)
    cross_section="metal2_routing",
    width=0.26,  # manual measure of the height of the via
)
via_m1_m2 = ihp.cells.via_stack(
    top_layer="Metal2",
    bottom_layer="Metal1",
    vn_columns=7,
    vn_rows=1,
)
via_m1_m2.ports["bottom"].orientation = 0
via_m1_m2.ports["top"].orientation = 180
for p in m1_3_source_ports:
    p.orientation = 90  # orient ports to face the top
    via_m1_m2_ref = m1_3.add_ref(via_m1_m2)
    via_m1_m2_ref.connect("bottom", p, allow_width_mismatch=True, allow_layer_mismatch=True)
```

```
    source_straight_ref = m1_3.add_ref(source_straight)
    source_straight_ref.connect(
        "e1", via_m1_m2_ref.ports["top"], allow_width_mismatch=True, allow_layer_mismatch=True
    )

# connect drains to M3
via_m1_m4 = ihp.cells.via_stack(
    top_layer="Metal4",
    bottom_layer="Metal1",
    vn_columns=1,
    vn_rows=7,
)

drain_via_refs_nmos = []
for p in m1_3_drain_ports:
    p.orientation = 90  # orient ports to face the top
    if p.name in ["DS_15"]:  # center drain column (x = 6.27), gets the shortened stack
        via_m1_m4_short = ihp.cells.via_stack(
            top_layer="Metal4",
            bottom_layer="Metal1",
            vn_columns=1,
            vn_rows=3,
        )
        via_m1_m4_short.ports["top"].orientation = 270
        drain_via_refs_nmos.append(m1_3.add_ref(via_m1_m4_short))
        drain_via_refs_nmos[-1].center = p.center
        drain_via_refs_nmos[-1].ymax = p.center[1] - 0.245  # manual measurement to align with metal1 below
    else:
        via_m1_m4.ports["bottom"].orientation = 90
        via_m1_m4.ports["top"].orientation = 90
        via_m1_m4_ref = m1_3.add_ref(via_m1_m4)
        via_m1_m4_ref.connect("bottom", p, allow_width_mismatch=True, allow_layer_mismatch=True)
        drain_via_refs_nmos.append(via_m1_m4_ref)

# horizontal drain connection for nmos
straight_drain_connection = ihp.cells.straight(
    length=abs(
        drain_via_refs_nmos[0].center[0] - drain_via_refs_nmos[-1].center[0]
    ),  # span from the first to the last drain via
    cross_section="metal4_routing",
    width=2,  # manual measure of the height of the via
)
straight_drain_connection_ref = m1_3.add_ref(straight_drain_connection)
straight_drain_connection_ref.xmin = drain_via_refs_nmos[0].center[0]  # leftmost drain via
straight_drain_connection_ref.ymin = drain_via_refs_nmos[0].center[1]

m1_3.add_ports(gate_via_refs_nmos[6].ports, prefix="gate_M1_")  # G_7's via
m1_3.ports["DS_16"].orientation = 90  # so the top-level vdd via stack connects unrotated
m1_3.add_port(name="drain_connection_M1", port=drain_via_refs_nmos[8].ports["top"])  # DS_22 column (x = 9.33)
m1_3.add_port(name="drain_connection_M3", port=drain_via_refs_nmos[5].ports["top"])
```

```
m1_3.add_port(
    name="gate_connection_M3",
    center=(
        (gate_via_refs_nmos[11].center[0] + gate_via_refs_nmos[12].center[0]) / 2,
        gate_via_refs_nmos[11].center[1],
    ),
    cross_section="metal2_routing",
    orientation=270,
    port_type="electrical",
)
m1_3_ref = c_output.add_ref(m1_3)
m1_3_ref.move((0, 30))

m2_4 = ihp.cells.pmos(
    w=width_pmos,
    l=GATE_LENGTH,
    ng=ng_mos,
    guardRingType="nwell",
    guardRingDistance=GUARD_RING_DISTANCE,
)

# connect gates of M2 and M4 together

m2_4.locked = False
gate_ports = m2_4.get_ports_list(layer=ihp.tech.LAYER.GatPolydrawing)
gate_ports.sort(key=lambda p: p.center[0])  # left to right, so list indices match the G_ numbering
gate_straight = ihp.cells.straight(
    length=GATE_EXTENSION_PMOS, cross_section="gatpoly_routing", width=GATE_LENGTH  # gate poly width
)
gate_via_refs_pmos = []
for i, p in enumerate(gate_ports):
    p.orientation += 180  # orient gates to face downwards for easier connection to nmos gates
    if p.name in ["G_1", "G_24"]:  # these gates will be connected to metal1
        gate_via_refs_pmos.append(m2_4.add_ref(via_gat_m2))
    elif p.name in ["G_12", "G_13"]:  # these gates will be connected to metal2
        gate_via_refs_pmos.append(m2_4.add_ref(via_gat_m2))
    else:  # all others to metal3
        gate_via_refs_pmos.append(m2_4.add_ref(via_gat_m3))
    gate_straight_ref = m2_4.add_ref(gate_straight)
    gate_straight_ref.connect("e1", m2_4.ports[p.name], allow_width_mismatch=True, allow_layer_mismatch=True)
    gate_via_refs_pmos[-1].connect(
        "bottom", gate_straight_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True
    )

# connect gates 1 and 24 with metal1 to ground
via_gat_m2.ports["top"].orientation = 180
start_point = gate_via_refs_pmos[23].ports["top"].center[0]  # G_24's via
end_point = m2_4.ports["DS_29"].center[0]  # right guard-ring tap
straight_ref = m2_4.add_ref(
    ihp.cells.straight(
```

```
        length=abs(end_point - start_point),
        cross_section="metal2_routing",
        width=1.11,  # manual measure of the height of the via
    )
)

straight_ref.connect("e1", gate_via_refs_pmos[23].ports["top"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_m1_m2 = ihp.cells.via_stack(
    top_layer="Metal2",
    bottom_layer="Metal1",
    vn_columns=3,
    vn_rows=1,
)
via_m1_m2_ref = m2_4.add_ref(via_m1_m2)
via_m1_m2_ref.connect("top", straight_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

straight_ref.connect("e2", via_m1_m2_ref.ports["top"], allow_width_mismatch=True, allow_layer_mismatch=True)

via_gat_m2.ports["bottom"].orientation = 0
straight_ref.connect("e1", gate_via_refs_pmos[23].ports["top"], allow_width_mismatch=True, allow_layer_mismatch=True)
straight_ref = m2_4.add_ref(
    ihp.cells.straight(
        length=abs(end_point - start_point),
        cross_section="metal2_routing",
        width=1.11,  # manual measure of the height of the via
    )
)
straight_ref.connect(
    "e1", gate_via_refs_pmos[0].ports["bottom"], allow_width_mismatch=True, allow_layer_mismatch=True
)

via_m1_m2_ref = m2_4.add_ref(via_m1_m2)
via_m1_m2_ref.connect("top", straight_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

# connect Gates 2-11 and 14-23 (20 gates) together with metal3
start_point = m2_4.ports["G_2"].center
end_point = m2_4.ports["G_23"].center
straight_ref = m2_4.add_ref(
    ihp.cells.straight(
        length=abs(end_point[0] - start_point[0]),
        cross_section="metal3_routing",
        width=max(
            gate_via_refs_pmos[1].xsize, gate_via_refs_pmos[1].ysize
        ),  # manual measure of the height of the via
    )
)
straight_ref.center = (
    (start_point[0] + end_point[0]) / 2,
    gate_via_refs_pmos[1].center[1],
)  # align with the via of the first M1 gate
```

```
# connect Gates 12-13 (2 gates) together with metal2
start_point = m2_4.ports["G_12"].center
end_point = m2_4.ports["G_13"].center
straight_ref = m2_4.add_ref(
    ihp.cells.straight(
        length=abs(end_point[0] - start_point[0]),
        cross_section="metal2_routing",
        width=0.6,  # manual measure of the height of the via
    )
)
straight_ref.center = (
    (start_point[0] + end_point[0]) / 2,
    gate_via_refs_pmos[11].center[1],
)  # align with the via of the first M3 gate

# connect source to vdd
# Source and drain fingers by their deterministic left-to-right port names
# (same column assignment as the nmos above).
m2_4_source_ports = [m2_4.ports[f"DS_{i}"] for i in (2, 3, 5, 7, 9, 11, 13, 17, 19, 21, 23, 25, 27, 28)]
m2_4_drain_ports = [m2_4.ports[f"DS_{i}"] for i in (4, 6, 8, 10, 12, 15, 18, 20, 22, 24, 26)]

# source vdd connection
source_straight = ihp.cells.straight(
    length=4.66,  # source finger to the center of the top guardring segment
    # (was |DS_1.y - DS_29.y| under the old port enumeration, whose
    # guard-rail pin ports are no longer emitted)
    cross_section="metal2_routing",
    width=0.26,  # manual measure of the height of the via
)
via_m1_m2 = ihp.cells.via_stack(
    top_layer="Metal2",
    bottom_layer="Metal1",
    vn_columns=13,
    vn_rows=1,
)
via_m1_m2.ports["bottom"].orientation = 180
via_m1_m2.ports["top"].orientation = 0
for p in m2_4_source_ports:
    p.orientation = 270  # orient ports to face the top
    via_m1_m2_ref = m2_4.add_ref(via_m1_m2)
    via_m1_m2_ref.connect("bottom", p, allow_width_mismatch=True, allow_layer_mismatch=True)
    source_straight_ref = m2_4.add_ref(source_straight)
    source_straight_ref.connect(
        "e1", via_m1_m2_ref.ports["top"], allow_width_mismatch=True, allow_layer_mismatch=True
    )

# connect drains to metal4
via_m1_m4 = ihp.cells.via_stack(
    top_layer="Metal4",
```

```
    bottom_layer="Metal1",
    vn_columns=1,
    vn_rows=13,
)
drain_via_refs_pmos = []
for p in m2_4_drain_ports:
    p.orientation = 90  # orient ports to face the top
    if p.name in ["DS_15"]:  # center drain column (x = 6.27), gets the shortened stack
        via_m1_m4_short = ihp.cells.via_stack(
            top_layer="Metal4",
            bottom_layer="Metal1",
            vn_columns=1,
            vn_rows=6,
        )
        via_m1_m4_short.ports["top"].orientation = 90
        drain_via_refs_pmos.append(m2_4.add_ref(via_m1_m4_short))
        drain_via_refs_pmos[-1].center = p.center
        drain_via_refs_pmos[-1].ymin = p.center[1] + 0.245  # manual measurement to align with metal1 below
    else:
        via_m1_m4.ports["bottom"].orientation = 90
        via_m1_m4.ports["top"].orientation = 270
        via_m1_m4_ref = m2_4.add_ref(via_m1_m4)
        via_m1_m4_ref.connect("bottom", p, allow_width_mismatch=True, allow_layer_mismatch=True)
        drain_via_refs_pmos.append(via_m1_m4_ref)

# horizontal drain connection for pmos
straight_drain_connection = ihp.cells.straight(
    length=abs(
        drain_via_refs_pmos[0].center[0] - drain_via_refs_pmos[-1].center[0]
    ),  # span from the first to the last drain via
    cross_section="metal4_routing",
    width=2.61,  #  manual measurement
)
straight_drain_connection_ref = m2_4.add_ref(straight_drain_connection)
straight_drain_connection_ref.xmin = drain_via_refs_pmos[0].center[0]  # leftmost drain via
straight_drain_connection_ref.ymin = drain_via_refs_pmos[0].ymin

m2_4.add_ports(gate_via_refs_pmos[6].ports, prefix="gate_M2_")  # G_7's via
m2_4.add_port(name="drain_connection_M2", port=drain_via_refs_pmos[8].ports["top"])  # DS_22 column (x = 9.33)
m2_4.add_port(name="drain_connection_M4", port=drain_via_refs_pmos[5].ports["top"])
m2_4.add_port(
    name="gate_connection_M4",
    center=(
        (gate_via_refs_pmos[11].center[0] + gate_via_refs_pmos[12].center[0]) / 2,
        gate_via_refs_pmos[11].center[1],
    ),
    cross_section="metal2_routing",
    orientation=90,
    port_type="electrical",
)
```

```
m2_4_ref = c_output.add_ref(m2_4)

spacing_between_nmos_pmos = NMOS_PMOS_SPACING  # manual measurement of the spacing between the drain vias of M1 and M2, used to set the length of the connection
drain_connection_m1_m2 = c_output.add_ref(
    ihp.cells.straight(
        length=spacing_between_nmos_pmos,  # manual measurement, used to set the distance between M1 and M2
        cross_section="metal4_routing",
        width=4.28,  #  manual measurement
    )
)

drain_connection_m1_m2.connect(
    "e1", m1_3_ref.ports["drain_connection_M1"], allow_width_mismatch=True, allow_layer_mismatch=True
)
m2_4_ref.connect(
    "drain_connection_M2", drain_connection_m1_m2.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True
)

# drain connection for replica circuit
routing_drain_m1_m3 = gf.routing.route_bundle_electrical(
    component=c_output,
    ports1=[m1_3_ref.ports["drain_connection_M3"]],
    ports2=[m2_4_ref.ports["drain_connection_M4"]],
    route_width=ROUTE_WIDTH_REPLICA,
    layer=ihp.tech.LAYER.Metal4drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# vias for resistor xr2 and xr4 connections
via_m1_m2 = ihp.cells.via_stack(
    top_layer="Metal2",
    bottom_layer="Metal1",
    vn_columns=2,
    vn_rows=2,
)
via_m1_m2.ports["top"].orientation = 90
via_m1_m3 = ihp.cells.via_stack(
    top_layer="Metal3",
    bottom_layer="Metal1",
    vn_columns=2,
    vn_rows=2,
)
via_m1_m4 = ihp.cells.via_stack(
    top_layer="Metal4",
    bottom_layer="Metal1",
    vn_columns=2,
```

```
    vn_rows=2,
)

xr4 = ihp.cells.rppd(width=RPPD_WIDTH, length=RPPD_LENGTH)
xr4_ref = c_output.add_ref(xr4).rotate(90)
xr4_ref.center = m1_3_ref.ports["DS_18"].center
xr4_ref.movex(0.01)  # manual measurement to align with the gate connection
xr4_ref.ymax = m1_3_ref.ymin - 2  # manual measurement to set the spacing between the nmos and the resistors

via_m1_m4_ref = c_output.add_ref(via_m1_m4)
via_m1_m4_ref.connect("bottom", xr4_ref.ports["e1"], allow_width_mismatch=True, allow_layer_mismatch=True)
c_output.add_port(name="vref", port=via_m1_m4_ref.ports["top"])


via_m1_m2.ports["top"].orientation = 180
via_m1_m2_ref = c_output.add_ref(via_m1_m2)
via_m1_m2_ref.connect("bottom", xr4_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)
c_output.add_ports(via_m1_m2_ref.ports, prefix="XR4_")

xr2 = ihp.cells.rppd(width=RPPD_WIDTH, length=RPPD_LENGTH)
xr2_ref = c_output.add_ref(xr2).rotate(90)
xr2_ref.ymax = xr4_ref.ymin - 1  # manual measurement to set the spacing between the two resistors to 0
xr2_ref.xmax = xr4_ref.xmax

via_m1_m4_ref = c_output.add_ref(via_m1_m4)
via_m1_m4_ref.connect("bottom", xr2_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

c_output.add_port(name="vout", port=via_m1_m4_ref.ports["top"])
via_m1_m3_ref = c_output.add_ref(via_m1_m3)
via_m1_m3_ref.connect("bottom", xr2_ref.ports["e1"], allow_width_mismatch=True, allow_layer_mismatch=True)


# bias2
route_gates_m3_m4 = gf.routing.route_bundle_electrical(
    component=c_output,
    ports1=[m1_3_ref.ports["gate_connection_M3"]],
    ports2=[via_m1_m2_ref.ports["top"]],
    route_width=ROUTE_WIDTH_BIAS,
    layer=ihp.tech.LAYER.Metal2drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
    steps=[
        {"dy": -2.46, "dx": 0.5},
        {"dx": 1},
    ],
)

# node bb_int routing
```

```
route_gates_m1_m2 = gf.routing.route_bundle_electrical(
    component=c_output,
    ports1=[m1_3_ref.ports["gate_M1_top"]],
    ports2=[m2_4_ref.ports["gate_M2_top"]],
    route_width=ROUTE_WIDTH_GATE,
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=0.5,
    end_straight_length=0.5,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
route_gates_m1_m2 = gf.routing.route_bundle_electrical(
    component=c_output,
    ports1=[m1_3_ref.ports["gate_M1_top"]],
    ports2=[xr2_ref.ports["e1"]],
    route_width=ROUTE_WIDTH_GATE,
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=0.5,
    end_straight_length=0.5,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

c_output.add_ports(m1_3_ref.ports, prefix="M1_")
c_output.add_ports(m2_4_ref.ports, prefix="M2_")
output_stage_ref = c.add_ref(c_output.rotate(90))
output_stage_ref.center = c2_ref.center
output_stage_ref.ymin = c2_ref.ymin
output_stage_ref.move((17.99 + 1.64, 9.02 + 5.23))

# create capacitor array for cap 3
c3 = gf.Component("C3")
# 28 cmim capacitors in columns of 3 x 5, 2 x 4 and 1 x 5, column pitch 0.52 + 11.2 for the TM1 spacing
spacing_between_caps = CMIM_SPACING_TM1
left_col = c3.add_ref(
    gf.components.array(
        component=cmim,
        columns=3,
        rows=5,
        column_pitch=spacing_between_caps + CMIM_PITCH,
        row_pitch=-(spacing_between_caps + CMIM_PITCH),
    )
)
middle_col = c3.add_ref(
    gf.components.array(
        component=cmim,
```

```
        columns=2,
        rows=4,
        column_pitch=spacing_between_caps + CMIM_PITCH,
        row_pitch=-(spacing_between_caps + CMIM_PITCH),
    )
)
right_col = c3.add_ref(
    gf.components.array(
        component=cmim,
        columns=1,
        rows=5,
        column_pitch=spacing_between_caps + CMIM_PITCH,
        row_pitch=-(spacing_between_caps + CMIM_PITCH),
    )
)
left_col.xmax = middle_col.xmin - spacing_between_caps  # min spacing according to design rules 5.17
right_col.xmin = middle_col.xmax + spacing_between_caps  # min spacing according to design rules 5.17
c3.add_ports(left_col.ports, prefix="LC_")
c3.add_ports(middle_col.ports, prefix="MC_")
c3.add_ports(right_col.ports, prefix="RC_")

# orient the outer most port of for horizontal connection
ihp.cells.utils.change_port_orientation(c3, ["LC_T_1_1", "LC_T_2_1", "LC_T_3_1", "LC_T_4_1", "LC_T_5_1"], 0)
ihp.cells.utils.change_port_orientation(c3, ["LC_B_1_1", "LC_B_2_1", "LC_B_3_1", "LC_B_4_1", "LC_B_5_1"], 0)

ihp.cells.utils.change_port_orientation(c3, ["RC_T_1_1", "RC_T_2_1", "RC_T_3_1", "RC_T_4_1", "LC_T_5_3"], 180)
ihp.cells.utils.change_port_orientation(c3, ["RC_B_1_1", "RC_B_2_1", "RC_B_3_1", "RC_B_4_1", "LC_B_5_3"], 180)

c3_connection_t = gf.routing.route_bundle_electrical(
    component=c3,
    ports1=[
        c3.ports["LC_T_1_1"],
        c3.ports["LC_T_2_1"],
        c3.ports["LC_T_3_1"],
        c3.ports["LC_T_4_1"],
        c3.ports["LC_T_5_1"],
    ],
    ports2=[
        c3.ports["RC_T_1_1"],
        c3.ports["RC_T_2_1"],
        c3.ports["RC_T_3_1"],
        c3.ports["RC_T_4_1"],
        c3.ports["LC_T_5_3"],
    ],
    route_width=cap_connection_thickness,
    layer=ihp.tech.LAYER.TopMetal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
```

```
)

c3_connection_b = gf.routing.route_bundle_electrical(
    component=c3,
    ports1=[
        c3.ports["LC_B_1_1"],
        c3.ports["LC_B_2_1"],
        c3.ports["LC_B_3_1"],
        c3.ports["LC_B_4_1"],
        c3.ports["LC_B_5_1"],
    ],
    ports2=[
        c3.ports["RC_B_1_1"],
        c3.ports["RC_B_2_1"],
        c3.ports["RC_B_3_1"],
        c3.ports["RC_B_4_1"],
        c3.ports["LC_B_5_3"],
    ],
    route_width=cap_connection_thickness,
    layer=ihp.tech.LAYER.Metal5drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# orient top row ports to face downwards to connect to bottom row ports
ihp.cells.utils.change_port_orientation(
    c3, ["LC_T_1_1", "LC_T_1_2", "LC_T_1_3", "MC_T_1_1", "MC_T_1_2", "RC_T_1_1"], 270
)
ihp.cells.utils.change_port_orientation(
    c3, ["LC_B_1_1", "LC_B_1_2", "LC_B_1_3", "MC_B_1_1", "MC_B_1_2", "RC_B_1_1"], 270
)

# orient bottom row ports to face downwards to connect to top row ports
ihp.cells.utils.change_port_orientation(
    c3, ["LC_T_5_1", "LC_T_5_2", "LC_T_5_3", "MC_T_4_1", "MC_T_4_2", "RC_T_5_1"], 90
)
ihp.cells.utils.change_port_orientation(
    c3, ["LC_B_5_1", "LC_B_5_2", "LC_B_5_3", "MC_B_4_1", "MC_B_4_2", "RC_B_5_1"], 90
)

c3_connection_top = gf.routing.route_bundle_electrical(
    component=c3,
    ports1=[
        c3.ports["LC_T_1_1"],
        c3.ports["LC_T_1_2"],
        c3.ports["LC_T_1_3"],
        c3.ports["MC_T_1_1"],
        c3.ports["MC_T_1_2"],
```

```
            c3.ports["RC_T_1_1"],
        ],
        ports2=[
            c3.ports["LC_T_5_1"],
            c3.ports["LC_T_5_2"],
            c3.ports["LC_T_5_3"],
            c3.ports["MC_T_4_1"],
            c3.ports["MC_T_4_2"],
            c3.ports["RC_T_5_1"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    c3_connection_bottom = gf.routing.route_bundle_electrical(
        component=c3,
        ports1=[
            c3.ports["LC_B_1_1"],
            c3.ports["LC_B_1_2"],
            c3.ports["LC_B_1_3"],
            c3.ports["MC_B_1_1"],
            c3.ports["MC_B_1_2"],
            c3.ports["RC_B_1_1"],
        ],
        ports2=[
            c3.ports["LC_B_5_1"],
            c3.ports["LC_B_5_2"],
            c3.ports["LC_B_5_3"],
            c3.ports["MC_B_4_1"],
            c3.ports["MC_B_4_2"],
            c3.ports["RC_B_5_1"],
        ],
        route_width=cap_connection_thickness,
        layer=ihp.tech.LAYER.Metal5drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )

    # orient top row ports to face downwards to connect to face the middle for connection purposes
    ihp.cells.utils.change_port_orientation(c3, ["MC_T_4_1", "MC_T_4_2", "LC_T_5_1", "LC_T_5_2", "LC_T_5_3"], 270)
    # orient top row ports to face downwards to connect to face c2 for connection purposes
    ihp.cells.utils.change_port_orientation(
        c3, ["MC_B_4_1", "MC_B_4_2", "LC_B_5_1", "LC_B_5_2", "LC_B_5_3", "RC_B_5_1"], 270
    )
```

```
c3_ref = c.add_ref(c3)

c3_ref.rotate(-90)
c3_ref.xmin = c2_ref.xmax + CMIM_SPACING_TM1  # manual measurement to set the spacing between c2 and c3
c3_ref.ymin = c2_ref.ymin

c_fill = ihp.cells.cmim(width=CMIM_FILL_WIDTH, length=CMIM_FILL_LENGTH)
c_fill.ports["B"].orientation = 0
c_fill_1_ref = c.add_ref(c_fill)
c_fill_2_ref = c.add_ref(c_fill)

straight_m5 = ihp.cells.straight(
    length=10.12,
    cross_section="metal5_routing",
    width=cap_connection_thickness,  # manual measurement to align with the via
)
straight_m5_ref = c.add_ref(straight_m5)
straight_m5_ref.connect("e1", c3_ref.ports["MC_B_4_1"], allow_width_mismatch=True, allow_layer_mismatch=True)
c_fill_1_ref.connect("B", straight_m5_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

straight_m5_ref = c.add_ref(straight_m5)
straight_m5_ref.connect("e1", c3_ref.ports["MC_B_4_2"], allow_width_mismatch=True, allow_layer_mismatch=True)
c_fill_2_ref.connect("B", straight_m5_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

route_c2_c3_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[c2_ref.ports["TR_B_1_1"], c2_ref.ports["TR_B_1_2"], c2_ref.ports["TR_B_1_3"], c2_ref.ports["TR_B_1_6"]],
    ports2=[c3_ref.ports["LC_B_5_1"], c3_ref.ports["LC_B_5_2"], c3_ref.ports["LC_B_5_3"], c3_ref.ports["RC_B_5_1"]],
    route_width=cap_connection_thickness,  # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

# connect nmos source/bulk to vss
via_m1_m5 = ihp.cells.via_stack(
    top_layer="Metal5",
    bottom_layer="Metal1",
    vn_columns=3,
    vn_rows=35,
)
via_m1_m5.ports["bottom"].orientation = 0
via_m1_m5_ref = c.add_ref(via_m1_m5)
via_m1_m5_ref.connect(
    "bottom", output_stage_ref.ports["M1_DS_16"], allow_width_mismatch=True, allow_layer_mismatch=True
)
```

```
# connect nmos source/bulk to vdd
via_m1_tm1 = ihp.cells.via_stack(
    top_layer="TopMetal1",
    bottom_layer="Metal1",
    vn_columns=3,
    vn_rows=35,
    vt1_columns=1,
    vt1_rows=17,
)
via_m1_tm1.ports["bottom"].orientation = 90
via_m1_tm1_ref = c.add_ref(via_m1_tm1)
via_m1_tm1_ref.connect(
    "bottom", output_stage_ref.ports["M2_DS_14"], allow_width_mismatch=True, allow_layer_mismatch=True
)

tm1_straight = ihp.cells.straight(
    length=abs(output_stage_ref.ports["M2_DS_14"].center[0] - c3_ref.ports["MC_T_4_1"].center[0]),
    cross_section="topmetal1_routing",
    width=2,  # manual measurement to align with the via
)
tm1_straight_ref = c.add_ref(tm1_straight)
tm1_straight_ref.connect("e1", c3_ref.ports["MC_T_4_1"], allow_width_mismatch=True, allow_layer_mismatch=True)
tm1_straight_ref = c.add_ref(tm1_straight)
tm1_straight_ref.connect("e1", c3_ref.ports["MC_T_4_2"], allow_width_mismatch=True, allow_layer_mismatch=True)

# connect c2 to bias1 network
via_m1_tm1 = ihp.cells.via_stack(
    top_layer="TopMetal1",
    bottom_layer="Metal1",
    vn_columns=3,
    vn_rows=4,
    vt1_columns=1,
    vt1_rows=2,
)
via_m1_tm1.ports["top"].orientation = 0
via_m1_tm1.ports["bottom"].orientation = 180
via_m1_tm1_ref = c.add_ref(via_m1_tm1).rotate(90)
via_m1_tm1_ref.xmax = c2_ref.xmax - 4.55
via_m1_tm1_ref.ymax = c3_ref.ymin - 5  # min M5 spacing

route_c2_bias = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[c2_ref.ports["TR_T_1_6"]],
    ports2=[via_m1_tm1_ref.ports["top"]],
    route_width=max(via_m1_tm1_ref.xsize, via_m1_tm1_ref.ysize),  # manual change to fit width
    layer=ihp.tech.LAYER.TopMetal1drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
```

```
    )

    route = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[via_m1_tm1_ref.ports["bottom"]],
        ports2=[xr1_ref.ports["e2"]],
        route_width=ROUTE_WIDTH_BIAS_M1,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
    via_m1_tm1.ports["bottom"].orientation = 270
    xr3 = ihp.cells.rhigh(width=RHIGH_WIDTH, length=RHIGH_LENGTH)
    xr3_ref = c.add_ref(xr3).rotate(90)
    xr3_ref.center = via_m1_tm1_ref.center
    xr3_ref.xmin = c3_ref.xmin
    route_xr3 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[via_m1_tm1_ref.ports["bottom"]],
        ports2=[xr3_ref.ports["e1"]],
        route_width=ROUTE_WIDTH_BIAS_M1,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
    via_m1_tm1 = ihp.cells.via_stack(
        top_layer="TopMetal1",
        bottom_layer="Metal1",
        vn_columns=4,
        vn_rows=3,
        vt1_columns=2,
        vt1_rows=1,
    )
    via_m1_tm1.ports["bottom"].orientation = 0
    via_m1_tm1_ref = c.add_ref(via_m1_tm1)
    via_m1_tm1_ref.connect("bottom", xr3_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

    route_c2_bias = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[c3_ref.ports["RC_T_5_1"]],
        ports2=[via_m1_tm1_ref.ports["top"]],
        route_width=max(via_m1_tm1_ref.xsize, via_m1_tm1_ref.ysize),  # manual change to fit width
        layer=ihp.tech.LAYER.TopMetal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
```

```
    separation=0,
)

# bb_int connect D1 to network
via_m1_m3 = ihp.cells.via_stack(
    top_layer="Metal3",
    bottom_layer="Metal1",
    vn_columns=6,
    vn_rows=3,
)
via_m1_m3.ports["top"].orientation = 90
via_m1_m3.ports["bottom"].orientation = 270
via_m1_m3_ref = c.add_ref(via_m1_m3)
via_m1_m3_ref.connect("bottom", d1_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

# bb_int
routing_bb_int = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[output_stage_ref.ports["M1_gate_M1_top"]],
    ports2=[via_m1_m3_ref.ports["top"]],
    route_width=ROUTE_WIDTH_GATE,
    layer=ihp.tech.LAYER.Metal3drawing,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    end_straight_length=2,
    auto_taper=False,
    separation=0,
    steps=[
        {"dx": 3.65, "dy": -1},  # strange behavior without the dy direction
        {"dy": -6},
        {"dx": -4, "dy": 0},
    ],
)

# bias2 connect D2 to network
via_m1_m2 = ihp.cells.via_stack(
    top_layer="Metal2",
    bottom_layer="Metal1",
    vn_columns=6,
    vn_rows=3,
)
via_m1_m2.ports["top"].orientation = 90
via_m1_m2.ports["bottom"].orientation = 270
via_m1_m2_ref = c.add_ref(via_m1_m2)
via_m1_m2_ref.connect("bottom", d2_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

# bias2 connection from gate of M3 to D2
routing_bias2 = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[output_stage_ref.ports["M1_gate_connection_M3"]],
```

```
        ports2=[via_m1_m2_ref.ports["top"]],
        route_width=ROUTE_WIDTH_BIAS,
        layer=ihp.tech.LAYER.Metal2drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        end_straight_length=3,
        auto_taper=False,
        separation=0,
        steps=[
            {"dx": 2.46, "dy": -1},  # strange behavior without the dy direction
            {"dy": -1},
            {"dx": 2.17, "dy": 0},
            {"dy": -2},
        ],
    )
    # bias2 connection from gate of M4 to D2
    routing_bias2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[output_stage_ref.ports["M2_gate_connection_M4"]],
        ports2=[via_m1_m2_ref.ports["top"]],
        route_width=ROUTE_WIDTH_BIAS,
        layer=ihp.tech.LAYER.Metal2drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        end_straight_length=3,
        auto_taper=False,
        separation=0,
        steps=[
            {"dx": -2.46, "dy": -1},  # strange behavior without the dy direction
            {"dy": -1},
            {"dx": -2.21, "dy": 0},
            {"dy": -2},
        ],
    )

    # SBD D1 D2 sub connection
    route_d1_d2 = gf.routing.route_bundle_electrical(
        component=c,
        ports1=[d1_ref.ports["e3"]],
        ports2=[d2_ref.ports["e1"]],
        route_width=ROUTE_WIDTH_SBD_SUB,
        layer=ihp.tech.LAYER.Metal1drawing,
        allow_layer_mismatch=True,
        allow_width_mismatch=True,
        auto_taper=False,
        separation=0,
    )
    via_m1_m3 = ihp.cells.via_stack(
        top_layer="Metal3",
        bottom_layer="Metal1",
```

```
    vn_columns=1,
    vn_rows=5,
)
via_m1_m3.ports["bottom"].orientation = 0
via_m1_m3.ports["top"].orientation = 180
via_m1_m3_ref = c.add_ref(via_m1_m3)
via_m1_m3_ref.connect("bottom", d2_ref.ports["e3"], allow_width_mismatch=True, allow_layer_mismatch=True)

straight_m3 = ihp.cells.straight(
    length=18,
    cross_section="metal3_routing",
    width=max(via_m1_m3_ref.xsize, via_m1_m3_ref.ysize),
)
straight_m3_ref = c.add_ref(straight_m3)
straight_m3_ref.connect("e1", via_m1_m3_ref.ports["top"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_m3_m5 = ihp.cells.via_stack(
    top_layer="Metal5",
    bottom_layer="Metal3",
    vn_columns=1,
    vn_rows=5,
)
via_m3_m5.ports["bottom"].orientation = 180
via_m3_m5_ref = c.add_ref(via_m3_m5)
via_m3_m5_ref.connect("bottom", straight_m3_ref.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)

schottky.ports["e1"].orientation = 90
# connect D1 and D2 sub connection together
schottky.ports["e3"].orientation = 90
route_sbd_d1_d2 = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[d1_ref.ports["e1"]],
    ports2=[d2_ref.ports["e3"]],
    route_width=ROUTE_WIDTH_SBD_GUARD,
    layer=ihp.tech.LAYER.Metal1drawing,
    start_straight_length=2.85,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

straight_m3 = ihp.cells.straight(
    length=10,
    cross_section="metal3_routing",
    width=2,
)
straight_m3_ref = c.add_ref(straight_m3)
straight_m3_ref.connect("e1", output_stage_ref.ports["vref"], allow_width_mismatch=True, allow_layer_mismatch=True)
rfin_connection = c.add_ref(ihp.cells.straight(length=RFIN_PD, cross_section="topmetal2_routing", width=7.2))
rfin_connection.connect("e1", via_tm1_tm2_ref.ports["top"], allow_width_mismatch=True)
```

```
# add ports to the power detector
vout_port = c.add_port(name="vout", port=output_stage_ref.ports["vout"])
c.ports["vout"].orientation = 90

vref_port = c.add_port(name="vref", port=straight_m3_ref.ports["e2"])
c.ports["vref"].center = (c.ports["vref"].center[0], c.ports["vref"].center[1] - 0.1)  # preventing notch
c.ports["vref"].orientation = 90

rfin_port = c.add_port(name="rfin", port=rfin_connection.ports["e2"])
c.ports["rfin"].orientation = 270

vss_port = c.add_port(name="vss", port=c3_ref.ports["LC_B_1_1"])
c.ports["vss"].orientation = 90

vdd_port = c.add_port(name="vdd", port=c3_ref.ports["LC_T_5_1"])
c.ports["vdd"].orientation = 90
c.pprint_ports()

gf.labels.add_port_labels(c, ports= [vout_port], layer = ihp.tech.LAYER.Metal4text)
gf.labels.add_port_labels(c, ports= [vref_port], layer = ihp.tech.LAYER.Metal3text)
gf.labels.add_port_labels(c, ports= [rfin_port], layer = ihp.tech.LAYER.TopMetal2text)
gf.labels.add_port_labels(c, ports= [vss_port], layer = ihp.tech.LAYER.Metal5text)
gf.labels.add_port_labels(c, ports= [vdd_port], layer = ihp.tech.LAYER.TopMetal1text)
gf.add_pins.add_pin_rectangle(c, port=vout_port, layer=ihp.tech.LAYER.Metal4pin)
gf.add_pins.add_pin_rectangle(c, port=vref_port, layer=ihp.tech.LAYER.Metal3pin)
gf.add_pins.add_pin_rectangle_inside(c, port=rfin_port, layer=ihp.tech.LAYER.TopMetal2pin)
gf.add_pins.add_pin_rectangle(c, port=vss_port, layer=ihp.tech.LAYER.Metal5pin)
gf.add_pins.add_pin_rectangle(c, port=vdd_port, layer=ihp.tech.LAYER.TopMetal1pin)

# No-fill exclusion zones for each metal layer
c.xmin = 0
c.ymin = 0
c_center = c.center
size_nofill_x = round(c.xsize,2)
size_nofill_y = round(c.ysize,2)
nofill = c.add_ref(gf.components.rectangle(size=(size_nofill_x + 2, size_nofill_y + 2), layer=ihp.tech.LAYER.Activnofill))
nofill.center = c_center
nofill.xmin = snap_to_grid(nofill.xmin)
nofill.ymin = snap_to_grid(nofill.ymin)

for nofill_layer in [
    ihp.tech.LAYER.GatPolynofill,
    ihp.tech.LAYER.Metal1nofill,
    ihp.tech.LAYER.Metal2nofill,
    ihp.tech.LAYER.Metal3nofill,
    ihp.tech.LAYER.Metal4nofill,
]:
    nofill = c.add_ref(gf.components.rectangle(size=(size_nofill_x, size_nofill_y), layer=nofill_layer, centered=True))
    # snap to grid
    nofill.center = c_center
```

```
            nofill.xmin = snap_to_grid(nofill.xmin)
            nofill.ymin = snap_to_grid(nofill.ymin)


        c.add_ref(
            gf.components.rectangle(size=(c1_ref.xsize + 40, c1_ref.ysize + 20), layer=ihp.tech.LAYER.Metal5nofill)
        ).center = c1_ref.center

        size_x = c2_ref.xsize + c3_ref.xsize
        size_y = c2_ref.ysize
        x = c2_ref.xmin + size_x / 2
        y = c2_ref.ymin + size_y / 2
        cap_m5_nofill = c.add_ref(
            gf.components.rectangle(size=(size_x - 10, size_y - 10), layer=ihp.tech.LAYER.Metal5nofill)
        ).center = (x, y)

        return c
```

3.9 Detector Placement, Bond Pads, and Routing

The four six-port outputs, ports `e2` and `e3` of BLC_1 and BLC_2, are extended by a short TopMetal2 line each. Together with the feed line of length `RFIN_PD` inside the detector cell, these lines bridge the gap `RFIN_GAP` between a BLC and its detector. The detectors are placed later by connecting their `rfin` port to the far end of these lines.

```
rfin_connection_pd1 = c.add_ref(ihp.cells.straight(
    length=RFIN_GAP - RFIN_PD,
    cross_section=signal_cross_section,
    width=blc_1_ref.ports["e2"].width,
))
rfin_connection_pd1.connect("e1", blc_1_ref.ports["e2"])

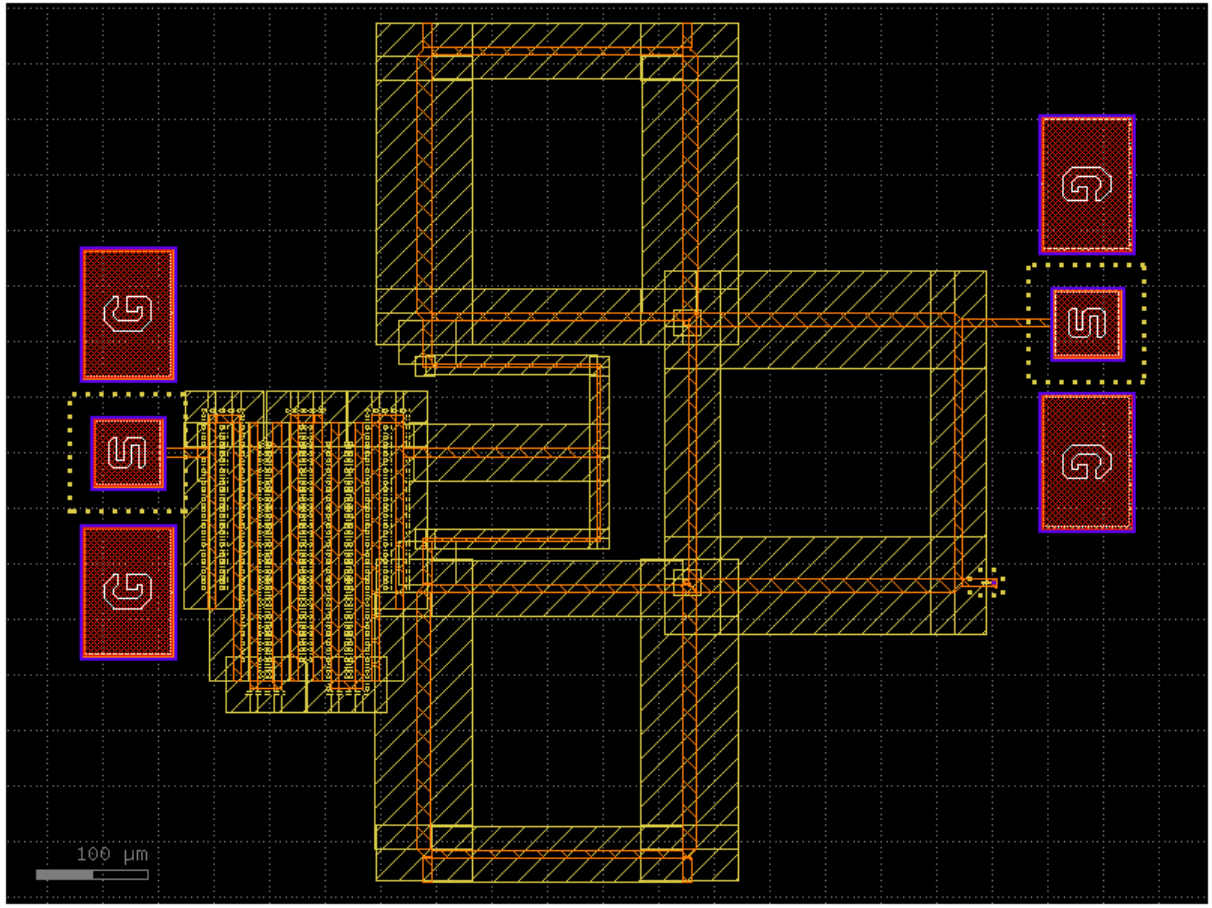
rfin_connection_pd2 = c.add_ref(ihp.cells.straight(
    length=RFIN_GAP - RFIN_PD,
    cross_section=signal_cross_section,
    width=blc_1_ref.ports["e3"].width,
))
rfin_connection_pd2.connect("e1", blc_1_ref.ports["e3"], allow_width_mismatch=True)

rfin_connection_pd3 = c.add_ref(ihp.cells.straight(
    length=RFIN_GAP - RFIN_PD,
    cross_section=signal_cross_section,
    width=blc_2_ref.ports["e3"].width,
))
rfin_connection_pd3.connect("e1", blc_2_ref.ports["e3"], allow_width_mismatch=True)

rfin_connection_pd4 = c.add_ref(ihp.cells.straight(
```

```
    length=RFIN_GAP - RFIN_PD,
    cross_section=signal_cross_section,
    width=blc_2_ref.ports["e2"].width,
))
rfin_connection_pd4.connect("e1", blc_2_ref.ports["e2"], allow_width_mismatch=True)

c.plot()
```



The detector outputs are low-frequency signals, so they leave the chip through bond pads for PCB bonding instead of probe pads. A row of seven bond pads (`SSGSGSS`) above and one below the core each serve two detectors, with the pinout VREF, VOUT, VSS, VDD, VSS, VOUT, VREF. The distance of a row to the core follows from `RFIN_GAP`, the height of the detector cell, and `PROBE_PD_GAP`, so it scales with the design frequency.

The next cell builds the detector cell once to get its height, moves the chip so that its lower left corner lies at the origin, and places the upper pad row centered above BLC_1. Metal5 no-fill regions around the VDD pad and on both sides of the row reduce the capacitance of the pads to the ground plane.

```
# the power detector height sets the distance of the bondpads to the six-port
pd = powdet_sbd()

# snap whole chip on grid
c.xmin = 0
c.ymin = 0

# probe pads top, positioned to clear PD extent
pd_height = pd.ysize
chip_center = c.center
probe_top = c.add_ref(ihp.cells.bondpad_array(
    config="SSGSGSS",
    signal_cross_section="topmetal1_routing",
    ground_cross_section="metal1_routing",
    ground_connection="psub",
))
probe_top.center = chip_center
probe_top.ymin = snap_to_grid(blc_1_ref.ymax + RFIN_GAP / 2 + pd_height + PROBE_PD_GAP)
probe_top.xmin = snap_to_grid(probe_top.xmin)

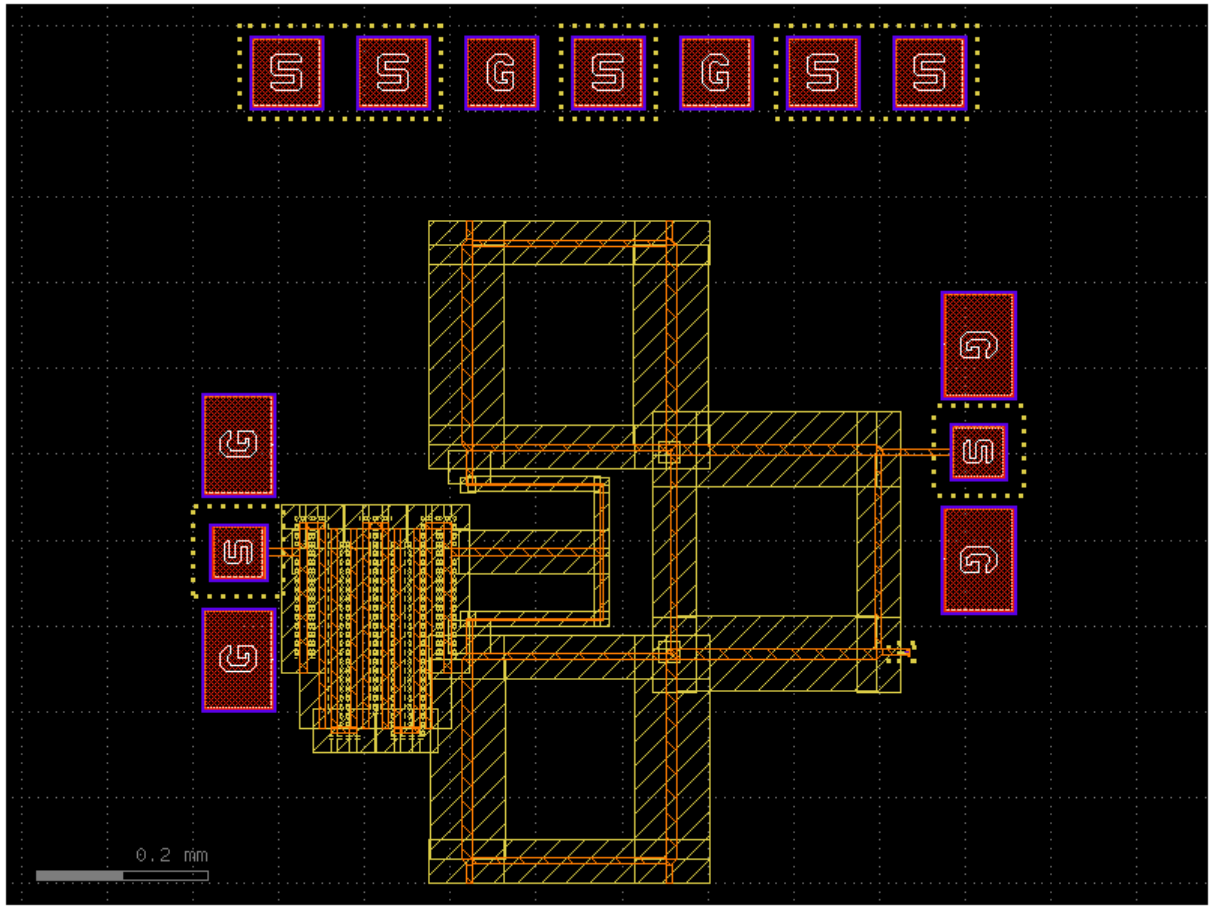
# no fill VDD
c.add_ref(gf.components.rectangle(size=(NOFILL_VDD_SIZE, NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill)).center = probe_top.center
c.add_ports(probe_top.ports, prefix="probe_top_")

# no fill top
no_fill_top_left = c.add_ref(gf.components.rectangle(size=(NOFILL_SIDE_WIDTH, NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill))
no_fill_top_left.xmin = probe_top.xmin - NOFILL_SIDE_OFFSET
no_fill_top_left.ymin = probe_top.ymin - NOFILL_SIDE_OFFSET

no_fill_top_right = c.add_ref(gf.components.rectangle(size=(NOFILL_SIDE_WIDTH, NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill))
no_fill_top_right.xmax = probe_top.xmax + NOFILL_SIDE_OFFSET
no_fill_top_right.ymin = probe_top.ymin - NOFILL_SIDE_OFFSET

c.plot()
```

name	width	orientation	layer	center	port_type
vout	0.61	90.0	Metal4drawing (50/0)	(7.37, 40.84)	electrical
vref	2.0	90.0	Metal3drawing (30/0)	(5.47, 48.68)	electrical
rfin	7.2	270.0	TopMetal2drawing (134/0)	(0.0, -18.400000000000002)	electrical
vss	11.2	90.0	Metal5drawing (67/0)	(70.32000000000001, 79.8)	electrical
vdd	10.08	90.0	TopMetal1drawing (126/0)	(23.44, 79.8)	electrical



The lower pad row is placed the same way below BLC_2, rotated by 180° so that its pads face the chip.

```
# probe pads bottom
probe_bottom = c.add_ref(ihp.cells.bondpad_array(
    config="SSGSGSS",
    signal_cross_section="topmetal1_routing",
    ground_cross_section="metal1_routing",
```

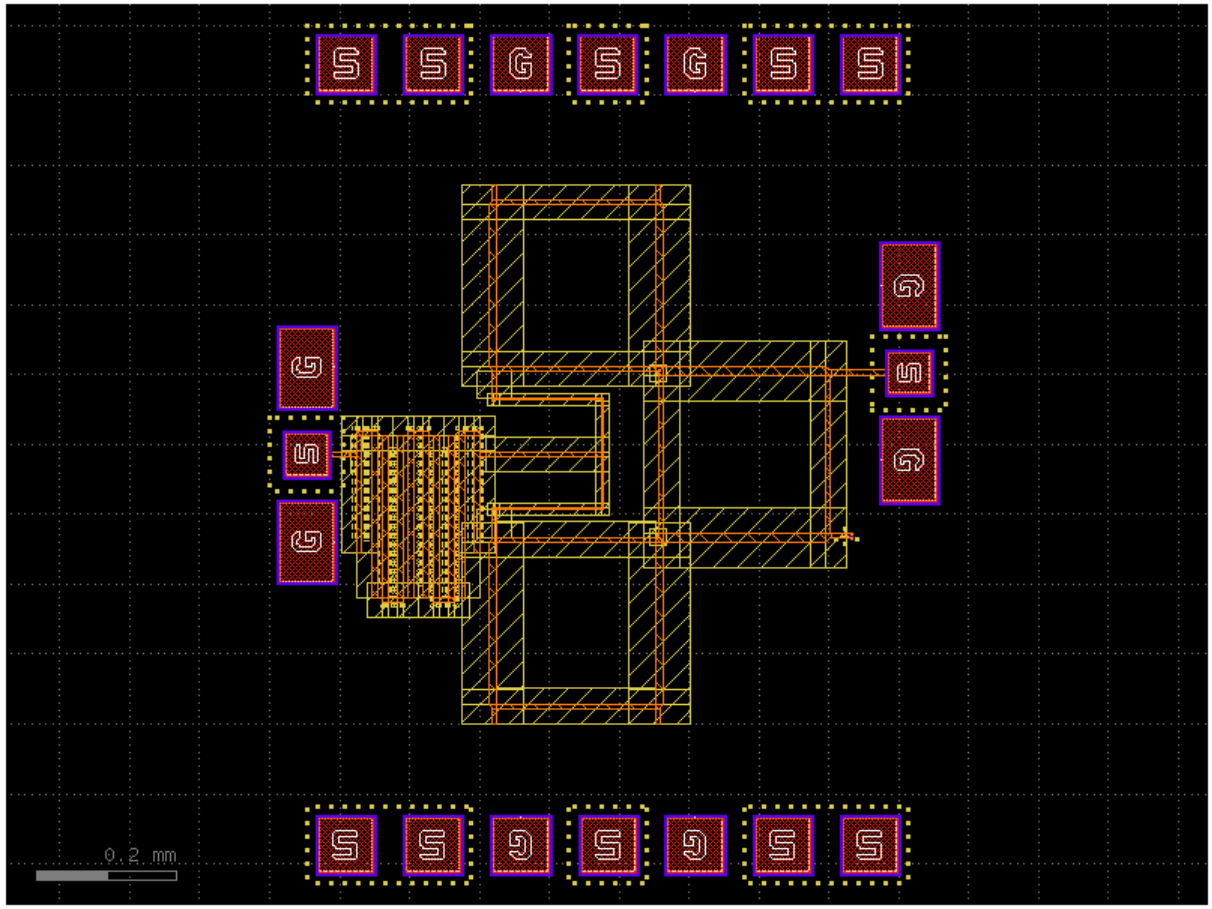
```
    ground_connection="psub",
)).rotate(180)
probe_bottom.center = chip_center
probe_bottom.ymax = snap_to_grid(blc_2_ref.ymin - RFIN_GAP / 2 - pd_height - PROBE_PD_GAP)
probe_bottom.xmin = snap_to_grid(probe_bottom.xmin)

# no fill VDD
c.add_ref(gf.components.rectangle(size=(NOFILL_VDD_SIZE, NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill)).center = probe_bottom.center
c.add_ports(probe_bottom.ports, prefix="probe_bottom_")

# no fill bottom
no_fill_bottom_left = c.add_ref(gf.components.rectangle(size=(NOFILL_SIDE_WIDTH, NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill))
no_fill_bottom_left.xmin = probe_bottom.xmin - NOFILL_SIDE_OFFSET
no_fill_bottom_left.ymin = probe_bottom.ymin - NOFILL_SIDE_OFFSET

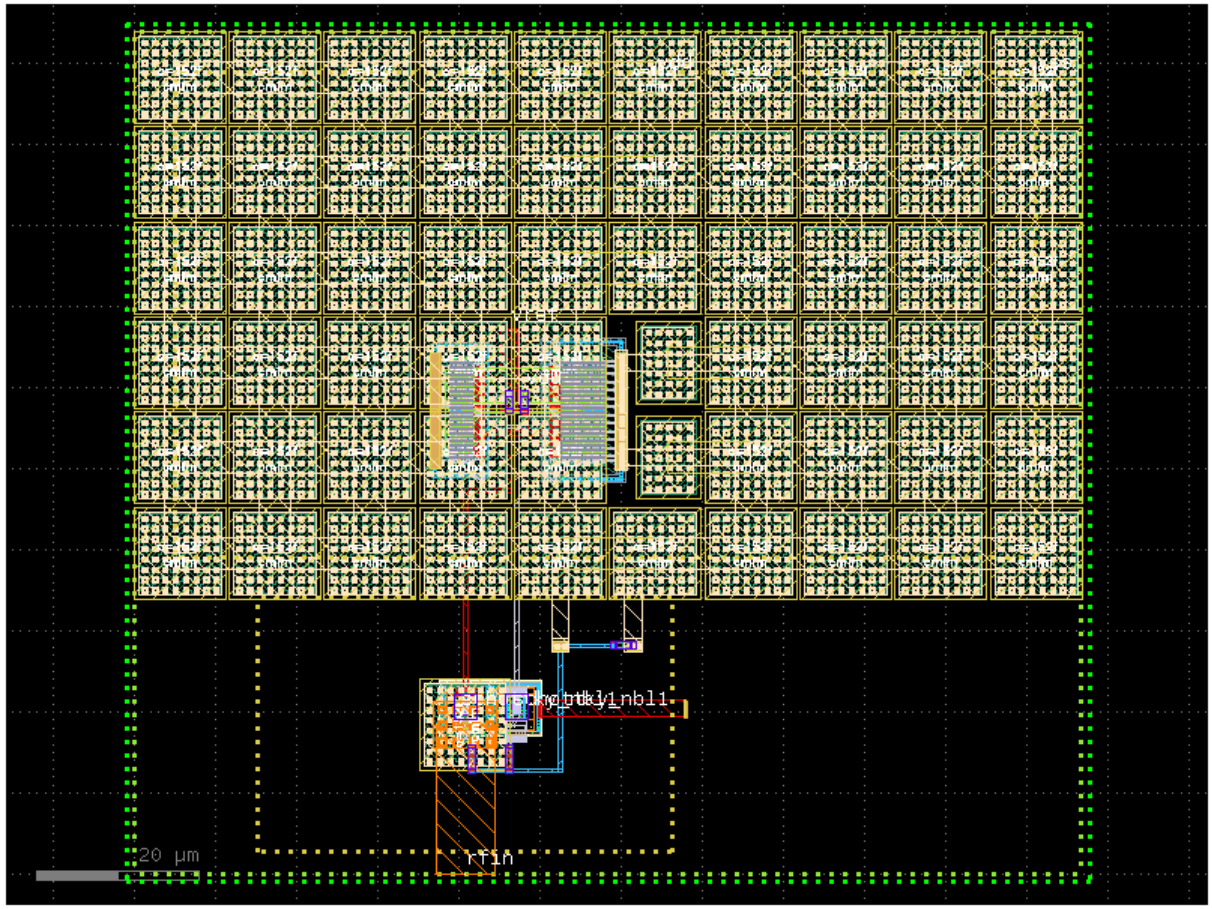
no_fill_bottom_right = c.add_ref(gf.components.rectangle(size=(NOFILL_SIDE_WIDTH, NOFILL_VDD_SIZE), layer=ihp.tech.LAYER.Metal5nofill))
no_fill_bottom_right.xmax = probe_bottom.xmax + NOFILL_SIDE_OFFSET
no_fill_bottom_right.ymin = probe_bottom.ymin - NOFILL_SIDE_OFFSET

c.plot()
```



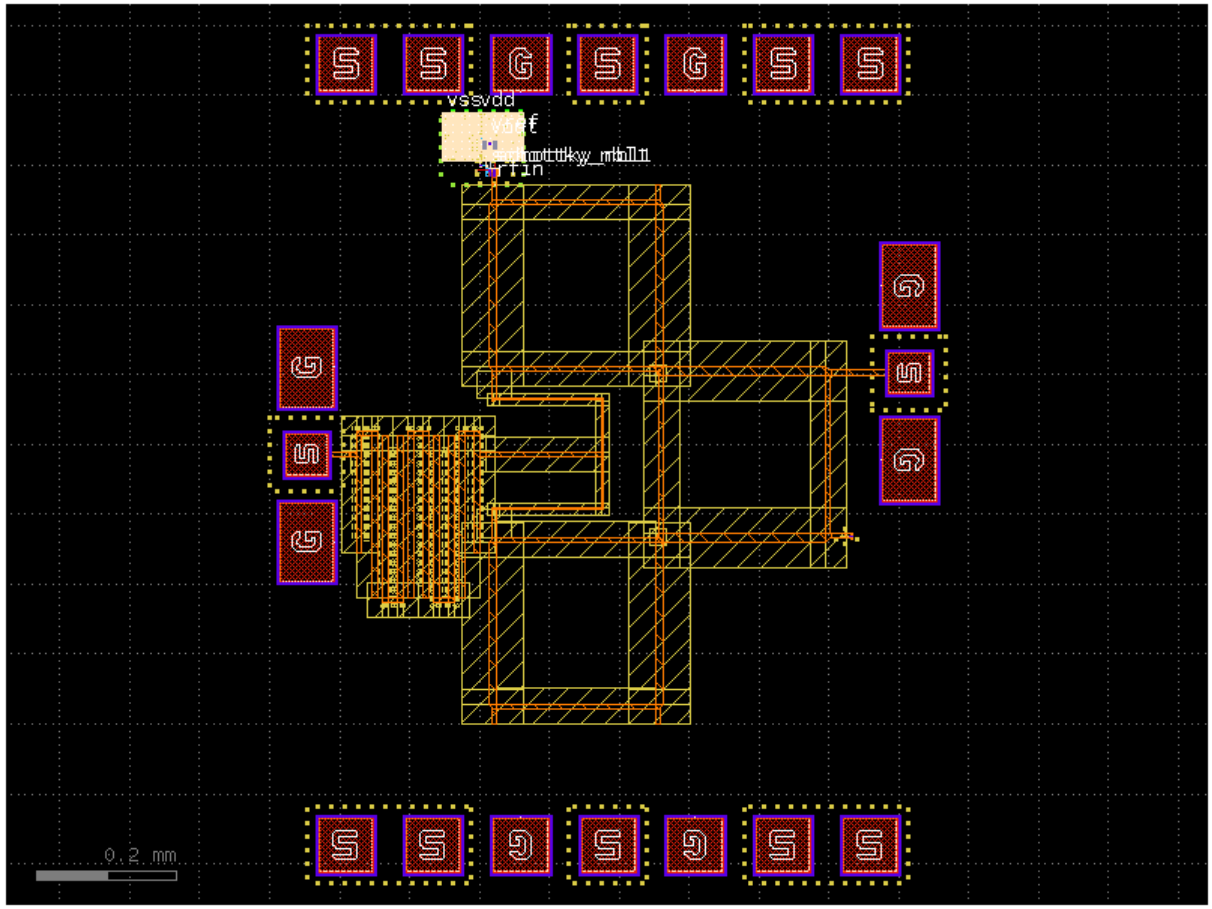
The plot shows the detector cell that all four channels use.

```
pd.plot()
```



The first detector, PD1, is a mirrored reference of the cell, placed by attaching its `rfin` port to the line at port `e2` of BLC_1. Its position follows from the RF network and moves with it when `f` changes.

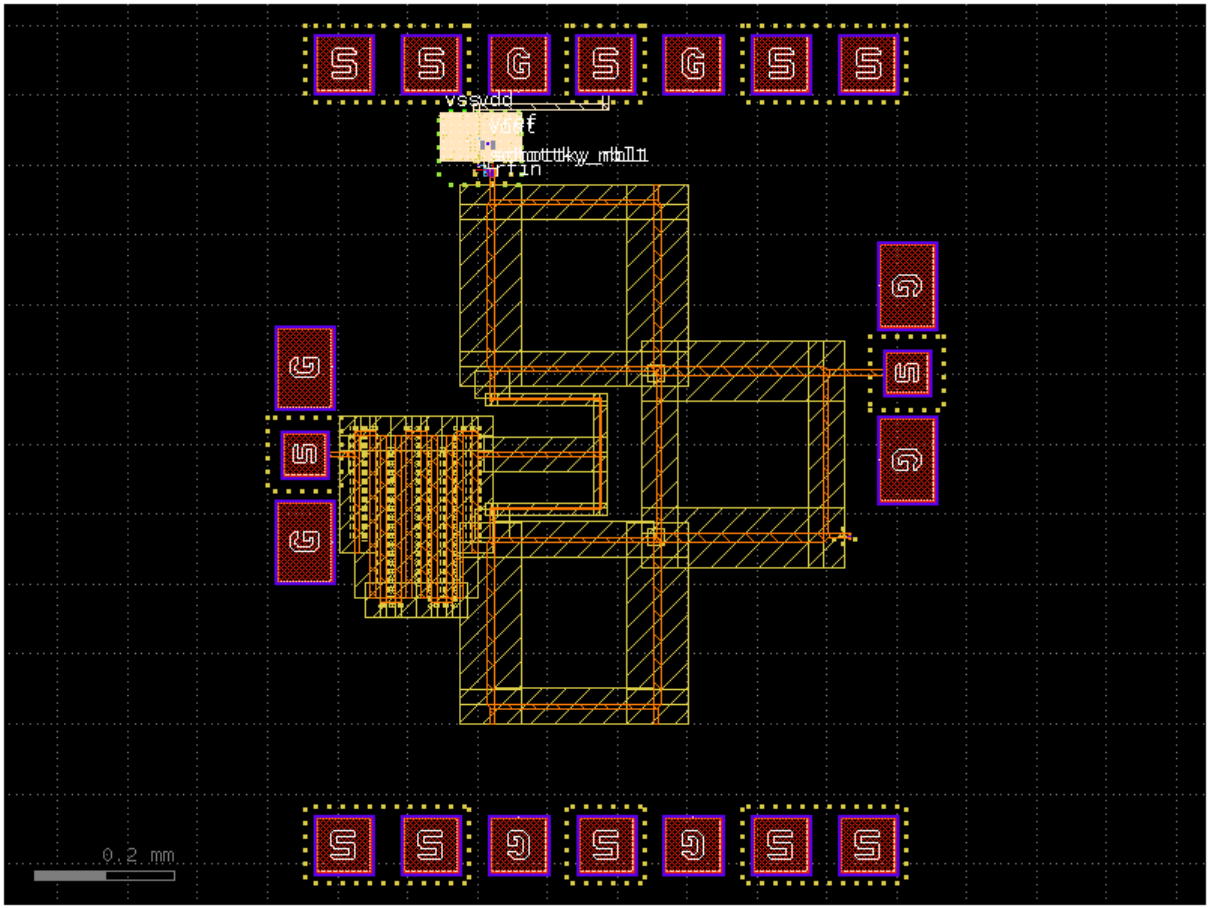
```
pd1_ref = c.add_ref(pd).mirror_x()
pd1_ref.connect("rfin", rfin_connection_pd1.ports["e2"], allow_width_mismatch=True)
c.plot()
```



The supply and output nets of PD1 are routed to the upper pad row with the electrical bundle router of GDSFactory, starting with `vdd` on TopMetal1.

```
route_pd1_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vdd"]],
    ports2=[probe_top.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
```

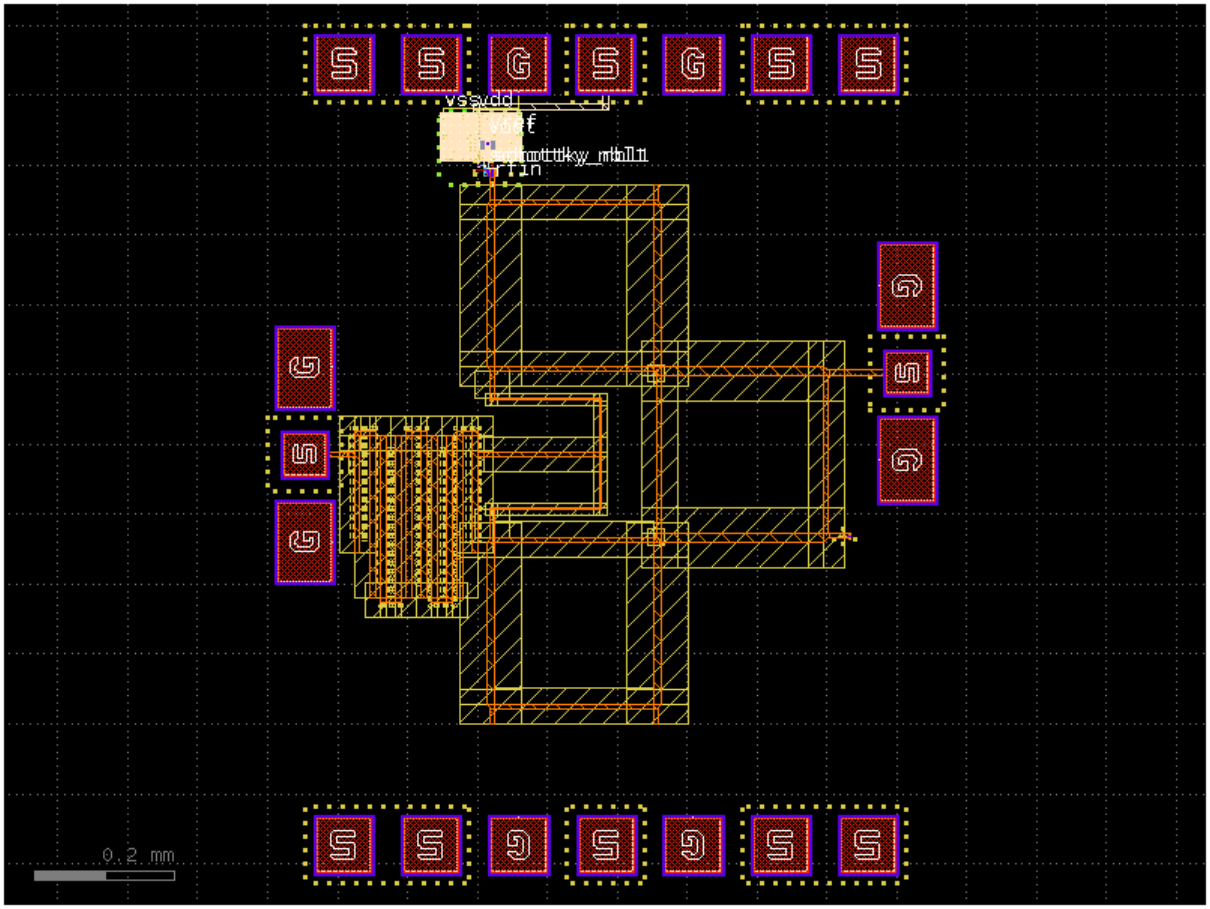
```
)
c.plot()
```



`vss` follows on Metal5, with a straight lead of 10 µm so that the route does not cross the detector itself.

```
route_pd1_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vss"]],
    ports2=[probe_top.ports["e3"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
```

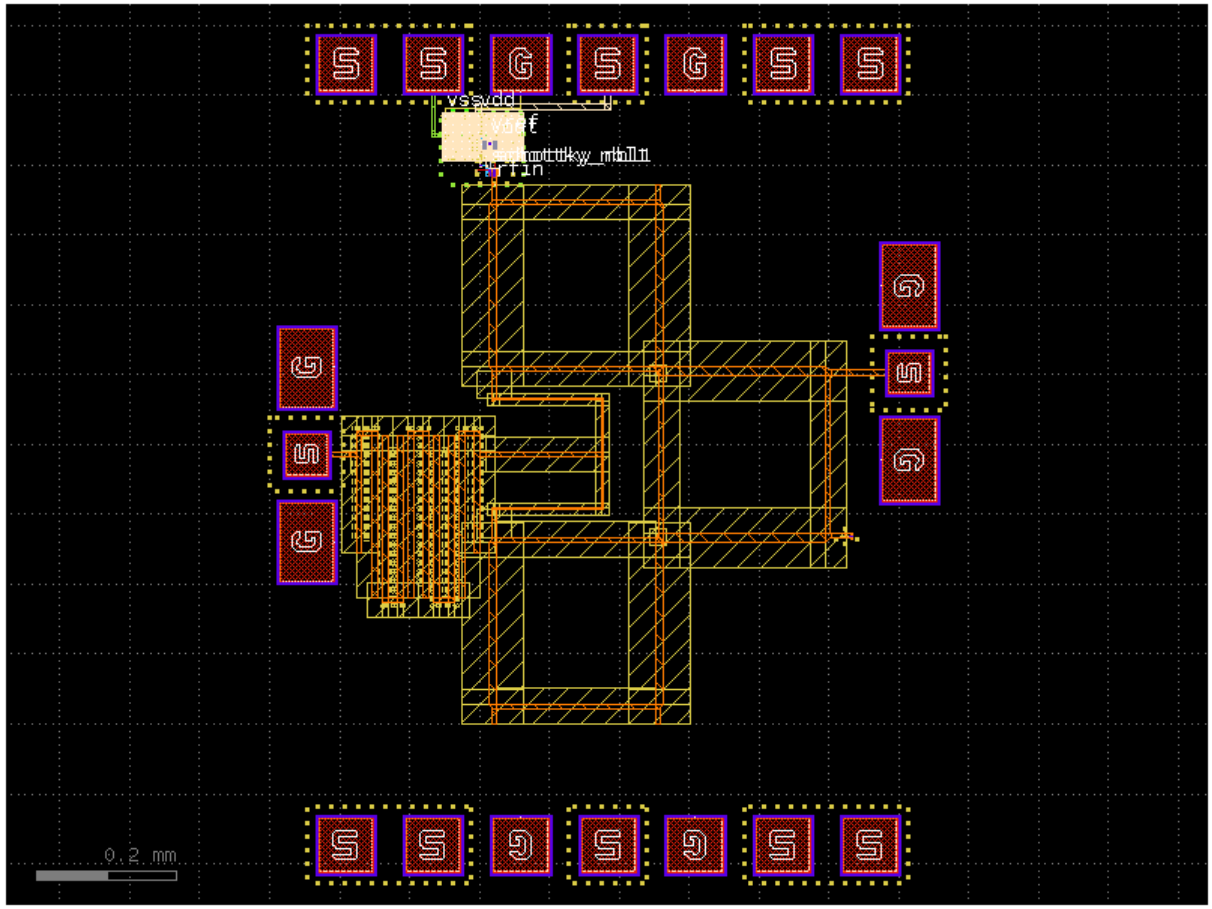
```
    separation=0,
)
c.plot()
```



`vout` is routed on Metal4 to a via stack that takes it up to the TopMetal1 pad.

```
via_M4_TM1 = ihp.cells.via_stack(
    top_layer="TopMetal1",
    bottom_layer="Metal4",
    vn_columns=10,
    vn_rows=3,
    vt1_columns=5,
    vt1_rows=1,
)
via_M4_TM1.ports["top"].orientation = 90
```

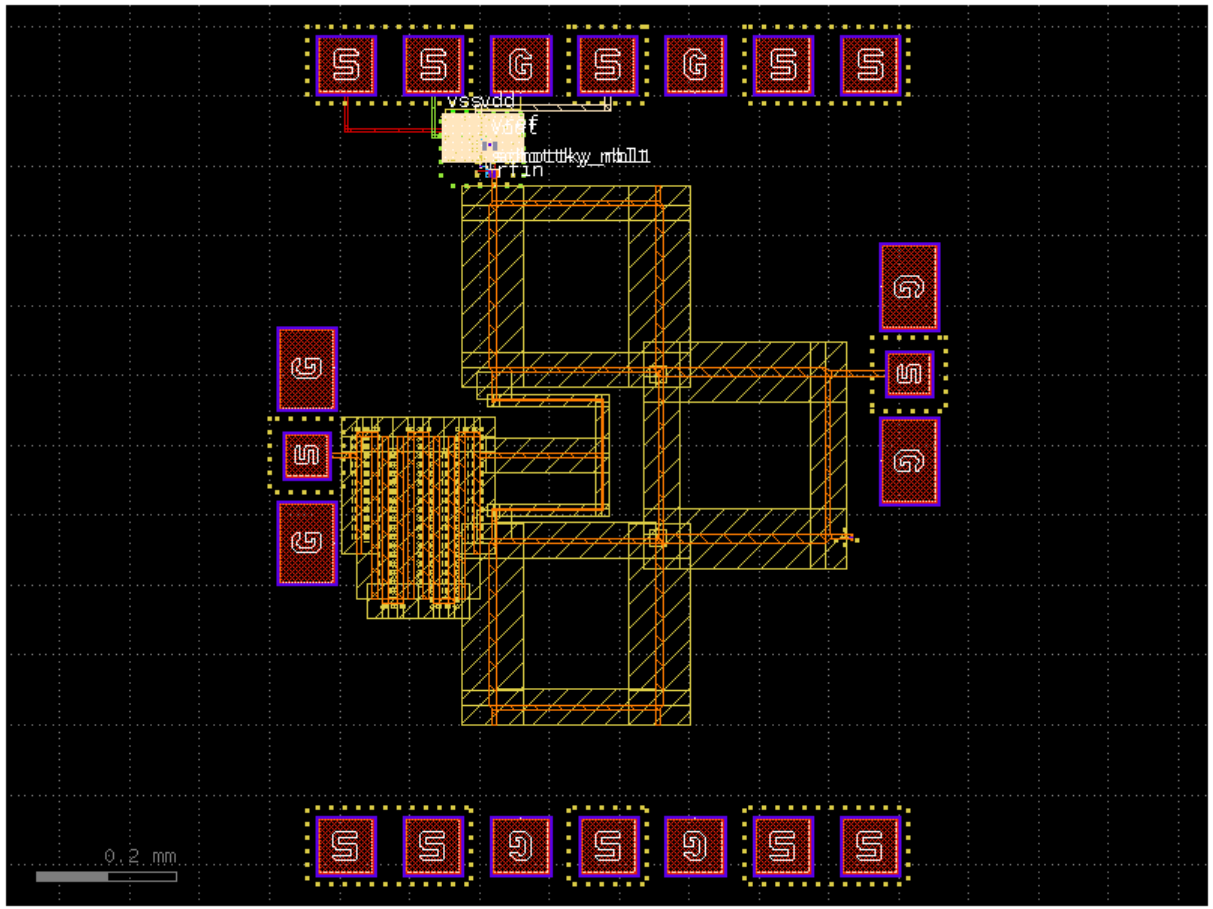
```
via_M4_TM1.ports["bottom"].orientation = 270
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_top.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, via_M4_TM1_ref.ysize/2))
route_pd1_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



`vref` is routed the same way on Metal3.

```
via_M3_TM1 = ihp.cells.via_stack(
    top_layer="TopMetal1",
    bottom_layer="Metal3",
    vn_columns=10,
    vn_rows=3,
    vt1_columns=5,
    vt1_rows=1,
)
via_M3_TM1.ports["top"].orientation = 90
via_M3_TM1.ports["bottom"].orientation = 270
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_top.ports["e1"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, via_M3_TM1_ref.ysize/2))
```

```
route_pd1_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd1_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



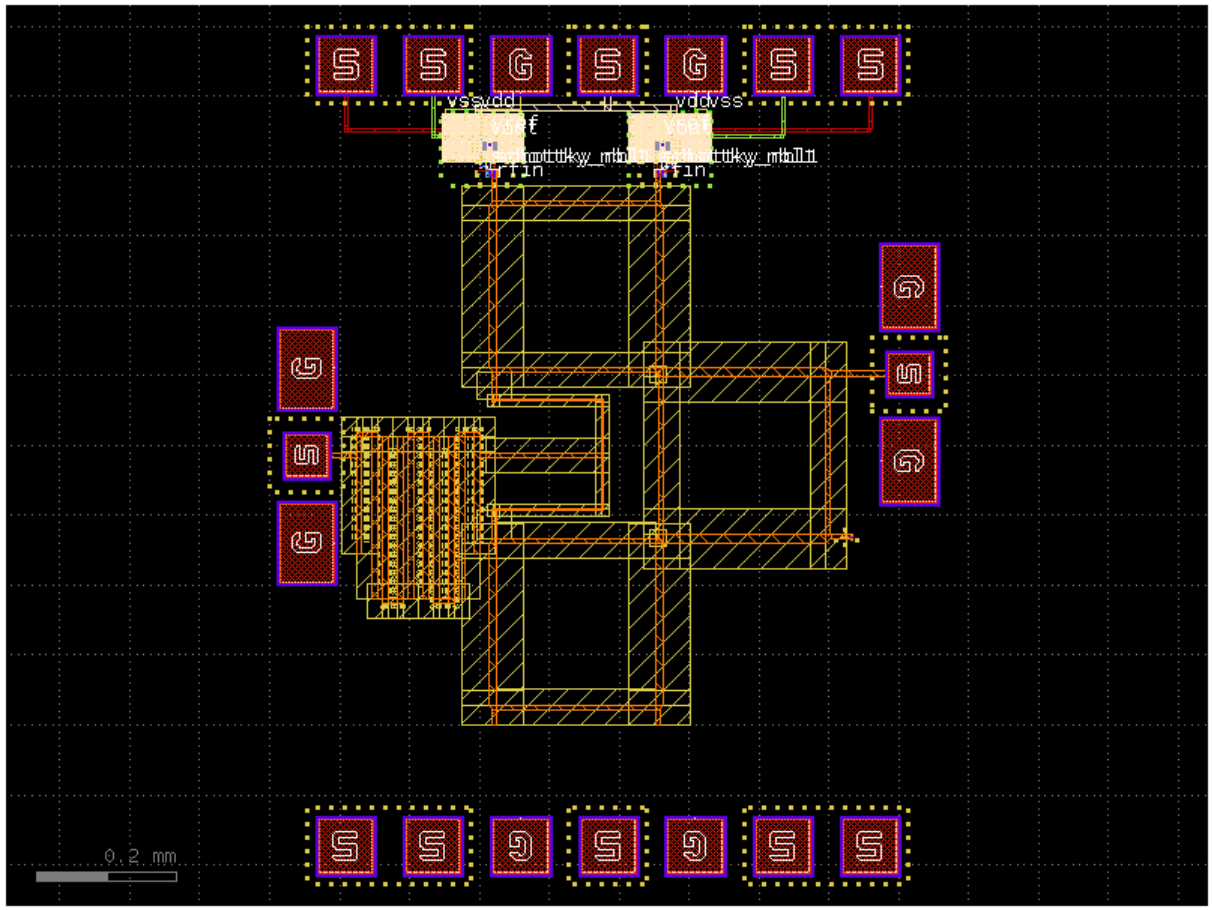
PD2, the second detector of BLC_1, is the cell without mirroring, attached to port `e3` of BLC_1 and routed to the upper pad row like PD1.

```
pd2_ref = c.add_ref(pd)
pd2_ref.connect("rfin", rfin_connection_pd2.ports["e2"], allow_width_mismatch=True)

# vdd connection of pd2 to probe top
route_pd2_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vdd"]],
    ports2=[probe_top.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vss connection of pd2 to probe top
route_pd2_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vss"]],
    ports2=[probe_top.ports["e5"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vout connection of pd2 to probe top
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_top.ports["e6"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, via_M4_TM1_ref.ysize/2))
route_pd2_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vref connection of pd2 to probe top
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_top.ports["e7"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, via_M3_TM1_ref.ysize/2))
```

```
route_pd2_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd2_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

c.plot()
```



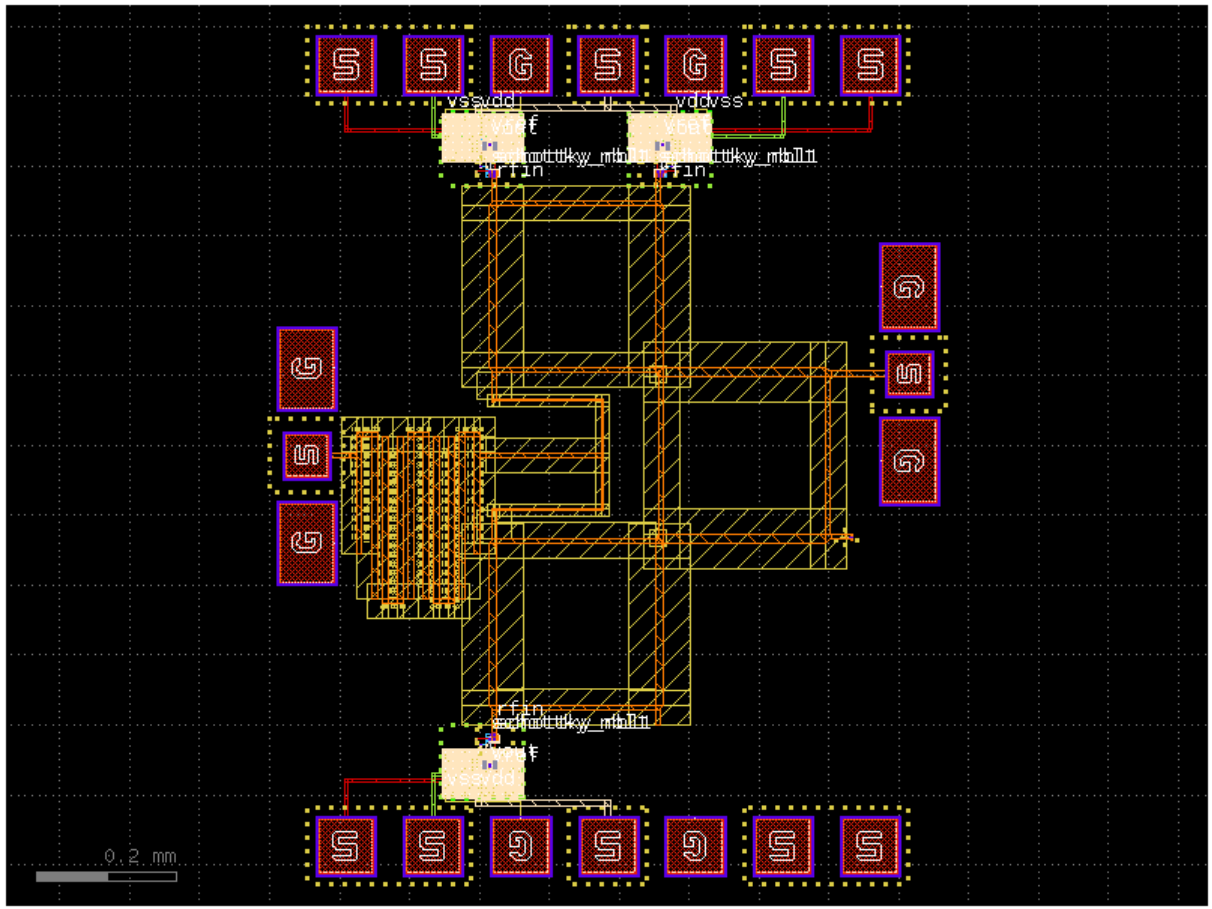
PD3 at port `e3` of BLC_2 uses the cell mirrored with `mirror_y().mirror_x()` and is routed to the lower pad row.

```
pd3_ref = c.add_ref(pd).mirror_y().mirror_x()
pd3_ref.connect("rfin", rfin_connection_pd3.ports["e2"], allow_width_mismatch=True)

# vdd connection of pd3 to probe bottom
route_pd3_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vdd"]],
    ports2=[probe_bottom.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vss connection of pd3 to probe bottom
route_pd3_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vss"]],
    ports2=[probe_bottom.ports["e5"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vout connection of pd3 to probe bottom
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_bottom.ports["e6"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, -via_M4_TM1_ref.ysize/2))

route_pd3_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vref connection of pd3 to probe bottom
```

```
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_bottom.ports["e7"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, -via_M3_TM1_ref.ysize/2))
route_pd3_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd3_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
c.plot()
```



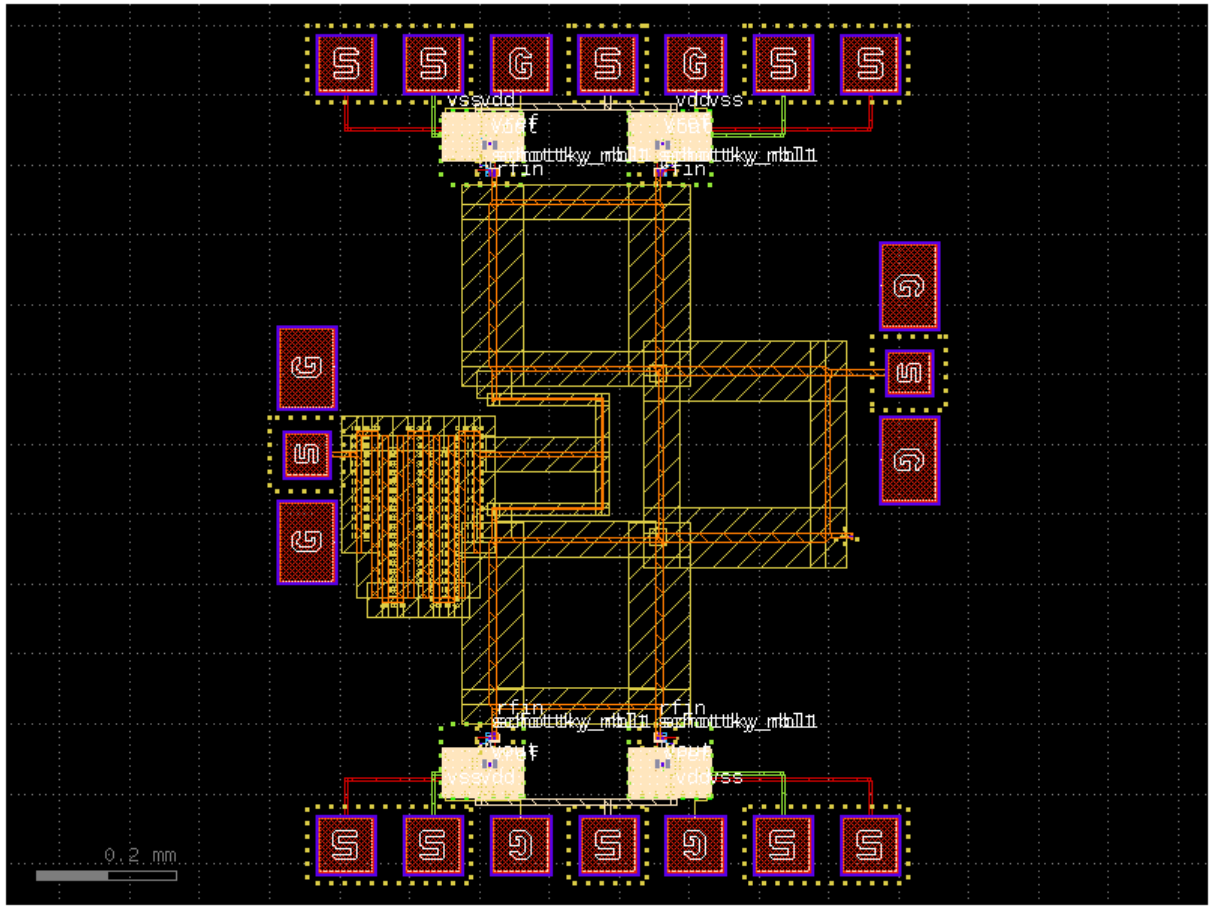
PD4 at port `e2` of BLC_2 uses the cell mirrored with `mirror_y()` and is also routed to the lower pad row.

```
pd4_ref = c.add_ref(pd).mirror_y()
pd4_ref.connect("rfin", rfin_connection_pd4.ports["e2"], allow_width_mismatch=True)

# vdd connection of pd4 to probe bottom
route_pd4_vdd = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vdd"]],
    ports2=[probe_bottom.ports["e4"]],
    route_width=8,    # adjust for thicker connections
    layer=ihp.tech.LAYER.TopMetal1drawing,
    start_straight_length=10,
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vss connection of pd4 to probe bottom
route_pd4_vss = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vss"]],
    ports2=[probe_bottom.ports["e3"]],
    route_width=2,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal5drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vout connection of pd4 to probe bottom
via_M4_TM1_ref = c.add_ref(via_M4_TM1)
via_M4_TM1_ref.connect("top", probe_bottom.ports["e2"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_M4_TM1_ref.move((0, -via_M4_TM1_ref.ysize/2))
route_pd4_vout = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vout"]],
    ports2=[via_M4_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal4drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)
# vref connection of pd4 to probe bottom
```

```
via_M3_TM1_ref = c.add_ref(via_M3_TM1)
via_M3_TM1_ref.connect("top", probe_bottom.ports["e1"], allow_width_mismatch=True, allow_layer_mismatch=True)
via_M3_TM1_ref.move((0, -via_M3_TM1_ref.ysize/2))
route_pd4_vref = gf.routing.route_bundle_electrical(
    component=c,
    ports1=[pd4_ref.ports["vref"]],
    ports2=[via_M3_TM1_ref.ports["bottom"]],
    route_width=4,    # adjust for thicker connections
    layer=ihp.tech.LAYER.Metal3drawing,
    start_straight_length=10, # 10 or longer, to avoid routing through itself
    allow_layer_mismatch=True,
    allow_width_mismatch=True,
    auto_taper=False,
    separation=0,
)

c.plot()
```



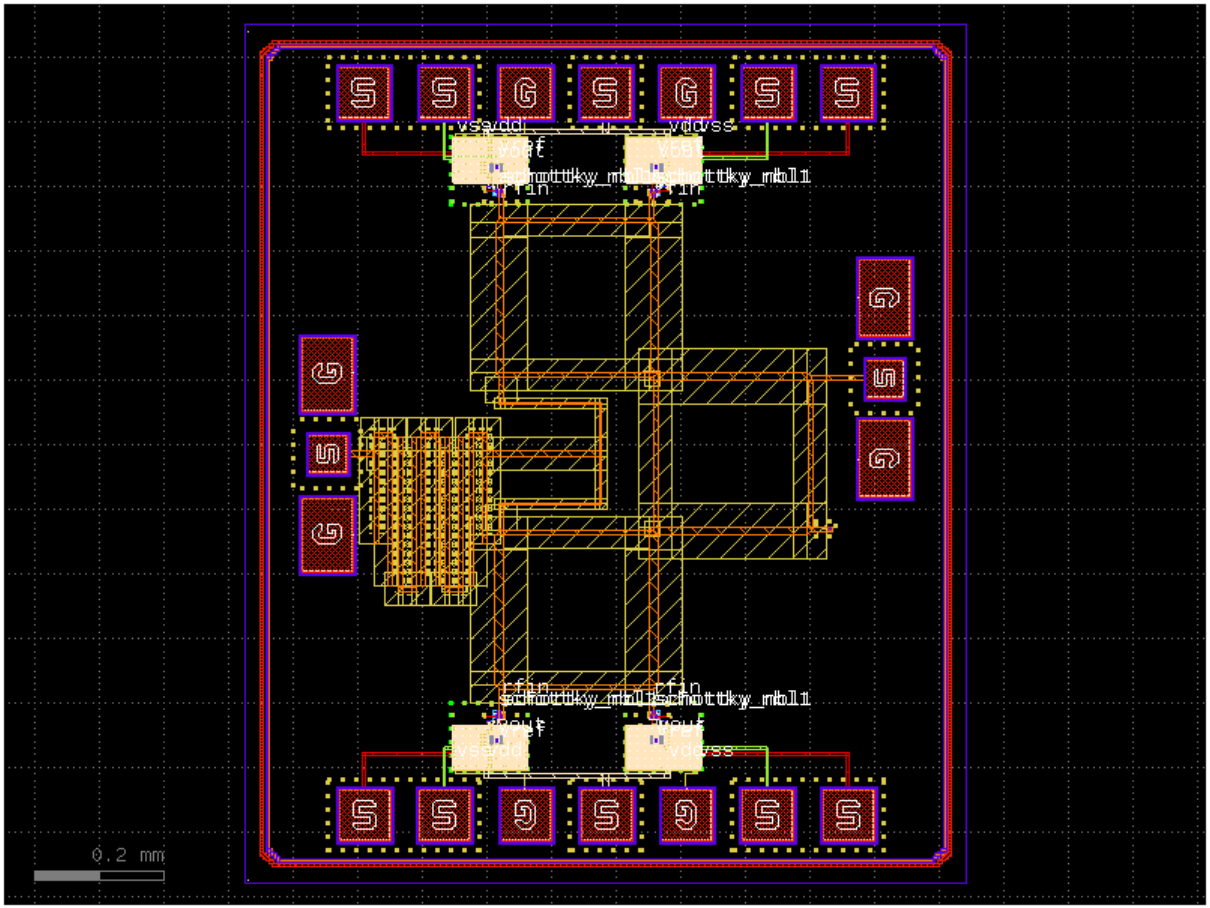
3.10 Seal Ring, Metal Fill, and Logos

A seal ring around the die stops cracks from the dicing saw and keeps moisture and contamination out of the back-end layers. The next cell sizes it from the extent of the chip so far, with a margin of `SEALRING_MARGIN` = 50 µm on the left and right and 25 µm at the top and bottom, and centers it on the chip. Its inner edge on the layer `EdgeSealboundary` is the reference for the metal fill.

```
sealring_margin = SEALRING_MARGIN  # margin around circuit for sealring
sealring_width = round(c.xsize + 2 * sealring_margin)
sealring_height = round(c.ysize + 2 * sealring_margin / 2)

sealring_center = (round(c.center[0], 2), round(c.center[1], 2))  # round it to avoid off grid errors
c.add_ref(ihp.cells.sealring(width=sealring_width, height=sealring_height)).center = sealring_center
```

```
c.plot()
```



The metal fill uses four helper cells: `fill_cell()`, a rectangle of a given size on a given layer, `fill_gat_active()`, a GatPoly filler with a centered, inset Activ filler, and `fill_ground()` and `slit_ground()`, the 1 µm × 1 µm Metal5 tile of the ground plane and the slit that is cut into it.

```
@gf.cell
def fill_cell(layer=ihp.tech.LAYER.Metal5slit, size=(3, 3)) -> gf.Component:
    """Simple rectangular fill cell for a given layer and size."""
    c = gf.Component()
    c << gf.c.rectangle(layer=layer, size=size)
    return c
```

```
@gf.cell
def fill_gat_active(size=(3, 3), active_extension=(0.18, 0.18)) -> gf.Component:
    """Gate-poly + active filler cell with centred Activ region.

    `size` sets GatPoly:filler directly (bounded by GFil.a's 5.00um max width);
    Activ:filler is inset by `active_extension` so GatPoly:filler encloses it (GFil.j).
    """
    c = gf.Component()
    gat_ref = c.add_ref(gf.c.rectangle(layer=ihp.tech.LAYER.GatPolyfiller, size=size))
    activ_ref = c.add_ref(
        gf.c.rectangle(
            layer=ihp.tech.LAYER.Activfiller,
            size=(size[0] - 2 * active_extension[0], size[1] - 2 * active_extension[1]),
        )
    )
    activ_ref.center = gat_ref.center
    return c


@gf.cell
def fill_ground() -> gf.Component:
    """1 x 1 um Metal5 ground fill tile."""
    c = gf.Component()
    c.add_polygon(
        layer=ihp.tech.LAYER.Metal5drawing,
        points=[
            (0, 0),
            (1, 0),
            (1, 1),
            (0, 1),
        ],
    )
    return c


@gf.cell
def slit_ground() -> gf.Component:
    """Metal5 slit opening for ground fill."""
    c = gf.Component()
    c.add_polygon(
        layer=ihp.tech.LAYER.Metal5slit,
        points=[
            (1.1, 1.1),
            (4, 1.1),
            (4, 4.1),
            (1.1, 4.1),
        ],
    )
    return c
```

The metal fill keeps the density of every layer within the limits of the design rules, which chemical-mechanical polishing (CMP) needs for flat layers. `c.fill()` places copies of a fill cell on a regular grid inside the seal boundary, minus exclusion zones around existing structures:

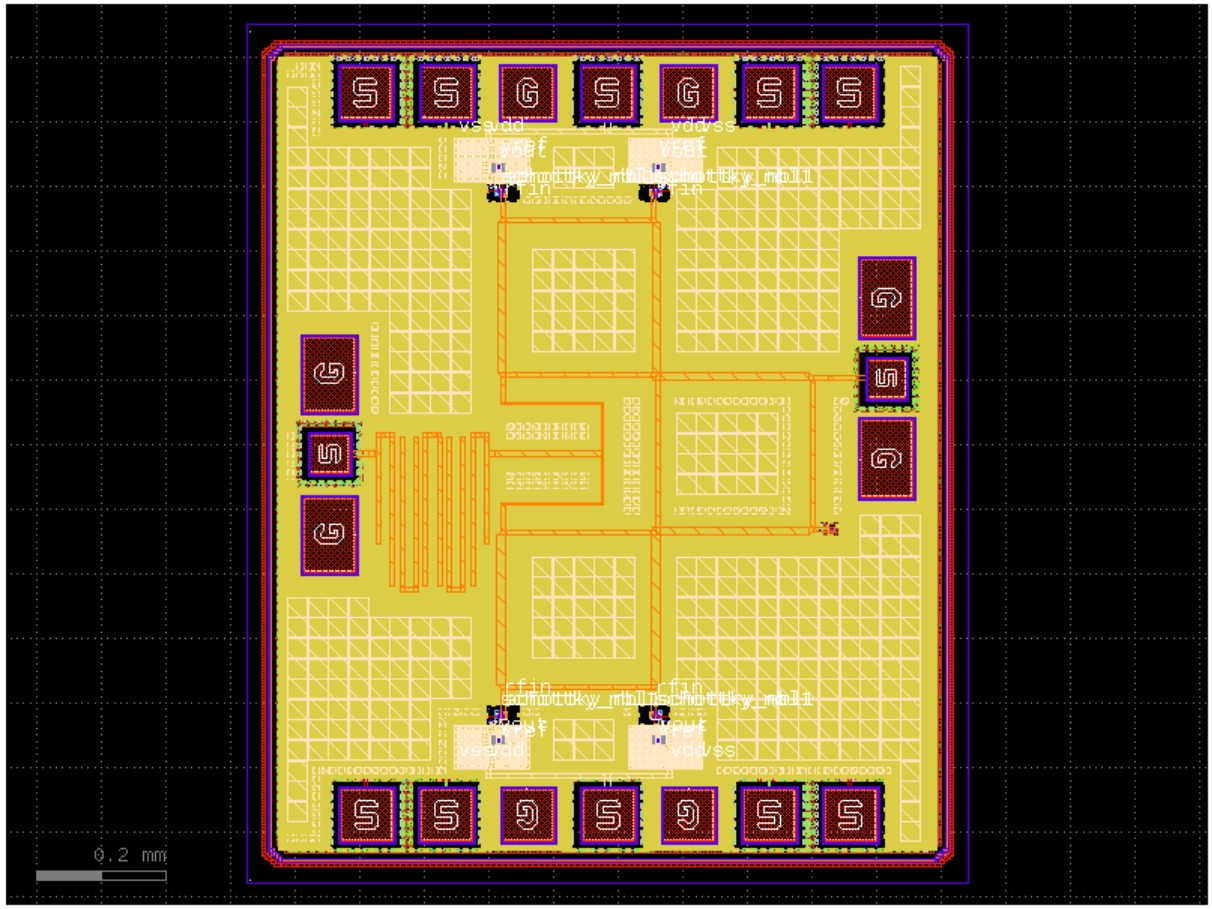
Layer	Fill cell	Spacing	Distance to the seal boundary	Kept away from
GatPoly and Activ	5 µm × 5 µm GatPoly filler with an Activ filler inset by 0.18 µm	0.85 µm	30 µm	devices (Activ, implants, wells, GatPoly, contacts) by 1.1 µm to 2 µm, pad openings by 12 µ
Metal1 and Metal2	5 µm × 5 µm	1.1 µm	40 µm	drawn metal and pad openings by 10 µm
Metal3 and Metal4	5 µm × 5 µm	1 µm	40 µm	drawn metal and pad openings by 10 µm
Metal5	solid ground plane of 1 µm × 1 µm tiles, then slits of 2.9 µm × 3 µm	0 µm, slits 1.1 µm × 1.2 µm	50 µm	pad openings and Metal5 no-fill regions, slits 5 µm away from TopMetal2 lines and 1 µm
TopMetal1	30 µm × 30 µm drawn squares, then 10 µm × 10 µm filler	1.65 µm, 3 µm	10 µm	drawn TopMetal1 by 10 µm, TopMetal2 by 20 µm

The Metal5 ground plane is the reference plane of all microstrip lines. The design rules require slits in wide metal, so slits are cut into it everywhere except under the TopMetal2 lines and the MIM capacitors. The plane stays 50 µm inside the seal boundary. Any closer, it merges with the Metal5 ring of the seal ring, which connects the whole seal ring to VSS and adds antenna violations at the transistor gates tied to VSS.

```
# active/gat poly fill
gatpoly_activ_fill_space = 0.85
c.fill(
    fill_cell=fill_gat_active(size=(5, 5)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -30)],
    exclude_layers=[
        (ihp.tech.LAYER.Activdrawing, 1.1),
        (ihp.tech.LAYER.pSDdrawing, 1.1),
        (ihp.tech.LAYER.nSDblock, 1.1),
        (ihp.tech.LAYER.SalBlockdrawing, 1.1),
        (ihp.tech.LAYER.NWelldrawing, 1.1),
        (ihp.tech.LAYER.nBuLaydrawing, 1.1),
        (ihp.tech.LAYER.PWellblock, 1.5),
        (ihp.tech.LAYER.GatPolydrawing, 2),
        (ihp.tech.LAYER.Contdrawing, 1.2),
        (ihp.tech.LAYER.Passivdrawing, 12),
        (ihp.tech.LAYER.GatPolynofill, 1),
        (ihp.tech.LAYER.Activnofill, 1),
    ],
    x_space=gatpoly_activ_fill_space,
    y_space=gatpoly_activ_fill_space,
)
# metal1 fill
metal_fill_space = 1.1
c.fill(
    fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal1filler, size=(5, 5)),
    fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -40)],
    exclude_layers=[
        (ihp.tech.LAYER.Metal1drawing, 10),
```

```
            (ihp.tech.LAYER.Passivdrawing, 10),
            (ihp.tech.LAYER.Metal1nofill, 1),
        ],
        x_space=metal_fill_space,
        y_space=metal_fill_space,
    )
    # metal2 fill
    c.fill(
        fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal2filler, size=(5, 5)),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -40)],
        exclude_layers=[
            (ihp.tech.LAYER.Metal2drawing, 10),
            (ihp.tech.LAYER.Passivdrawing, 10),
            (ihp.tech.LAYER.Metal2nofill, 1),
        ],
        x_space=metal_fill_space,
        y_space=metal_fill_space,
    )
    # metal3 fill
    metal_fill_space = 1
    c.fill(
        fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal3filler, size=(5, 5)),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -40)],
        exclude_layers=[
            (ihp.tech.LAYER.Metal3drawing, 10),
            (ihp.tech.LAYER.Passivdrawing, 10),
            (ihp.tech.LAYER.Metal3nofill, 1),
        ],
        x_space=metal_fill_space,
        y_space=metal_fill_space,
    )
    # metal4 fill
    c.fill(
        fill_cell=fill_cell(layer=ihp.tech.LAYER.Metal4filler, size=(5, 5)),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -40)],
        exclude_layers=[
            (ihp.tech.LAYER.Metal4drawing, 10),
            (ihp.tech.LAYER.Passivdrawing, 10),
            (ihp.tech.LAYER.Metal4nofill, 1),
        ],
        x_space=metal_fill_space,
        y_space=metal_fill_space,
    )
    # metal5 fill (groundplate)
    c.fill(
        fill_cell=fill_ground(),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
        exclude_layers=[(ihp.tech.LAYER.Passivdrawing, 0), (ihp.tech.LAYER.Metal5nofill, 0)],
        x_space=0,
        y_space=0,
```

```
    )
    c.fill(
        fill_cell=slit_ground(),
        fill_layers=[(ihp.tech.LAYER.Metal5drawing, -2)],
        exclude_layers=[
            (ihp.tech.LAYER.TopMetal2drawing, 5),
            (ihp.tech.LAYER.MIMdrawing, 1),
            (ihp.tech.LAYER.Metal5slit, 1),
        ],
        x_space=1.1,
        y_space=1.2,
    )
    # topmetal1 fill
    c.fill(
        fill_cell=fill_cell(layer=ihp.tech.LAYER.TopMetal1drawing, size=(30, 30)),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -10)],
        exclude_layers=[(ihp.tech.LAYER.TopMetal1drawing, 10), (ihp.tech.LAYER.TopMetal2drawing, 20)],
        x_space=1.65,
        y_space=1.65,
    )
    c.fill(
        fill_cell=fill_cell(layer=ihp.tech.LAYER.TopMetal1filler, size=(10, 10)),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -10)],
        exclude_layers=[(ihp.tech.LAYER.TopMetal1drawing, 10), (ihp.tech.LAYER.TopMetal2drawing, 20)],
        x_space=3,
        y_space=3,
    )
    c.plot()
```



The logos and labels are placed relative to the layout, so they follow the design frequency: the JKU logo on TopMetal2 in the lower right corner above the lower pad row, the IWS logo next to the upper right corner of the BLCs up to 200 GHz, where there is room for it, and the designer and supervisor names, rotated by -90°, between the pad rows where they fit. The labels are fixed GDS images, so every generated layout reads "SPARX 160", whatever its frequency.

```
logo_dir = Path("/tmp/assets")

logo_gds_path = logo_dir / "jku_logo_m5.gds"
kellerer_gds_path = logo_dir / "D_Kellerer.gds"
iws_gds_path = logo_dir / "IWS.gds"
supervisors_gds_path = logo_dir / "supervisors.gds"

# JKU logo in the lower right corner, relative to right chip edge, just above south pads
jku_logo_ref = c.add_ref(gf.import_gds(str(logo_gds_path), cellname="JKU_LOGO"))
```

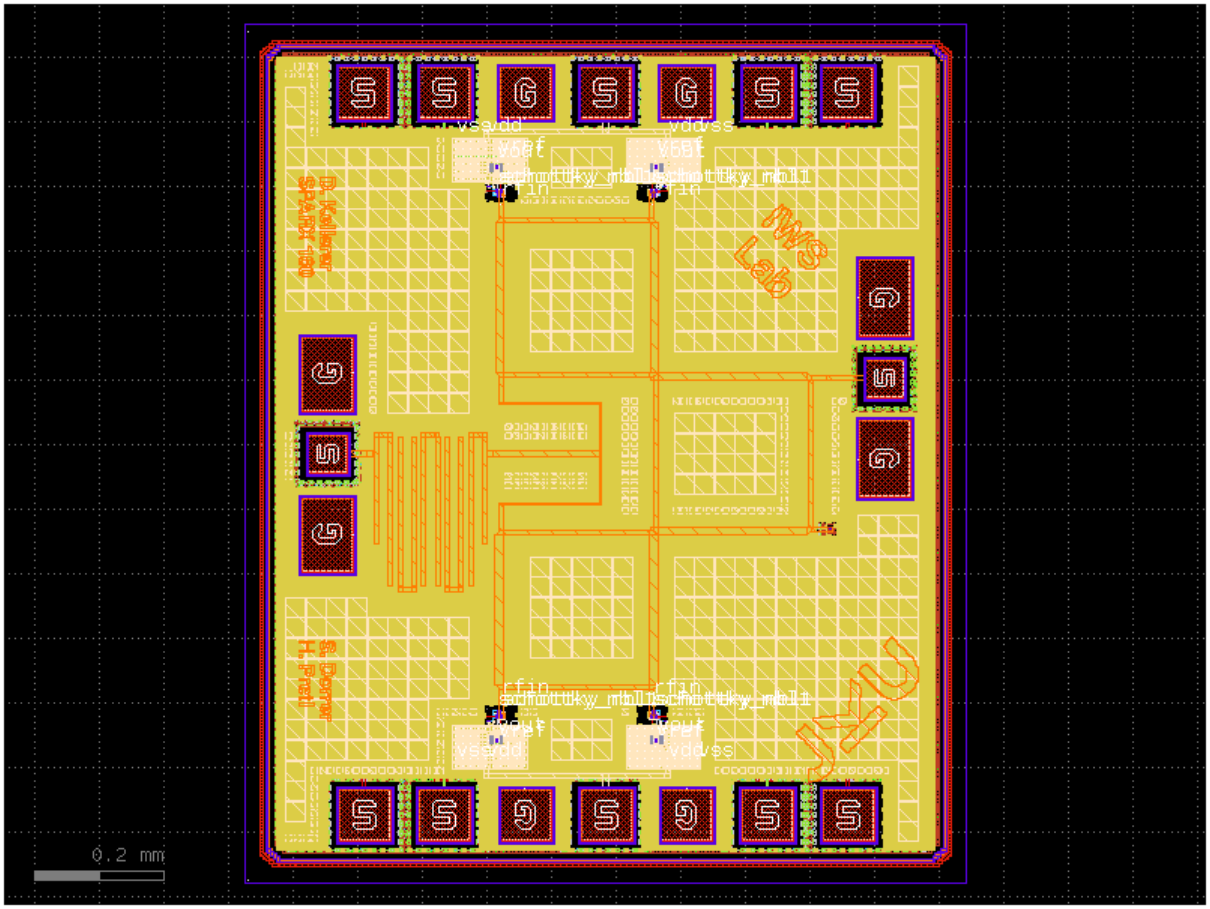
```
chip_right = sealring_center[0] + sealring_width / 2
jku_logo_ref.xmax = chip_right - sealring_margin
jku_logo_ref.ymin = probe_bottom.ymax + 10

# place IWS logo only up to 200 GHz, to avoid overcrowding the layout for higher frequencies where the design is more compact
if f <= 200e9:
    iws = c.add_ref(gf.import_gds(str(iws_gds_path), cellname="Logo_IWS"))
    iws.xmax = blc_3_ref.xmax  # align right edge of IWS logo with right edge of BLC block
    iws.ymax = blc_1_ref.ymax  # align top edge of IWS logo with top edge of BLC block

# place gds with D. Kellerer only if space is available between top probe pads and left probe pads
kellerer = gf.import_gds(str(kellerer_gds_path), cellname="Name_D").rotate(-90)
if abs(probe_top.ymin - probe_left.ymax) > kellerer.ysize + 2 * LOGO_VERTICAL_CLEARANCE:
    kellerer_ref = c.add_ref(kellerer)
    kellerer_ref.xmin = probe_left.xmin
    kellerer_ref.center = (kellerer_ref.center[0], probe_left.ymax + (abs(probe_top.ymin - probe_left.ymax) / 2))
    kellerer_ref.xmin = snap_to_grid(kellerer_ref.xmin)
    kellerer_ref.ymin = snap_to_grid(kellerer_ref.ymin)

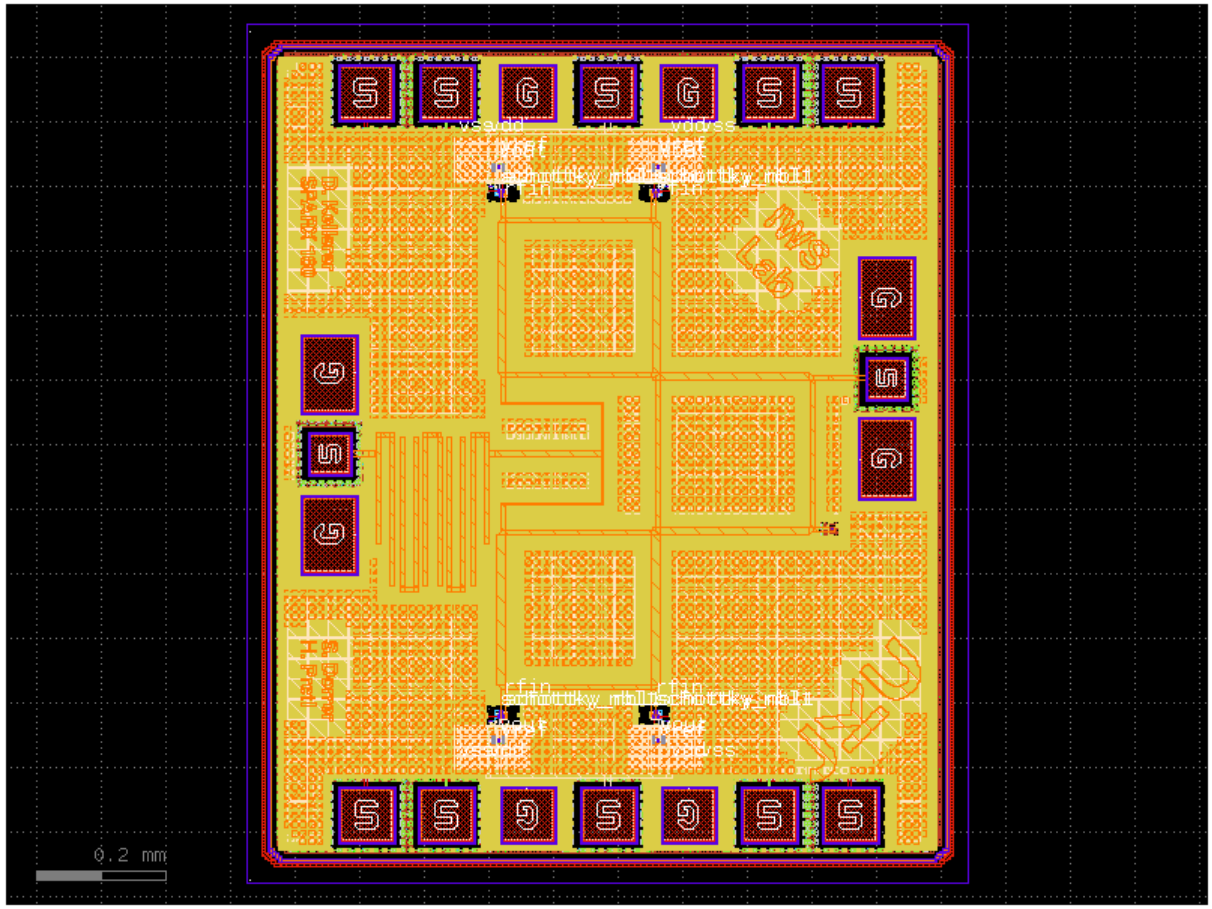
# place supervisor names only if space is available between bottom probe pads and left probe pads
supervisors = gf.import_gds(str(supervisors_gds_path), cellname="supervisors").rotate(-90)
if abs(probe_bottom.ymax - probe_left.ymin) > supervisors.ysize + 2 * LOGO_VERTICAL_CLEARANCE:
    supervisors_ref = c.add_ref(supervisors)
    supervisors_ref.xmin = probe_left.xmin
    supervisors_ref.center = (supervisors_ref.center[0], probe_bottom.ymax + (abs(probe_bottom.ymax - probe_left.ymin) / 2))
    supervisors_ref.xmin = snap_to_grid(supervisors_ref.xmin)
    supervisors_ref.ymin = snap_to_grid(supervisors_ref.ymin)

c.plot()
```



The last fill step covers TopMetal2 with 9 µm × 9 µm fill cells at a spacing of 3 µm, at least 20 µm away from the drawn TopMetal2 lines, and the plot shows the finished chip.

```
c.fill(
        fill_cell=fill_cell(layer=ihp.tech.LAYER.TopMetal2filler, size=(9, 9)),
        fill_layers=[(ihp.tech.LAYER.EdgeSealboundary, -50)],
        exclude_layers=[(ihp.tech.LAYER.TopMetal2drawing, 20)],
        x_space=3,
        y_space=3,
    )
c.plot()
```



3.11 Pad Labels and GDS Export

Finally, the chip is moved to its final origin, and every probe pad and bond pad gets a text label and a pin rectangle on its metal layer. The labels name the pad nets, `vlo`, `vrf`, `vdd`, `vss`, `vout1` to `vout4`, and `vref1` to `vref4`, for LVS and for finding the pads in a layout viewer. `bondpad_array` puts the ports at the outer edge of each pad, so `add_pad_pin()` first moves a port to the pad center, then adds the label on the text datatype (25) and the pin on the pin datatype (2) of the pad layer.

```
def add_pad_pin(c: gf.Component, port_name: str, pad_name: str, pad_length: float ) -> None:
    """Add a port label and pin rectangle centered on a probe pad.

    Ports on pad arrays sit at the pad's outer edge rather than its
    center (see bondpad_array), offset by half the pad size in the
```

```
    direction the port faces. This recenters the label/pin on the pad
    before adding it, based on the port's orientation.

    Args:
        c: Parent component (`c`) that owns the port.
        port_name: Name of the port on `c` to label.
        pad_name: Text to use for the port label (e.g. "VOUT"). The text
            and pin layers are derived from the port's own drawing layer.
    """
    port = c.ports[port_name].copy()
    orientation = port.orientation % 360

    if orientation == 0:
        port.center = (port.center[0] - pad_length / 2, port.center[1])
    elif orientation == 90:
        port.center = (port.center[0], port.center[1] - pad_length / 2)
    elif orientation == 180:
        port.center = (port.center[0] + pad_length / 2, port.center[1])
    elif orientation == 270:
        port.center = (port.center[0], port.center[1] + pad_length / 2)
    else:
        raise ValueError(f"Unsupported port orientation: {port.orientation}")

    layer_number, _ = gf.get_layer_tuple(port.layer)
    text_layer = (layer_number, 25)
    pin_layer = (layer_number, 2)

    gf.labels.add_port_labels(c, ports=[port], layer=text_layer, texts=[pad_name])
    gf.add_pins.add_pin_rectangle(c, port=port, layer=pin_layer)


c.xmin = 0
c.ymin = 0
c.move((-25, -25))


# add port labels for probe pads on the left side of the chip
add_pad_pin(c = c, port_name="probe_left_e1", pad_name="vlo", pad_length=PROBE_SIGNAL_SIZE)
add_pad_pin(c = c, port_name="probe_left_e2", pad_name="vss", pad_length=PROBE_SIGNAL_SIZE)
add_pad_pin(c = c, port_name="probe_left_e3", pad_name="vss", pad_length=PROBE_SIGNAL_SIZE)

# add port labels for probe pads on the right side of the chip
add_pad_pin(c = c, port_name="probe_right_e1", pad_name="vrf", pad_length=PROBE_SIGNAL_SIZE)
add_pad_pin(c = c, port_name="probe_right_e2", pad_name="vss", pad_length=PROBE_SIGNAL_SIZE)
add_pad_pin(c = c, port_name="probe_right_e3", pad_name="vss", pad_length=PROBE_SIGNAL_SIZE)

# add port labels for probe pads on the top side of the chip
PROBE_SIGNAL_SIZE_TOP_BOTTOM = c.ports["probe_top_e1"].width  # size of the probe signal pads in um
add_pad_pin(c = c, port_name="probe_top_e1", pad_name="vref1", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e2", pad_name="vout1", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
```

```
add_pad_pin(c = c, port_name="probe_top_e3", pad_name="vss", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e4", pad_name="vdd", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e5", pad_name="vss", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e6", pad_name="vout2", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_top_e7", pad_name="vref2", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)

# add port labels for probe pads on the bottom side of the chip
add_pad_pin(c = c, port_name="probe_bottom_e1", pad_name="vref4", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e2", pad_name="vout4", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e3", pad_name="vss", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e4", pad_name="vdd", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e5", pad_name="vss", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e6", pad_name="vout3", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
add_pad_pin(c = c, port_name="probe_bottom_e7", pad_name="vref3", pad_length=PROBE_SIGNAL_SIZE_TOP_BOTTOM)
```

The finished chip is written to `sparx_top.gds` without GDSFactory metadata, and on Colab the file is downloaded to the local machine. At `f` = 160 GHz it is geometrically identical to `layout/sparx160_top.gds`, which the generator of the repository writes. The generator has evolved since the tapeout, so both differ in outline and placement details from the fabricated die, whose GDS is `release/v.1.0.0/gds/RFFE6027.gds` in the repository.

```
c.write_gds("sparx_top.gds", with_metadata=False)

# on Google Colab, download the GDS to the local machine
try:
    from google.colab import files
    files.download("sparx_top.gds")
except ImportError:
    pass
```

4 Physical Verification

The generated chip is checked with the KLayout DRC decks of the IHP Open-PDK. The decks are Ruby DRC scripts, so they need the full KLayout program in addition to the `klayout` Python module that GDSFactory installs. LVS and parasitic extraction run in the repository [2], where top-level LVS is still open and the detector cell is verified on its own.

4.1 KLayout Installation

The KLayout package for the Ubuntu version of the runtime is downloaded from the KLayout website and installed with `apt`.

```
import platform

KLAYOUT_VERSION = "0.30.12"
UBUNTU = platform.freedesktop_os_release()["VERSION_ID"].split(".")[0]

! wget -q https://www.klayout.org/downloads/Ubuntu-{UBUNTU}/klayout_{KLAYOUT_VERSION}-1_amd64.deb -O /tmp/klayout.deb
! apt-get update -qq > /dev/null 2>&1; apt-get install -y -qq /tmp/klayout.deb > /dev/null
assert shutil.which("klayout"), "KLayout was not installed, see the apt output above"
! klayout -b -v
```

```
KLayout 0.30.12
```

4.2 Design Rule Check

The DRC runs through the `run_drc.py` wrapper of the IHP Open-PDK at one of three levels:

- `precheck`: the minimal rule set of the foundry precheck, the fastest option.
- `macro`: all rules except density, antenna, and the extra rules. This is the level of `make klayout-drc` in the repository. The 160 GHz layout passes it without violations, and so do the layouts of the other twelve frequencies from 60 GHz to 300 GHz.
- `regular`: the full top-level DRC, which checks the complete chip against all rules, density and antenna included.

This notebook runs `regular`, the full top-level DRC, which takes about 10 minutes. The next cell prints the progress per rule and the result, and the full logs go to the folder `drc_regular`.

> **Note: The DRC reports 33 violations at the `regular` level. All of them are known antenna violations that IHP waived for the March 2026 tapeout.** The wrapper therefore ends with *KLayout DRC Check Failed*. Every other rule passes. The result after the violation count below gives the causes, and the known issues of the repository list the violations as well.

```
import time

RUN_DRC = FULL_RUN  # 1 = run the DRC and the violation count below, 0 = skip both

DRC_LEVEL = "regular"  # precheck, macro, or regular
DRC_DIR   = f"drc_{DRC_LEVEL}"
DRC_FLAGS = {
    "precheck": "--precheck_drc --no_density --no_offgrid --no_angle --disable_extra_rules --no_recommended",
    "macro":    "--no_density --disable_extra_rules",
    "regular":  "--antenna",
}[DRC_LEVEL]

# the rule deck is split into one part per physical CPU core, hyperthreads of the same core do not help because every part reloads the layout
DRC_CORES = len({open(f"/sys/devices/system/cpu/cpu{cpu}/topology/thread_siblings_list").read() for cpu in os.sched_getaffinity(0)})

if RUN_DRC:
    start = time.time()
```

```
    # the full logs are written to DRC_DIR, only a rule counter, the progress per rule table, and the result are shown here
    ! rm -rf {DRC_DIR} && python3 {PDK_ROOT}/ihp-sg13g2/libs.tech/klayout/tech/drc/run_drc.py --path=sparx_top.gds --topcell=Six-Port-CAC --run_dir={DRC_DIR} --mp={
        | awk '/Executing rule/ {{ if (++n % 50 == 0) print n " rules checked" }} /completed in|Check Passed|Check Failed|Violated rules|Total DRC Run time|ERROR/ {
    print(f"DRC ({DRC_LEVEL}) finished in {(time.time() - start) / 60:.1f} min")
else:
    print("RUN_DRC = 0, DRC skipped")
```

```
50 rules checked
100 rules checked
150 rules checked
200 rules checked
250 rules checked
300 rules checked
350 rules checked
400 rules checked
450 rules checked
500 rules checked
550 rules checked
2026-10-30 21:20:00 +0000: Memory Usage (1760964K) : KLayout DRC run for tables 'main' completed in 327.9 seconds
600 rules checked
2026-10-30 21:26:45 +0000: Memory Usage (1156784K) : KLayout DRC run for density table completed in 230.65 seconds
2026-10-30 21:31:04 +0000: Memory Usage (2295868K) : KLayout DRC run for maximum ruleSet completed in 256.16 seconds
30-Oct-2026 21:31:04 | ERROR   | ================================================================================
30-Oct-2026 21:31:04 | ERROR   | ❌ --- KLayout DRC Check Failed: Violations were detected in the layout. --- ❌
30-Oct-2026 21:31:04 | ERROR   | ================================================================================
30-Oct-2026 21:31:04 | ERROR   | Violated rules are : {'Ant.e_TopMetal1', 'Ant.h', 'Ant.e_Metal5', 'Ant.f_TopVia2', 'Ant.e_TopMetal2'}
30-Oct-2026 21:31:04 | INFO    | Total DRC Run time: 994.60 seconds (including execution, analysis, and reporting)
DRC (regular) finished in 16.6 min
```

The violations are counted per rule from the KLayout result databases (`.lyrdb`), which can also be opened in the marker browser of KLayout.

```
import glob
import xml.etree.ElementTree as ET
from collections import Counter

if RUN_DRC:
    # every <item> in a result database is one violation, its <category> is the rule name
    lyrdbs = glob.glob(f"{DRC_DIR}/*.lyrdb")
    assert lyrdbs, f"No DRC result in {DRC_DIR}, the DRC did not run, see the output above"
    violations = Counter()
    for lyrdb in lyrdbs:
        for item in ET.parse(lyrdb).getroot().iterfind("./items/item"):
            violations[item.findtext("category").strip("'")] += 1

    for rule, count in sorted(violations.items()):
        print(f"{rule:<24} {count:>6}")
    print(f"DRC ({DRC_LEVEL}): {sum(violations.values())} violations in {len(violations)} rules")
else:
    print("RUN_DRC = 0, no DRC result to count")
```

```
Ant.e_Metal5                  8
Ant.e_TopMetal1               8
Ant.e_TopMetal2               8
Ant.f_TopVia2                 8
Ant.h                         1
DRC (regular): 33 violations in 5 rules
```

DRC Result:

rule	violations	cause
`Ant.e_Metal5`, `Ant.e_TopMetal1`, `Ant.e_TopMetal2`	8 each	large RF and supply metal, the Metal5 ground plane on VSS included, connects to the gates of the detector transistors and the dummy devices, so the ratio of the cum
`Ant.f_TopVia2`	8	the same nets, the ratio of the cumulative TopVia2 area to the gate area exceeds 500
`Ant.h`	1	the antenna diode built into the Schottky PCell of the PDK sits in an NWell, which the rule forbids

All 33 violations are the antenna violations that IHP waived for the tapeout [2], and every other rule of the `regular` level passes.

5 Circuit Simulation with VACASK

The passive core of SPARX was simulated in AWS Palace in the repository [2]. This section takes the EM result to the circuit level and simulates the power detector and the receiver with VACASK [10], an open-source circuit simulator. Most open-source RFIC work uses Qucs-S with Xyce, which for a long time was the only open-source SPICE-compatible simulator with harmonic balance (HB), but without its small-signal and noise extensions and without transient noise (TN) or periodic steady-state (PSS) analysis [1]. VACASK offers all of them: device-level TN, HB with the small-signal analysis HBAC and the noise analysis HBNOISE on top, and shooting PSS with periodic ac (PAC) and periodic noise (PNOISE) analyses. Except for HB, VACASK gained these analyses in 2026, after the tapeout of SPARX. A detector pumped by a strong LO needs them, since a small-signal analysis at the dc operating point misses the effect of the LO.

After the installation, this section runs the model fit and five testbenches of the repository:

Section	Simulation	Analyses
5.2	lumped-element model of the six-port core	vector fit with snp2le, thermal noise by Bosma's theorem
5.3	detector transfer curve	HB, PSS
5.4	detector noise figure	noise at the dc operating point, HBAC, HBNOISE, PAC, PNOISE
5.5	detector transient noise	TN
5.6	receiver	HB, HBAC, two-tone HB, transient
5.7	receiver noise figure	noise at the dc operating point, HBAC, HBNOISE, PAC, PNOISE

The testbench README of the repository explains every result in more detail. The netlists were exported from the Xschem schematics of the repository and are fetched from it together with the post-processing scripts.

> **Note:** The simulations use the 160 GHz design. The EM result of the core (`sparx160_core.s7p`), the post-layout extraction of the detector, and the LO and RF frequencies of the testbenches do not follow the frequency `f` of Section 3. A layout at another frequency needs a new EM simulation and extraction in the repository flow first.

> **Note:** Each simulation cell has a `RUN_...` flag at its top, set it to `0` to skip the cell. The flags of Sections 5.5 to 5.7 follow `FULL_RUN` at the top. The noise-figure simulation of the detector reads the responsivity of the transfer-curve simulation, and the transient-noise simulation reads the result of the noise-figure simulation, so they have to run in this order.

5.1 VACASK Installation

IIC-OSIC-TOOLS ships VACASK with PAC and PNOISE since its 2026.10 image, with VACASK `fb8cc20` [16]. The next cell downloads the VACASK and OpenVAF-Reloaded layers of that image from Docker Hub, checks them against their digests, and unpacks them to the paths of the image. With `BUILD_VACASK = 1`, it builds the same VACASK commit from source instead, which takes about 12 minutes.

VACASK reads the device models of the IHP Open-PDK as its own model libraries, which the converter `sg13g2tovc` of VACASK generates from the ngspice models of the PDK. Before the conversion, the cell applies the three fixes that IIC-OSIC-TOOLS applies to this converter for its image, which add PDK model files and Verilog-A models the converter does not list. Finally, it fetches the testbench netlists, the fitted models, the post-layout netlist of the detector, the EM results, and the post-processing scripts from the SPARX repository and defines the helpers `run_testbench()` and `show_figure()`.

> **Runtime:** about 1 minute on Colab.

```
import hashlib
import json
import re
import subprocess

RUN_VACASK_INSTALL = 1  # 1 = run this cell, 0 = skip it
BUILD_VACASK       = 0  # 1 = build VACASK from source, 0 = download the IIC-OSIC-TOOLS build

# VACASK (with pac and pnoise) and OpenVAF-Reloaded of the IIC-OSIC-TOOLS 2026.10 image, one image layer each
IIC_IMAGE     = "hpretl/iic-osic-tools"
IIC_LAYERS    = {
    "vacask":  "sha256:12b2b9dbcb0b0465bfbcd7f0143c98d7e8ab376b401f760c9901da33bf3a0976",
    "openvaf": "sha256:cf84ea92ff0036047cff9aa2fcf2937355fe190427a63bffdcb8fc8308676ea9",
}
VACASK_COMMIT = "fb8cc2055a4f10cff1de8017b554d22ea57ce448"  # the VACASK commit of the 2026.10 image
IIC_COMMIT    = "68d3f07e6eeb9ded2cf79d4db9ca9d574862db02"  # the 2026.10 tag of IIC-OSIC-TOOLS, for its fixes of the model converter
IIC_FIXES     = ["fix_sg13g2_vacask_converter.py", "fix_vacask_swsoa.py", "fix_sg13g2_vacask_va.py"]
VACASK_DIR    = "/foss/tools/vacask"
OPENVAF_DIR   = "/foss/tools/openvaf"
SPARX_DIR     = "/tmp/sparx"  # sparse checkout of the SPARX repository
XSCHEM_DIR    = f"{SPARX_DIR}/testbenches/xschem"
SIM_DIR       = f"{XSCHEM_DIR}/simulations"
PLOT_DIR      = f"{XSCHEM_DIR}/plot_simulations"

if RUN_VACASK_INSTALL:
```

```
    start = time.time()
    if BUILD_VACASK:
        # the build recipe of IIC-OSIC-TOOLS: Boost 1.88 is built because Ubuntu 24.04 ships 1.83, OpenVAF-Reloaded compiles the built-in device models
        ! apt-get update -qq > /dev/null 2>&1; apt-get install -y -qq cmake ninja-build bison flex libfftw3-dev libsuperlu-dev libopenblas-dev liblapack-dev libsuit
        ! mkdir -p {OPENVAF_DIR} && wget -qO- https://github.com/OpenVAF/OpenVAF-Reloaded/releases/download/v24.0.2mob/openvaf-r-v24.0.2mob-linux-x86_64.tar.gz | ta
        ! cd /tmp && wget -q https://archives.boost.io/release/1.88.0/source/boost_1_88_0.tar.gz && tar xzf boost_1_88_0.tar.gz
        ! cd /tmp/boost_1_88_0 && ./bootstrap.sh --with-toolset=gcc > /tmp/boost_build.log 2>&1 && ./b2 -j{os.cpu_count()} --with-filesystem --with-process --with-a
        ! rm -rf /tmp/vacask && git clone -q --filter=blob:none --no-checkout https://codeberg.org/arpadbuermen/VACASK.git /tmp/vacask && git -C /tmp/vacask checkou
        ! cmake -G Ninja -S /tmp/vacask -B /tmp/vacask/build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CXX_FLAGS="-march=x86-64-v2 -DBOOST_PROCESS_V2_DISABLE_PIDFD_OPEN" -
        ! cmake --build /tmp/vacask/build -j{os.cpu_count()} >> /tmp/vacask_build.log 2>&1 && cmake --install /tmp/vacask/build --prefix {VACASK_DIR} --strip >> /tm
        ! rm -rf /tmp/boost_1_88_0 /tmp/boost_1_88_0.tar.gz /tmp/vacask
        # the post-processing scripts import rawfile, which VACASK ships as vacask.rawfile, the alias IIC-OSIC-TOOLS adds
        open(f"{VACASK_DIR}/lib/vacask/python/rawfile.py", "w").write("import sys\nimport vacask.rawfile\nsys.modules[__name__] = vacask.rawfile\n")
    else:
        # the layers are fetched from Docker Hub as `docker pull` would, and unpacked to the paths of the image
        token = json.load(urllib.request.urlopen(f"https://auth.docker.io/token?service=registry.docker.io&scope=repository:{IIC_IMAGE}:pull"))["token"]
        for name, digest in IIC_LAYERS.items():
            request = urllib.request.Request(f"https://registry-1.docker.io/v2/{IIC_IMAGE}/blobs/{digest}", headers={"Authorization": f"Bearer {token}"})
            layer = urllib.request.urlopen(request).read()
            assert "sha256:" + hashlib.sha256(layer).hexdigest() == digest, f"the {name} layer does not match its digest"
            open(f"/tmp/{name}.tar.gz", "wb").write(layer)
            ! tar xzf /tmp/{name}.tar.gz -C / && rm /tmp/{name}.tar.gz
        # the image build links against KLU, OpenBLAS, FFTW, and LLVM 18, which a plain Ubuntu 24.04 runtime lacks
        ! apt-get update -qq > /dev/null 2>&1; apt-get install -y -qq libklu2 libopenblas0 libfftw3-double3 libllvm18 > /dev/null

    # IIC-OSIC-TOOLS adapts the converter to the PDK before it runs it: model files and Verilog-A models of the PDK that the converter does not list
    converter = f"{VACASK_DIR}/lib/vacask/python/vacask/sg13g2tovc.py"
    for fix in IIC_FIXES:
        ! wget -qO /tmp/{fix} https://raw.githubusercontent.com/iic-jku/IIC-OSIC-TOOLS/{IIC_COMMIT}/_build/images/open_pdks/scripts/{fix} && python3 /tmp/{fix} {con
    assert "fix_sg13g2_vacask_va.py" in open(converter).read(), "the converter fixes of IIC-OSIC-TOOLS were not applied"
    # the VACASK model libraries of the PDK are generated from its ngspice models, and its Verilog-A models are compiled to OSDI
    ! PYTHONPATH={VACASK_DIR}/lib/vacask/python OPENVAF_DIR={OPENVAF_DIR}/bin python3 -m vacask.sg13g2tovc --openvaf-options --target_cpu generic > /tmp/sg13g2tovc.
    assert os.path.exists(f"{PDK_ROOT}/ihp-sg13g2/libs.tech/vacask/osdi/psp103.osdi"), "the PDK models were not generated, see /tmp/sg13g2tovc.log"
    ! cp {VACASK_DIR}/share/doc/vacask/demo/ihp-sg13g2/.vacaskrc.toml ~/

    # testbench netlists, fitted passive models, post-layout extraction, EM results, and post-processing scripts
    ! rm -rf {SPARX_DIR} && git clone -q --depth 1 --filter=blob:none --sparse -b {SPARX_BRANCH} https://github.com/iic-jku/SG13CMOS_SPARX.git {SPARX_DIR} > /tmp/sp
    ! git -C {SPARX_DIR} sparse-checkout set testbenches/xschem netlist scripts verification/em/s-parameter >> /tmp/sparx_checkout.log 2>&1 || cat /tmp/sparx_checko
    assert os.path.isdir(f"{XSCHEM_DIR}/netlists"), "the testbenches were not fetched from the SPARX repository"
    ! mkdir -p {SIM_DIR} && cp {XSCHEM_DIR}/netlists/* {SIM_DIR}/
    print(f"VACASK installed in {(time.time() - start) / 60:.1f} min")

# the post-processing scripts read the VACASK output with the rawfile module shipped with VACASK
os.environ["PATH"]       = f"{VACASK_DIR}/bin:{os.environ['PATH']}"
os.environ["PYTHONPATH"] = f"{VACASK_DIR}/lib/vacask/python"
version = subprocess.run(["vacask", "--version"], capture_output=True, text=True)
version = version.stdout + version.stderr
assert "This is vacask" in version, f"VACASK does not start:\n{version}"
print(version.splitlines()[0])
```

```
def run_testbench(tb, variant=""):
    """Simulate a testbench with VACASK and post-process it, as `make sim-xschem` does in the SPARX repository."""
    global RUN_START
    sim_dir = f"{SIM_DIR}/{variant}" if variant else SIM_DIR
    start = RUN_START = time.time()
    with open(f"{sim_dir}/{tb}.log", "w") as log:
        vacask = subprocess.run(["vacask", "-qp", "-sp", f"{tb}.spectre"], cwd=sim_dir, stdin=subprocess.DEVNULL, stdout=log, stderr=subprocess.STDOUT)
    print(f"{tb}: VACASK finished in {(time.time() - start) / 60:.1f} min")
    # VACASK exits with 0 even when an analysis aborts or is missing from the build, so its log is checked as well
    log = open(f"{sim_dir}/{tb}.log").read()
    if vacask.returncode or re.search("aborted|failed|not found", log, re.IGNORECASE):
        raise RuntimeError(f"VACASK failed, see {sim_dir}/{tb}.log:\n{log[-3000:]}")
    plot = subprocess.run(["python3", f"{PLOT_DIR}/plot_{tb}.py"], cwd=SPARX_DIR, env={**os.environ, "POWDET_VARIANT": variant}, capture_output=True, text=True)
    print(plot.stdout)
    if plot.returncode:
        raise RuntimeError(f"the post-processing of {tb} failed:\n{plot.stderr[-3000:]}")


def show_figure(name):
    """Display a figure of the post-processing, but only if the last run_testbench() has written it."""
    path = f"{PLOT_DIR}/figures/{name}"
    # the repository holds a committed copy of every figure, which must not pass for a new result
    assert os.path.getmtime(path) >= RUN_START, f"{name} was not written by this simulation"
    display(Image(filename=path, width=1024))
```

```
VACASK installed in 1.0 min
This is vacask fb8cc20.
```

5.2 From EM to Circuit: Lumped-Element Model of the Core

A circuit simulator cannot use the S-parameters of an EM simulation directly in harmonic balance or transient analyses. snp2le [14] fits the 7-port S-parameters of the six-port core, the BPF, the WPD, and the three BLCs in one EM solve, with a rational model of order up to 24 by vector fitting [21], enforces passivity [22], and writes the model as a VACASK subcircuit of resistors, capacitors, and controlled sources. Both receiver testbenches below include this subcircuit.

Feeding EM results back into circuit simulation has been a gap in open-source RF flows, where EM-verified passives are often approximated by hand-tuned lumped elements [1]. The core can be modeled either by one fit of the complete 7-port result, as here, or by a composition of block fits of order 13 for the BPF, 10 for the WPD, and 6 for each BLC. The composition converges better and simulates faster, but it misses the coupling between adjacent couplers and the interconnects, which is why the receiver uses the single fit.

Thermal noise by Bosma's theorem. A lossy passive is a noise source, and at a uniform temperature its noise follows from its S-parameters alone. Every port k of a noisy N-port emits a noise wave c_k on top of the scattered signal, $\mathbf{b} = \mathbf{S}\,\mathbf{a} + \mathbf{c}$, with $\mathbf{a}$ and $\mathbf{b}$ the incident and outgoing power waves. Bosma's theorem [23] gives the correlation matrix of these noise waves for a linear passive N-port in thermal equilibrium at the temperature T,

$$\mathbf{C}_\mathrm{S} = kT\,(\mathbf{I} - \mathbf{S}\mathbf{S}^\mathrm{H}),$$

with the Boltzmann constant k, the identity matrix $\mathbf{I}$, and the conjugate transpose $\mathbf{S}^\mathrm{H}$ of the S-matrix. The diagonal entry $(\mathbf{C}_\mathrm{S})_{kk} = kT\,(1 - \sum_j |S_{kj}|^2)$ is the noise power per hertz that port k sends into a matched load. In a reciprocal network such as this core, it is kT times the share of the power entering port k that the network dissipates, so a lossless network is noiseless and a matched resistor emits the full kT. The off-diagonal entries correlate the noise of ports that see the same lossy elements. As a covariance, $\mathbf{C}_\mathrm{S}$ has to be positive semidefinite, which holds exactly when the largest singular value of $\mathbf{S}$ is at most one, so passivity and physical noise are the same condition. For the EM result of the core at 160 GHz, the formula gives 0.69 kT at the LO port, whose path runs through the BPF, the WPD, and a coupler, 0.29 kT at the RF port, and 0.45 kT to 0.55 kT at the four outputs.

The resistors of a vector fit place its poles and do not model the loss of the structure, so snp2le emits them noiseless (`noisy=0`). With `--thermal-noise`, as in `make sparx-core` of the repository, it factors $\mathbf{I} - \mathbf{S}\mathbf{S}^\mathrm{H}$ of the fitted model into a noise generator, driven by one white source of density kT per port, and appends it to the subcircuit without touching the signal network. VACASK sees this noise in its noise, HBNOISE, PNOISE, and transient-noise analyses. The documentation derives the theorem in more detail.

The fit uses `sparx160_core.s7p`, the EM result without de-embedding, as the repository does, and the new fit replaces `netlist/spectre/sparx_core_le.inc` in the checkout. The result line reports the RMS error of the fit against the EM data, the model order the fit reached and its number of poles, the largest singular value of the S-matrix (`sigma_max`, at most 1 for a passive model), and whether the dc operating point of the netlist can be solved.

```
RUN_SNP2LE = 1  # 1 = run this cell, 0 = skip it (the receiver then uses the fit stored in the repository)

if RUN_SNP2LE:
    %pip install -q --progress-bar off snp2le==0.1.12 scikit-rf==2.1.0
    start = time.time()
    ! cd {SPARX_DIR} && QT_QPA_PLATFORM=offscreen snp2le -b convert verification/em/s-parameter/sparx160_core.s7p --mode universal --order 24 --format vacask --ther
    print(f"snp2le finished in {(time.time() - start) / 60:.1f} min")
    ! head -12 {SPARX_DIR}/netlist/spectre/sparx_core_le.inc
```

```
[ OK ] verification/em/s-parameter/sparx160_core.s7p -> netlist/spectre/sparx_core_le.inc  (vacask, rms=3.86e-03  order=23 (12 poles)  sigma_max=1.000  dc=solvable
       note: passivity enforced (n_samples=200, rms=3.86e-03)
snp2le finished in 0.1 min
// sparx_core_le - lumped-element equivalent (VACASK / Spectre), generated by snp2le
// EQUIVALENT CIRCUIT FOR VECTOR FITTED S-MATRIX
// Created using scikit-rf vectorFitting.py
//
//
// thermal noise of the passive: kT(I - S S^H) at the ports, from a spectral factor of this fit
// sources Rnz_e1..Rnz_e7 (0.25 ohm, the only noisy elements), 161 states nz_x*, controlled sources Gnz_*
// device models needed (declared by your testbench): capacitor, cccs, resistor, vccs, vsource
// NOTE: ground is 'GND'. Your testbench must declare it as the ground (Xschem's spectre netlist does this with `ground GND`), otherwise the circuit floats and the r
subckt sparx_core_le (p1 p2 p3 p4 p5 p6 p7)
  V1 (p1 s1) vsource dc=0
  R1 (s1 GND) resistor r=50 noisy=0
```

snp2le Result:

The fit reaches an RMS error of 3.9e-3 at order 23, which counts a real pole once and a complex pole pair twice. With one real pole and eleven complex pairs, snp2le prints this as 12 poles. The model is passive (`sigma_max=1.000`), its dc operating point is solvable, it carries the thermal noise of the core (`noise=thermal`), and it reproduces the fit committed in the repository. Resimulated in VACASK with the repository testbench `sparx_core_le_tb_acsp_vacask`, it matches all 49 EM S-parameters at 160 GHz within 0.45 dB and 9.9°. Both extremes fall on one output-to-output coupling at -35 dB, and every other S-parameter is within 0.36 dB and 5°, see section 1 of the testbench README.

5.3 Power Detector: Transfer Curve and Input Impedance

At the tapeout in March 2026, the detector was checked with a two-tone HB sweep in VACASK, which confirmed square-law detection, an IF output proportional to the product of the RF and the LO amplitude, and nothing more. The analyses that VACASK gained later in 2026 characterize the detector as a receiver front-end in this and the next two sections.

The detector is driven by a single tone at 161 GHz from a 50 Ω source, swept from -76 dBm to +6.9 dBm of available power with HB at 15 harmonics. HB solves for the periodic steady state in the frequency domain, where a transient simulation would have to integrate thousands of carrier periods until the detected output settles. Nine shooting PSS runs, one every 10 dB from -76 dBm to +4 dBm, solve the same steady state in the time domain. The two algorithms share no code, so their agreement is an independent check of the transfer curve.

The figure shows, from top to bottom, the detected output voltage against the available input power with the square-law line and the 1 dB compression point, the responsivity, and the large-signal input impedance at 161 GHz.

> **Runtime:** about 1 minute on Colab.

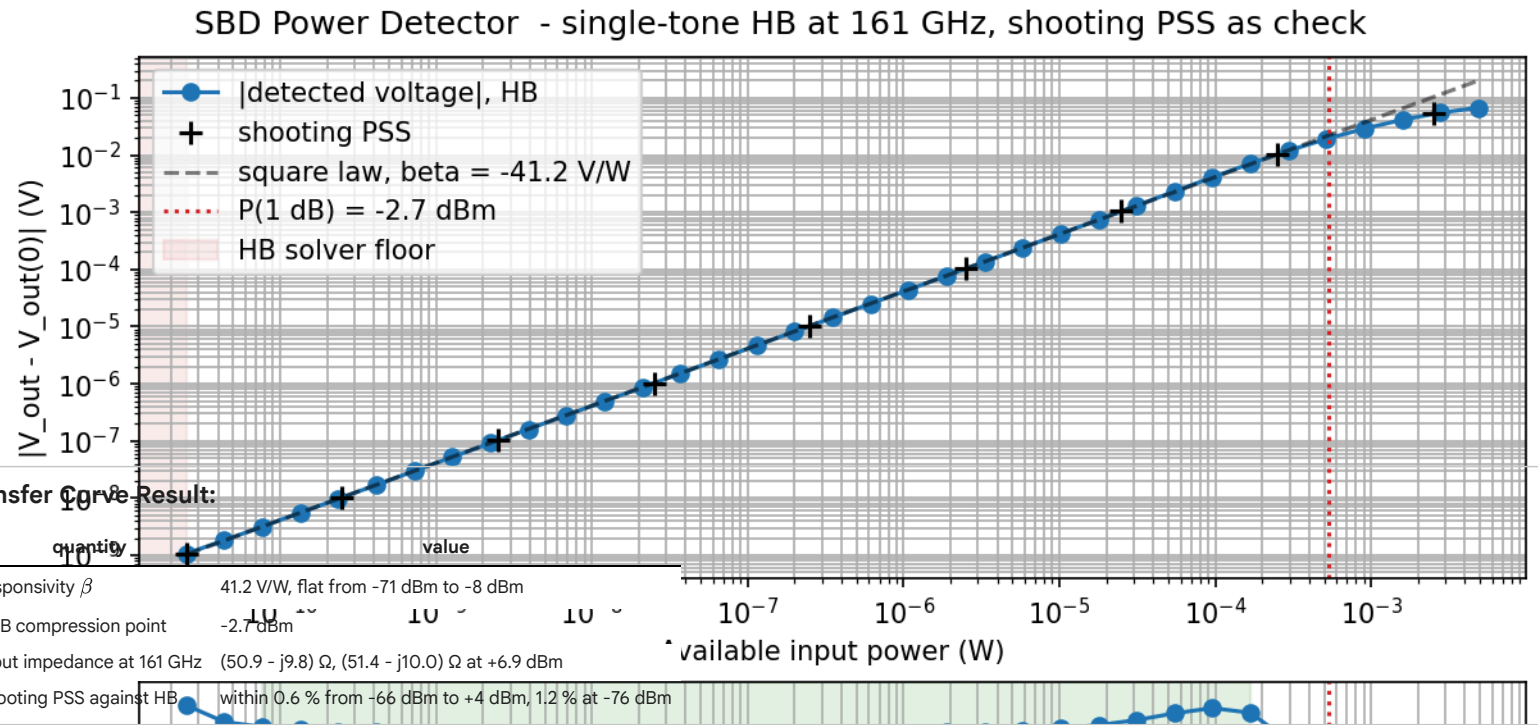
```
RUN_PD_PSS = 1  # 1 = run this simulation, 0 = skip it

if RUN_PD_PSS:
    run_testbench("sparx_powdet_sbd_tb_pss_vacask")
    show_figure("sparx_powdet_sbd_pss_sweep.png")
```



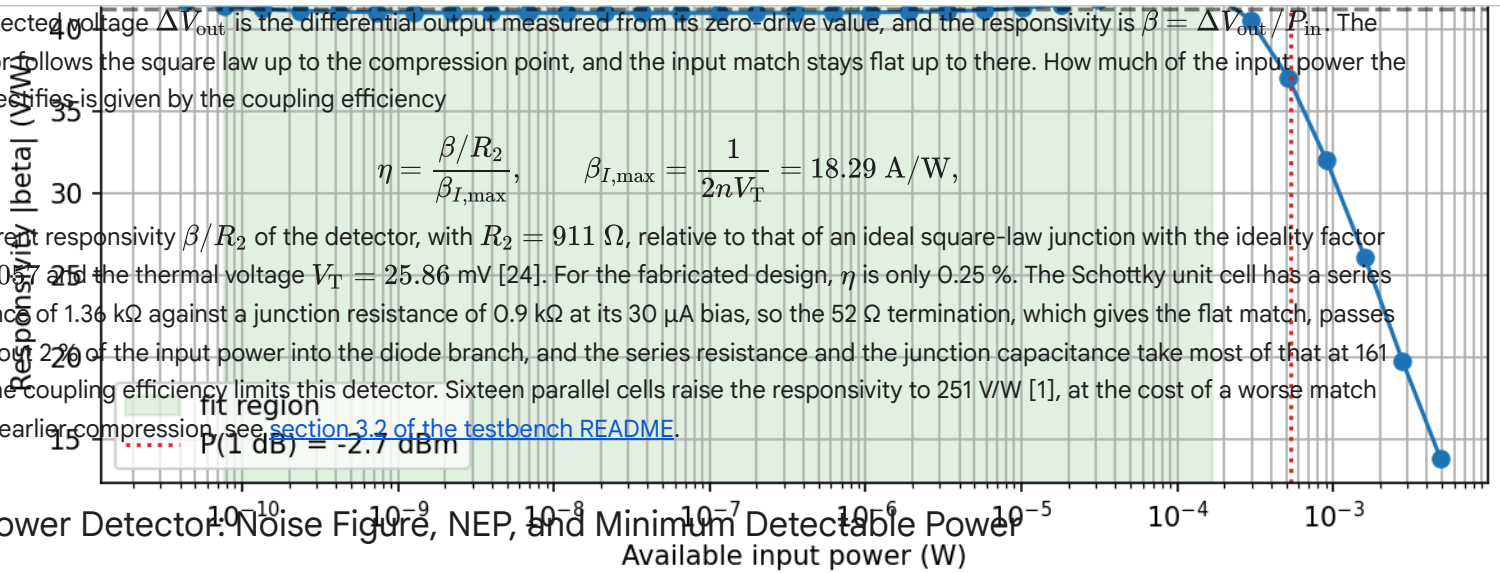
```
Variant               : as fabricated
RF frequency          : 161.0 GHz
Source resistance     : 50 Ohm
Output offset V_dc(0) : -7.0327 mV  (fitted; OP analysis says -7.0332 mV, HB DC bin sits +548.2 nV from it)
Responsivity beta     : -41.22 V/W  (fit over -71.1 .. -7.7 dBm)
1 dB compression      : -2.74 dBm
HB solver floor       : -76.0 dBm (points below this scatter off the square-law line)
Solver floor to P1dB  : 73.3 dB
Z_in at 161 GHz     : 50.9 -9.8j Ohm (small signal), 51.4 -10.0j Ohm at 6.9 dBm

Shooting PSS against HB, detected voltage at equal source amplitude:
  P_avail           HB          PSS   PSS/HB
   -76.0dBm   -0.00000mV   -0.00000mV   0.9884  (below the resolved range)
   -66.0dBm   -0.00001mV   -0.00001mV   0.9995  (below the resolved range)
   -56.0dBm   -0.00010mV   -0.00010mV   0.9996
   -46.0dBm   -0.00102mV   -0.00102mV   0.9996
   -36.0dBm   -0.01024mV   -0.01024mV   0.9996
   -26.0dBm   -0.10259mV   -0.10255mV   0.9996
   -16.0dBm   -1.04161mV   -1.04110mV   0.9995
    -6.0dBm  -10.25365mV  -10.29166mV   1.0037
     4.0dBm  -52.15749mV  -52.45610mV   1.0057
  largest deviation over the resolved range: 0.57 %
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_beta.json
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_pss.csv
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_pss_check.csv
```



Transfer Curve Result:

quantity	value
responsivity β	41.2 V/W, flat from -71 dBm to -8 dBm
1 dB compression point	-2.7 dBm
input impedance at 161 GHz	(50.9 - j9.8) Ω, (51.4 - j10.0) Ω at +6.9 dBm
shooting PSS against HB	within 0.6 % from -66 dBm to +4 dBm, 1.2 % at -76 dBm

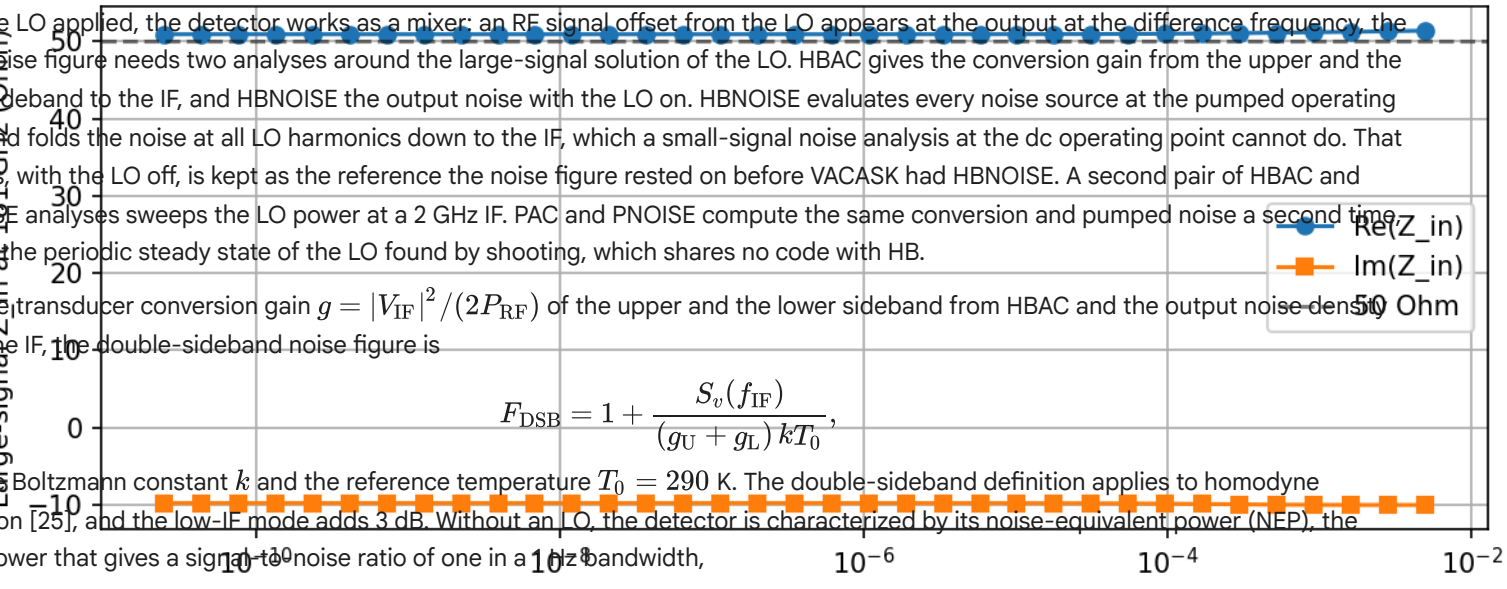


The detected voltage ΔV_{out} is the differential output measured from its zero-drive value, and the responsivity is $\beta = \Delta V_{\mathrm{out}}/P_{\mathrm{in}}$. The detector follows the square law up to the compression point, and the input match stays flat up to there. How much of the input power the diode rectifies is given by the coupling efficiency

$$\eta = \frac{\beta/R_2}{\beta_{I,\max}}, \qquad \beta_{I,\max} = \frac{1}{2nV_{\mathrm{T}}} = 18.29\ \mathrm{A/W},$$

the current responsivity β/R_2 of the detector, with $R_2 = 911\ \Omega$, relative to that of an ideal square-law junction with the ideality factor $n = 1.057$ and the thermal voltage $V_{\mathrm{T}} = 25.86$ mV [24]. For the fabricated design, η is only 0.25 %. The Schottky unit cell has a series resistance of 1.36 kΩ against a junction resistance of 0.9 kΩ at its 30 µA bias, so the 52 Ω termination, which gives the flat match, passes only about 2 % of the input power into the diode branch, and the series resistance and the junction capacitance take most of that at 161 GHz. The coupling efficiency limits this detector. Sixteen parallel cells raise the responsivity to 251 V/W [1], at the cost of a worse match and an earlier compression, see section 3.2 of the testbench README.

5.4 Power Detector: Noise Figure, NEP, and Minimum Detectable Power



With the LO applied, the detector works as a mixer: an RF signal offset from the LO appears at the output at the difference frequency, the IF. Its noise figure needs two analyses around the large-signal solution of the LO. HBAC gives the conversion gain from the upper and the lower sideband to the IF, and HBNOISE the output noise with the LO on. HBNOISE evaluates every noise source at the pumped operating point and folds the noise at all LO harmonics down to the IF, which a small-signal noise analysis at the dc operating point cannot do. That analysis, with the LO off, is kept as the reference the noise figure rested on before VACASK had HBNOISE. A second pair of HBAC and HBNOISE analyses sweeps the LO power at a 2 GHz IF. PAC and PNOISE compute the same conversion and pumped noise a second time around the periodic steady state of the LO found by shooting, which shares no code with HB.

With the transducer conversion gain $g = |V_{\mathrm{IF}}|^2/(2P_{\mathrm{RF}})$ of the upper and the lower sideband from HBAC and the output noise density S_v at the IF, the double-sideband noise figure is

$$F_{\mathrm{DSB}} = 1 + \frac{S_v(f_{\mathrm{IF}})}{(g_{\mathrm{U}} + g_{\mathrm{L}})\,kT_0},$$

with the Boltzmann constant k and the reference temperature $T_0 = 290$ K. The double-sideband definition applies to homodyne operation [25], and the low-IF mode adds 3 dB. Without an LO, the detector is characterized by its noise-equivalent power (NEP), the input power that gives a signal-to-noise ratio of one in a 1 Hz bandwidth,

$$\mathrm{NEP}(f) = \frac{\sqrt{S_v(f)}}{\beta\,|H(f)|},$$

with the responsivity β of Section 5.3 and the normalized video response $|H(f)|$ from HBAC [24].

The figure shows the output noise density with its largest contributors (top left), the conversion of both sidebands to the IF (top right), and the double-sideband noise figure against the IF (bottom left) and against the LO power at a 2 GHz IF (bottom right). The open circles are the PAC and PNOISE results.

> **Runtime:** about 1 minute on Colab.

```
RUN_PD_NF = 1  # 1 = run this simulation, 0 = skip it

if RUN_PD_NF:
    run_testbench("sparx_powdet_sbd_tb_nf_vacask")
    show_figure("sparx_powdet_sbd_nf.png")
```



```
sparx_powdet_sbd_tb_nf_vacask: VACASK finished in 1.7 min
Variant               : as fabricated
LO                    : 159.0 GHz at -6.5 dBm available
Responsivity beta     : 41.22 V/W  (small signal, from the PSS testbench)
Conversion g at LO    : 0.6695 V^2/W at low IF from hbac, closed form 2 S'^2 P_lo = 0.6502 V^2/W (S' = 38.01 V/W), hbac - closed form = +0.13 dB
hbnoise gain vs hbac  : within 0.0000 dB over the IF sweep, 0.0048 dB over the LO sweep
IF output at -46.5 dBm RF: 173.6 uV at low IF, 61.2 uV at 2 GHz
Video bandwidth       : 1.24 GHz (-3 dB of the conversion, interpolated)
Sideband imbalance    : g_L/g_U = +0.02 dB at 2 GHz
Video band 1e+03 .. 5e+09 Hz :  1305.6 uV RMS, MDS  -15.0 dBm  (without PCell PNP:  -23.7 dBm)
Video band 1e+06 .. 5e+09 Hz :   905.3 uV RMS, MDS  -16.6 dBm  (without PCell PNP:  -23.9 dBm)

Output noise, pumped (hbnoise, LO on) against the small-signal noise at the dc operating
point (noise, LO off), and the noise figure from each. The pumped column is the result,
the other one is what the bench reported before hbnoise existed.
      f_IF  ASD pumped  ASD quiesc    p/q   NF_DSB  NF_DSB q   NF_SSB NF no PNP   beta(f)         NEP  NEP no PNP
      [Hz]    [V/rtHz]    [V/rtHz]            [dB]      [dB]     [dB]      [dB]     [V/W]    [W/rtHz]    [W/rtHz]
 1.000e+03   1.379e-05   1.131e-05   1.49   105.50    103.78   108.51     78.23      41.2   2.744e-07   1.493e-08
 9.996e+03   4.361e-06   3.577e-06   1.49    95.50     93.78    98.51     68.23      41.2   8.678e-08   4.724e-09
 9.993e+04   1.379e-06   1.131e-06   1.49    85.50     83.78    88.51     58.28      41.2   2.745e-08   1.501e-09
 9.989e+05   4.363e-07   3.578e-07   1.49    75.50     73.78    78.51     48.68      41.2   8.682e-09   4.958e-10
 9.986e+06   1.380e-07   1.132e-07   1.49    65.52     63.80    68.53     41.41      41.2   2.750e-09   2.119e-10
 9.982e+07   4.262e-08   3.388e-08   1.58    55.11     53.12    58.13     37.69      42.1   8.047e-10   1.353e-10
 5.002e+08   1.812e-08   1.360e-08   1.78    47.49     44.99    50.50     35.05      43.1   3.156e-10   9.762e-11
 9.979e+08   1.077e-08   7.988e-09   1.82    44.68     42.08    47.69     34.90      35.4   2.259e-10   9.387e-11
 1.991e+09   3.359e-09   2.606e-09   1.66    42.20     40.00    45.23     35.21      14.7   1.779e-10   9.872e-11
 5.000e+09   4.596e-10   3.985e-10   1.33    40.28     39.04    43.32     37.07       2.5   1.596e-10   1.258e-10

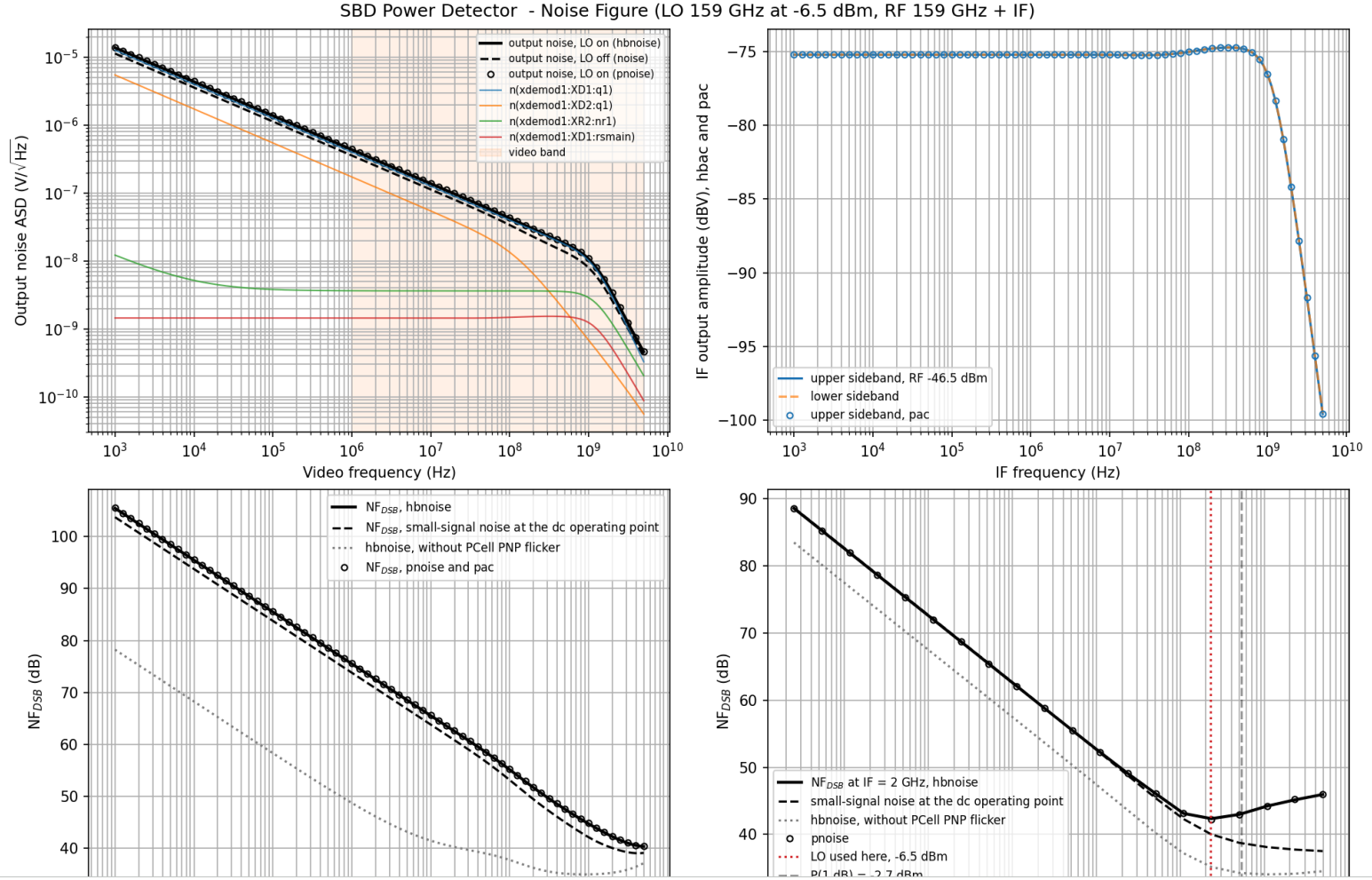
Noise figure against LO drive at 2 GHz IF:
  pumped noise   : best 42.3 dB at -6.3 dBm LO, 42.4 dB at the -6.5 dBm used here
  dc op. noise   : best 37.5 dB at 6.9 dBm LO, 40.1 dB at the -6.5 dBm used here
 P_LO [dBm]  S_int pumped  pumped/quiesc   NF_DSB  NF_DSB q
      -56.0     6.666e-18           0.98    88.58     88.66
      -52.7     6.666e-18           0.98    85.27     85.35
      -49.4     6.666e-18           0.98    81.96     82.04
      -46.1     6.666e-18           0.98    78.65     78.73
      -42.8     6.667e-18           0.98    75.34     75.42
      -39.5     6.669e-18           0.98    72.03     72.11
      -36.2     6.672e-18           0.98    68.72     68.79
      -32.8     6.680e-18           0.98    65.41     65.48
      -29.5     6.695e-18           0.99    62.10     62.16
      -26.2     6.729e-18           0.99    58.80     58.84
      -22.9     6.801e-18           1.00    55.51     55.50
      -19.6     6.956e-18           1.02    52.25     52.14
      -16.3     7.288e-18           1.07    49.06     48.76
      -13.0     7.940e-18           1.17    46.07     45.39
       -9.7     8.231e-18           1.21    43.12     42.28
       -6.3     1.166e-17           1.72    42.34     39.99
       -3.0     1.798e-17           2.65    42.96     38.74
        0.3     2.774e-17           4.09    44.21     38.10
        3.6     3.742e-17           5.51    45.14     37.73
        6.9     4.740e-17           6.98    45.94     37.50

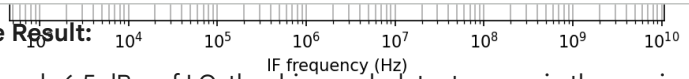
Shooting cross-check, pac against hbac and pnoise against hbnoise:
  pac - hbac conversion  : +0.0050 .. +0.0068 dB, both sidebands over the IF sweep
```

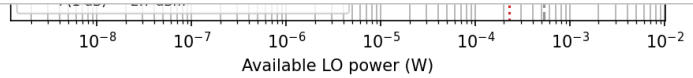
```
  pnoise - hbnoise noise : +0.046 .. +0.095 dB over the IF sweep, +0.078 dB at 2 GHz
  pnoise gain vs pac     : within 5.2e-13 dB
  NF_DSB at 2 GHz        : 42.28 dB from pnoise, 42.20 dB from hbnoise
  against LO drive       : pnoise - HB conversion -0.041 .. +0.011 dB, NF -0.073 .. +0.099 dB, best NF 42.3 dB at -6.3 dBm

Top output-noise contributors over 1e+06 .. 5e+09 Hz, pumped, and their pumped-over-quiescent ratio:
  n(xdemod1:XD1:q1)                              86.70 %   x1.84
  n(xdemod1:XD2:q1)                              10.99 %   x0.76
  n(xdemod1:XR2:nr1)                              1.31 %   x1.05
  n(xdemod1:XD1:rsmain)                           0.24 %   x0.91
  n(xdemod1:XR4:nr1)                              0.20 %   x1.03
  n(xdemod1:XD1:dmain)                            0.19 %   x1.99

Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_nf.json
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_nf.csv
Wrote /tmp/sparx/testbenches/xschem/plot_simulations/data/sparx_powdet_sbd_nf_lo.csv
```







Noise Figure Result:

At a 2 GHz IF and -6.5 dBm of LO, the drive each detector sees in the receiver:

quantity	value
noise figure (DSB), HBNOISE	42.2 dB
noise figure (DSB), PNOISE with PAC	42.3 dB
noise figure (DSB), small-signal noise at the dc operating point	40.0 dB
lowest noise figure against the LO drive, HBNOISE	42.3 dB at -6.3 dBm
output noise with the LO on over the LO off	1.66 (1.5 at low IF, 1.8 near 1 GHz)
video bandwidth	1.24 GHz
NEP	8.7 nW/√Hz at 1 MHz, 226 pW/√Hz at 1 GHz
minimum detectable power, 1 MHz to 5 GHz	-16.6 dBm

- About 98 % of the pumped output noise power between 1 MHz and 5 GHz (97 % of the small-signal noise at the dc operating point) is flicker noise of the parasitic vertical PNP in the PDK model of the Schottky diode, so no device sized in this design sets the noise floor. Without it, the noise figure at 2 GHz would be 35.2 dB and the minimum detectable power -23.9 dBm.
- Since this noise is 1/f across the whole video band, detection at dc, as in a conventional six-port, pays for it. An LO offset to a 1 GHz IF improves the NEP 38-fold.
- The LO raises the output noise. Over the IF, the small-signal noise at the dc operating point is 1.7 dB to 2.6 dB optimistic. Against the LO drive, the gap grows to 2.3 dB at the minimum and 8.4 dB at +6.9 dBm, where the small-signal estimate keeps falling to 37.5 dB while HBNOISE and PNOISE rise from 42.3 dB to 45.9 dB. A mixer noise figure therefore needs a large-signal noise analysis. The conversion is consistent across the analyses, since the gain that HBNOISE reports agrees with HBAC within 0.005 dB.
- PAC agrees with HBAC within 0.007 dB and PNOISE with HBNOISE within 0.1 dB over the whole IF range (42.3 dB against 42.2 dB at 2 GHz), and they also agree within 0.1 dB against the LO drive.
- The dominant noise source is a model extrapolation, since the PNP flicker model is evaluated at about 20 pA of base current, far below any plausible characterization current. Measurements will show which figure applies, see section 3.3 of the testbench README.

5.5 Power Detector: Transient Noise

Transient noise solves the circuit in the time domain with random noise sources and shares no code with HBNOISE, so it checks the pumped noise without any linearization. Three 100 ns transients with the LO on at -6.5 dBm differ only in the noise amplitude (`noisescale` 0.01, 0.0316, and 0.1). The output spectrum of each run is split into a term linear in the noise power and a remainder, and the linear term is compared with HBNOISE, PNOISE, and the small-signal noise at the dc operating point of Section 5.4. The noise amplitude has to stay this small, since at full amplitude, transient noise on this stiff, strongly nonlinear circuit produces a numerical excess that is not physical.